

Information Retrieval

5. Index construction

Ghislain Fourny
Spring 2022

Stock picture 123RF / fotoreactor

5



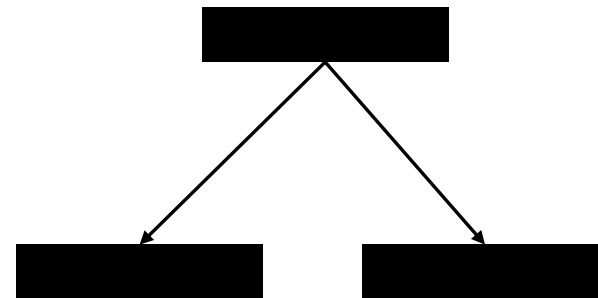
So far...

Standard inverted index

ETH	➡	6	1	2	3	5	6	8
Zürich	➡	5	3	4	7	8	9	
computer	➡	5	1	2	4	5	7	
data	➡	5	1	3	5	8	9	
CPU	➡	4	2	3	4	7		
information	➡	6	1	2	4	5	8	9
retrieval	➡	4	3	5	7	8		

Search structures

Hash tables



Trees (B, B+)

Tolerant retrieval

comput*

Wildcards

cmputer

Spell correction

New indices

**\$ETH
H\$ET
TH\$E
ETH\$**

Permuterm

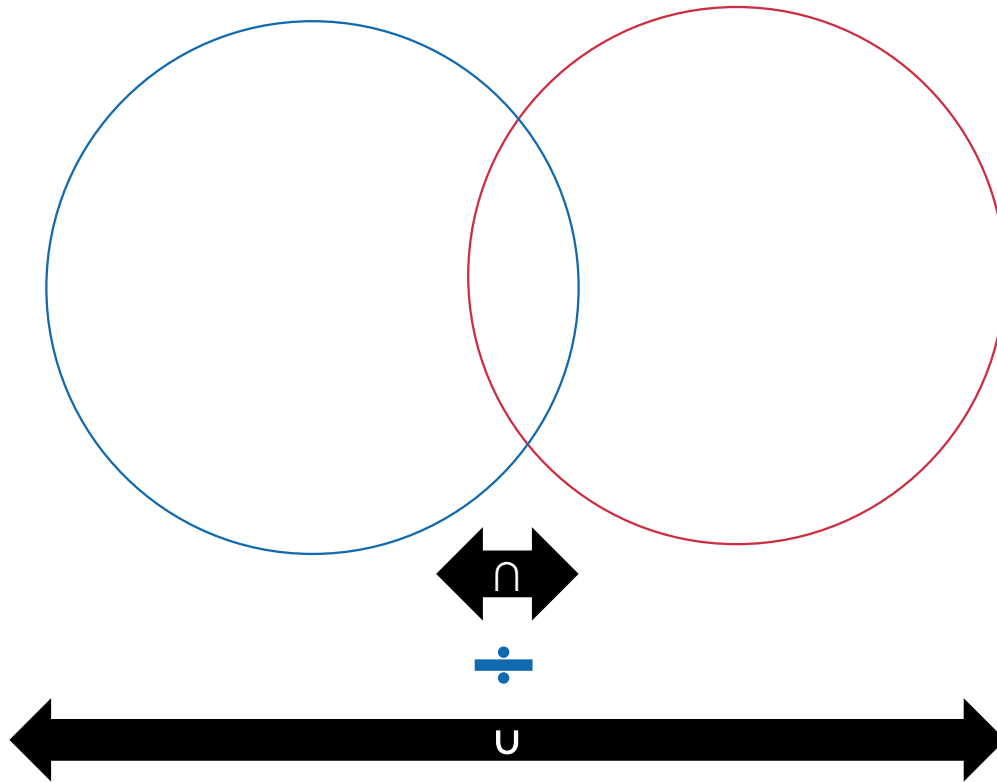
**\$E
ET
TH
H\$**

k-gram

Edit distance

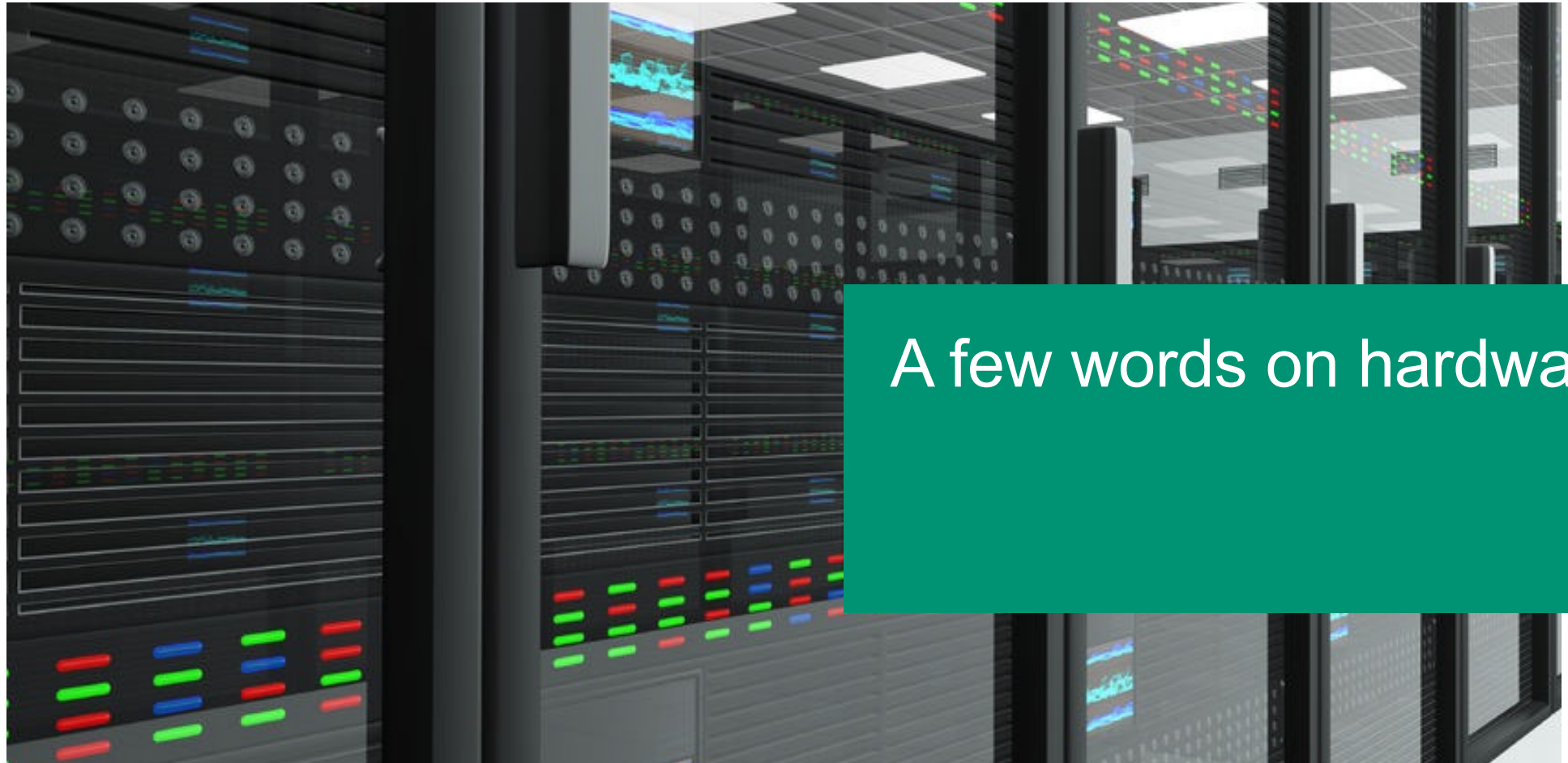
cat → at → ar → are

Jaccard coefficient



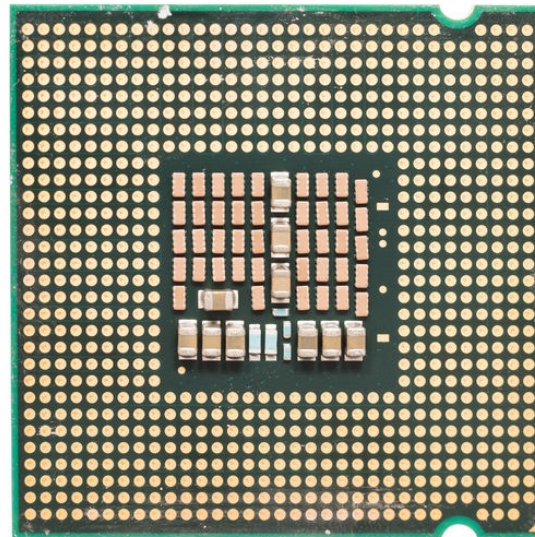
Soundex encoding



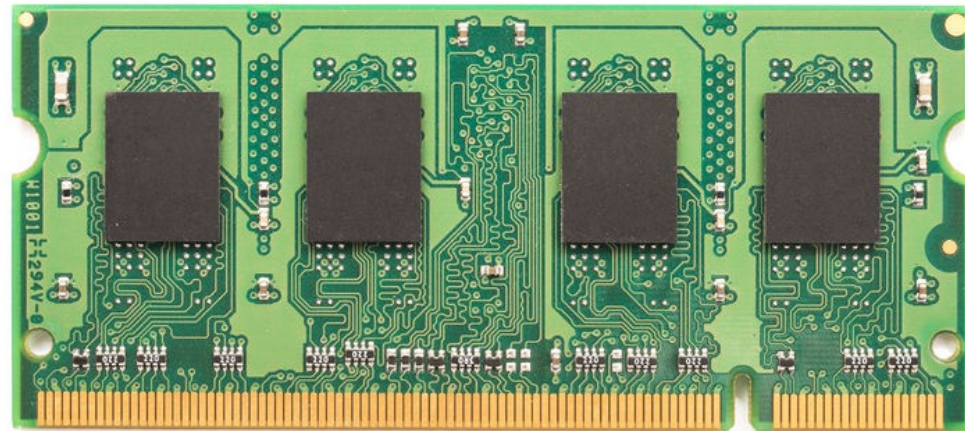


A few words on hardware

CPU



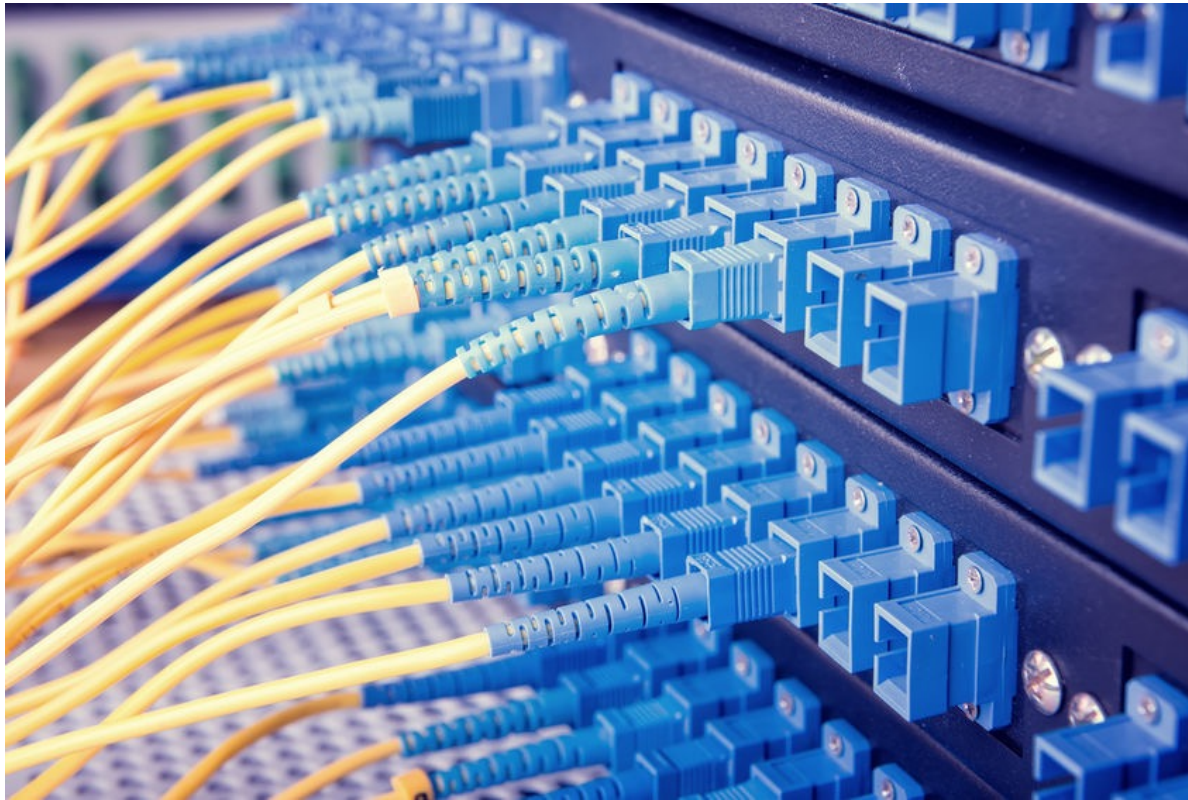
Memory



Disk



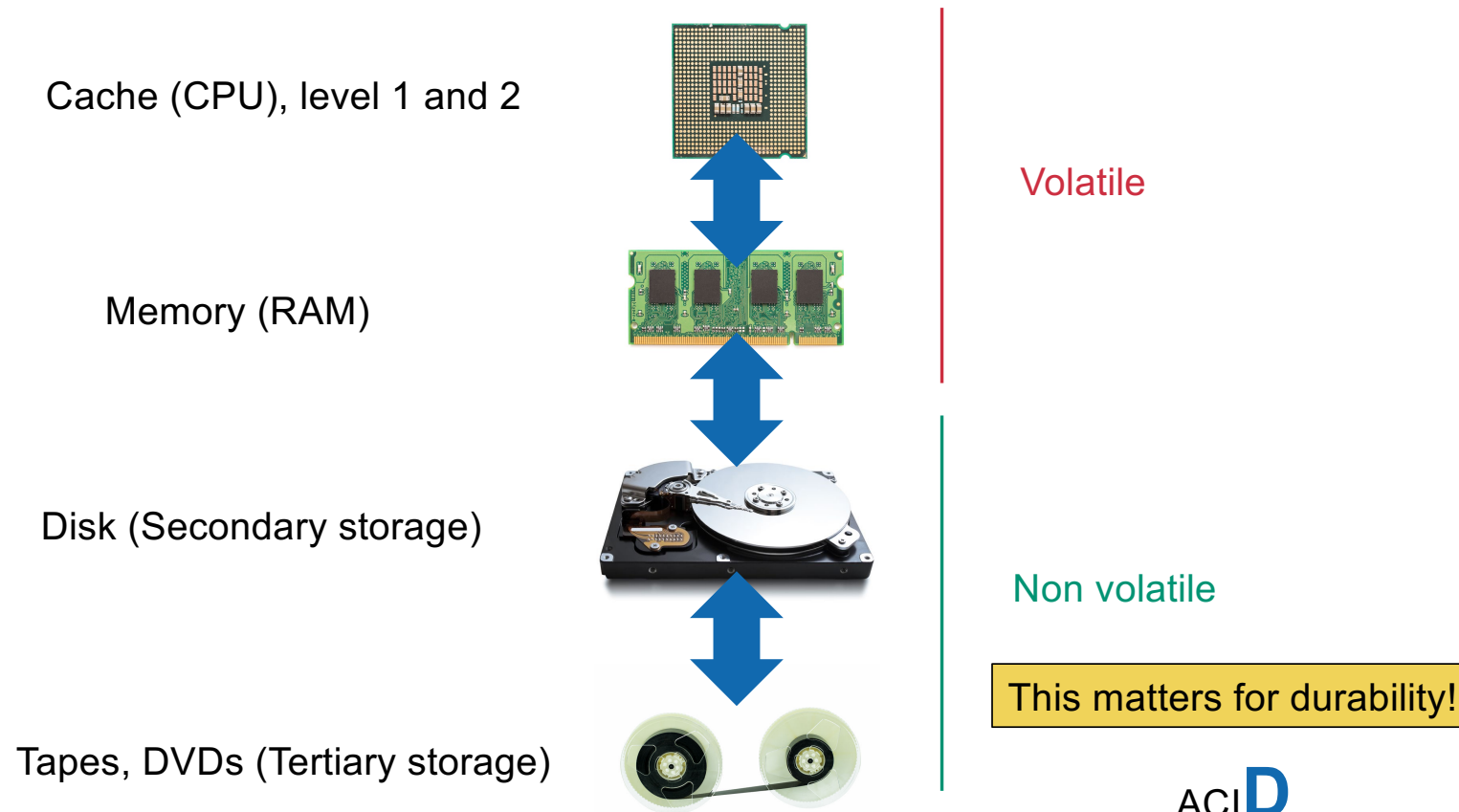
Network



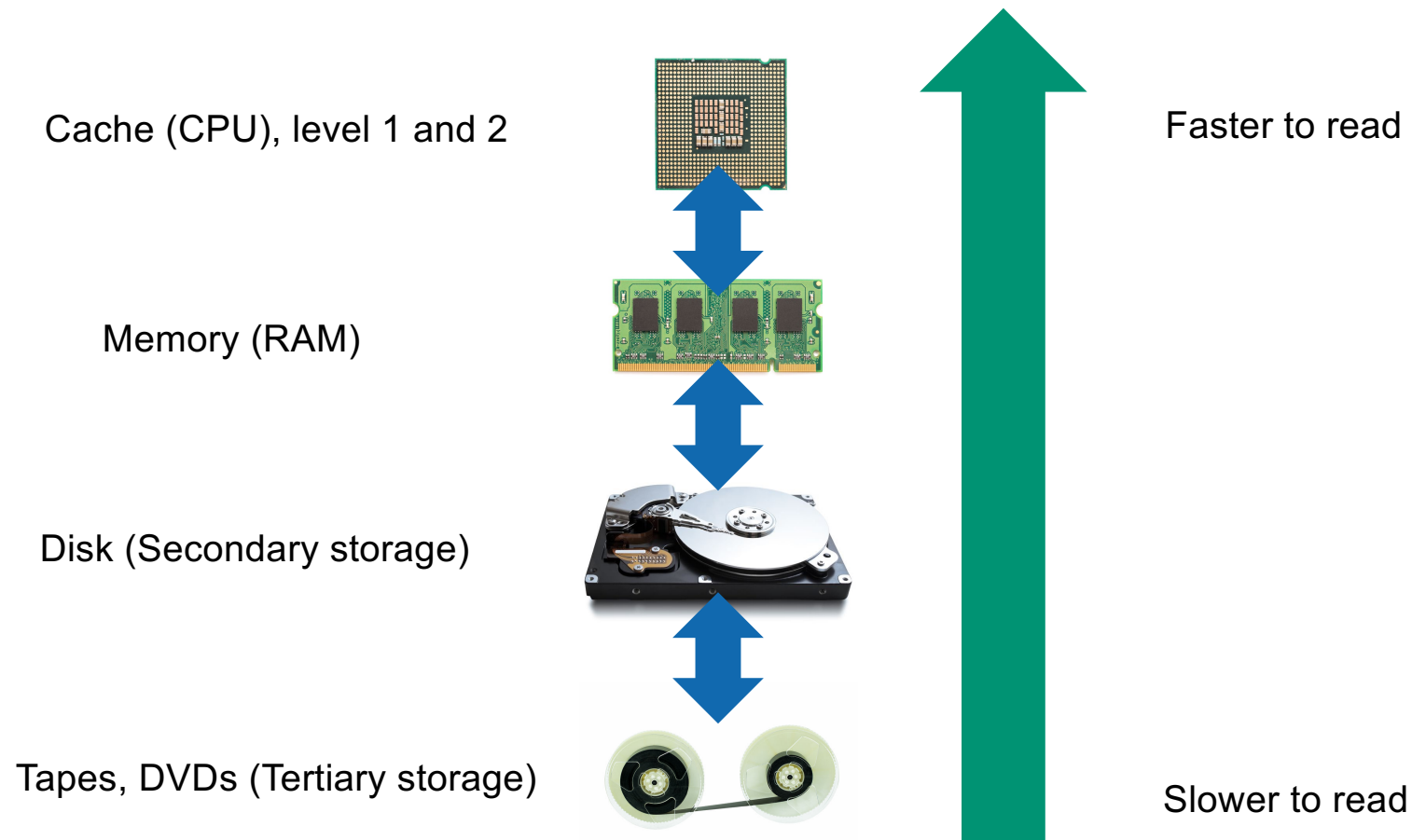
Today: what happens behind the scenes?



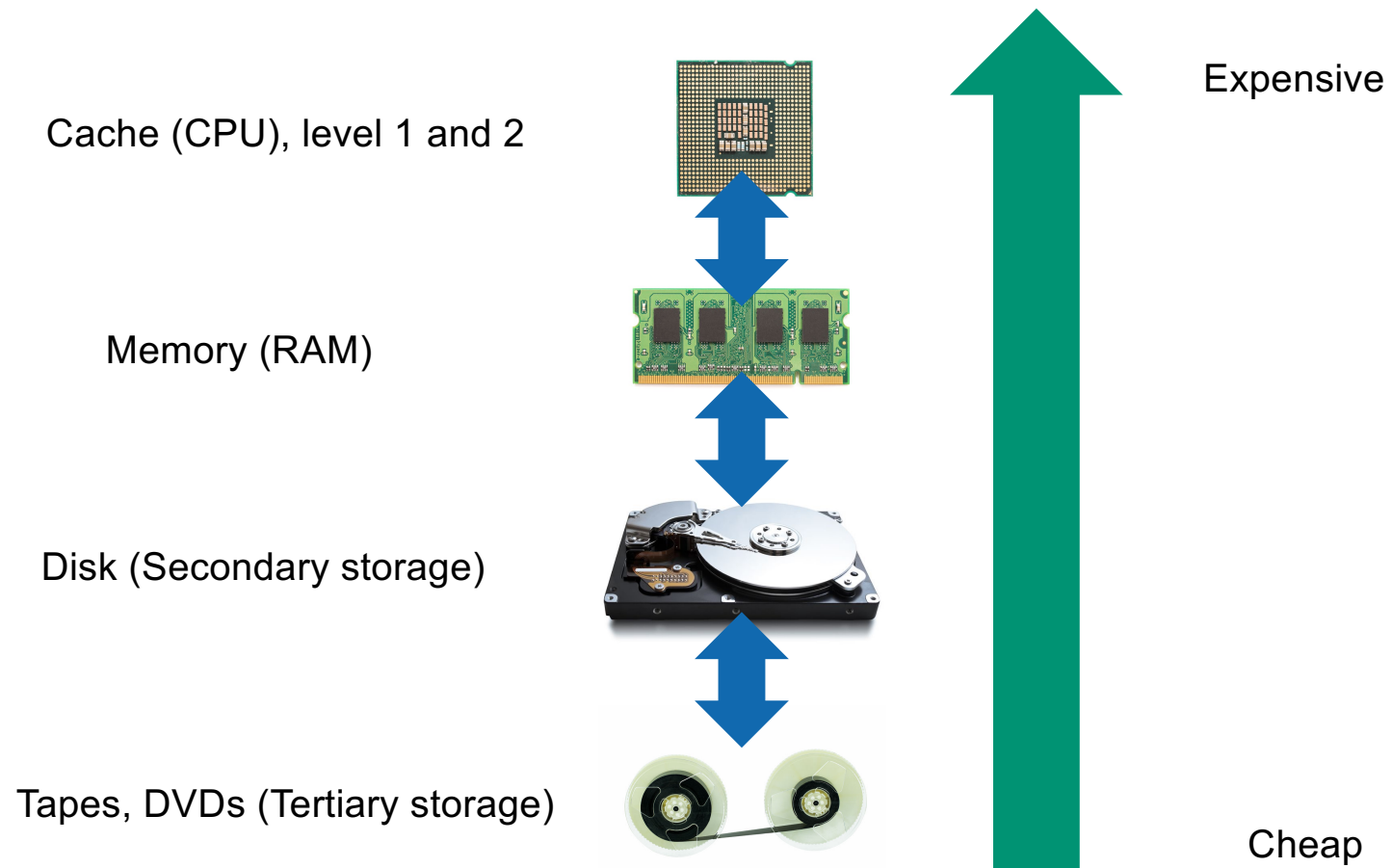
Memory hierarchy



Latency time



Costs



Three paramount factors

Capacity

Throughput

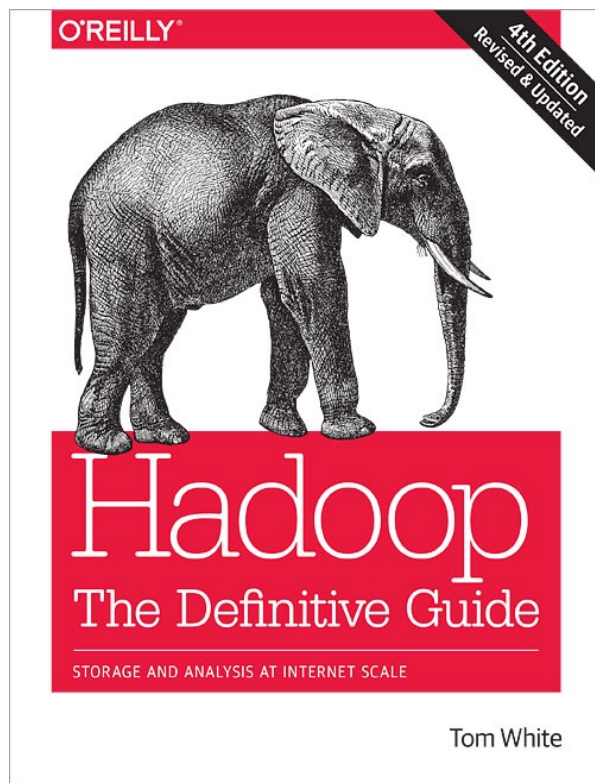
Latency

Capacity



"How much data can we store?"

Capacity: Example



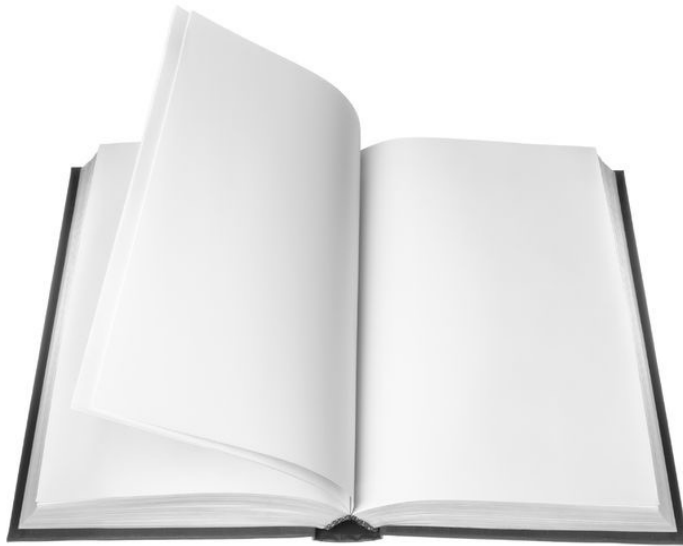
~ 600,000 words.

Throughput



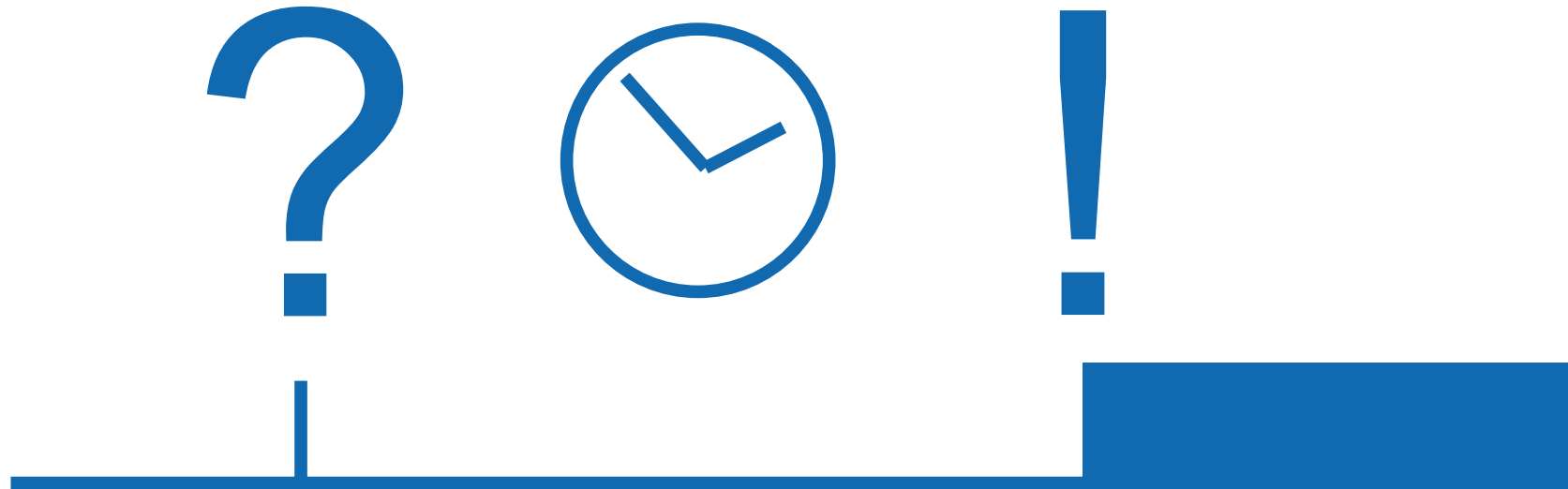
"How fast can we transmit data?"

Throughput: Example



~1,000 word
per minute

Latency



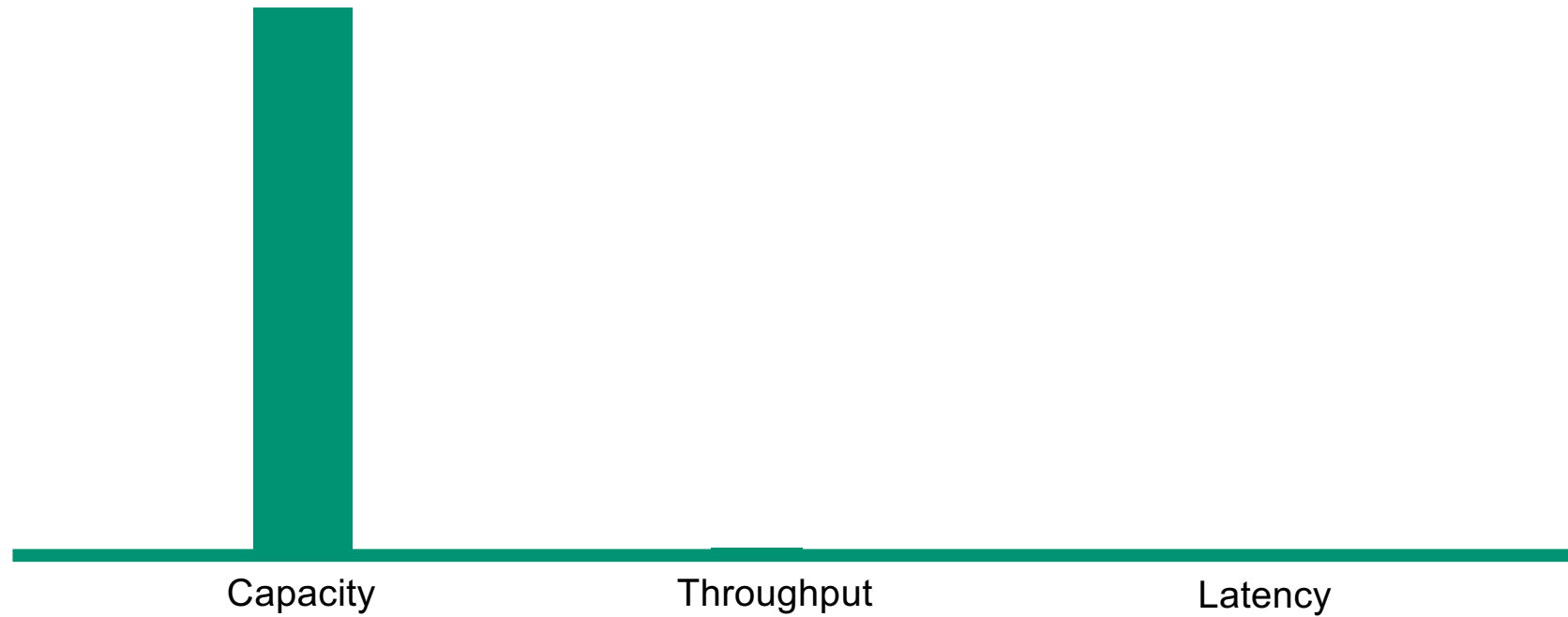
"When do I start receiving data?"

Latency: Example



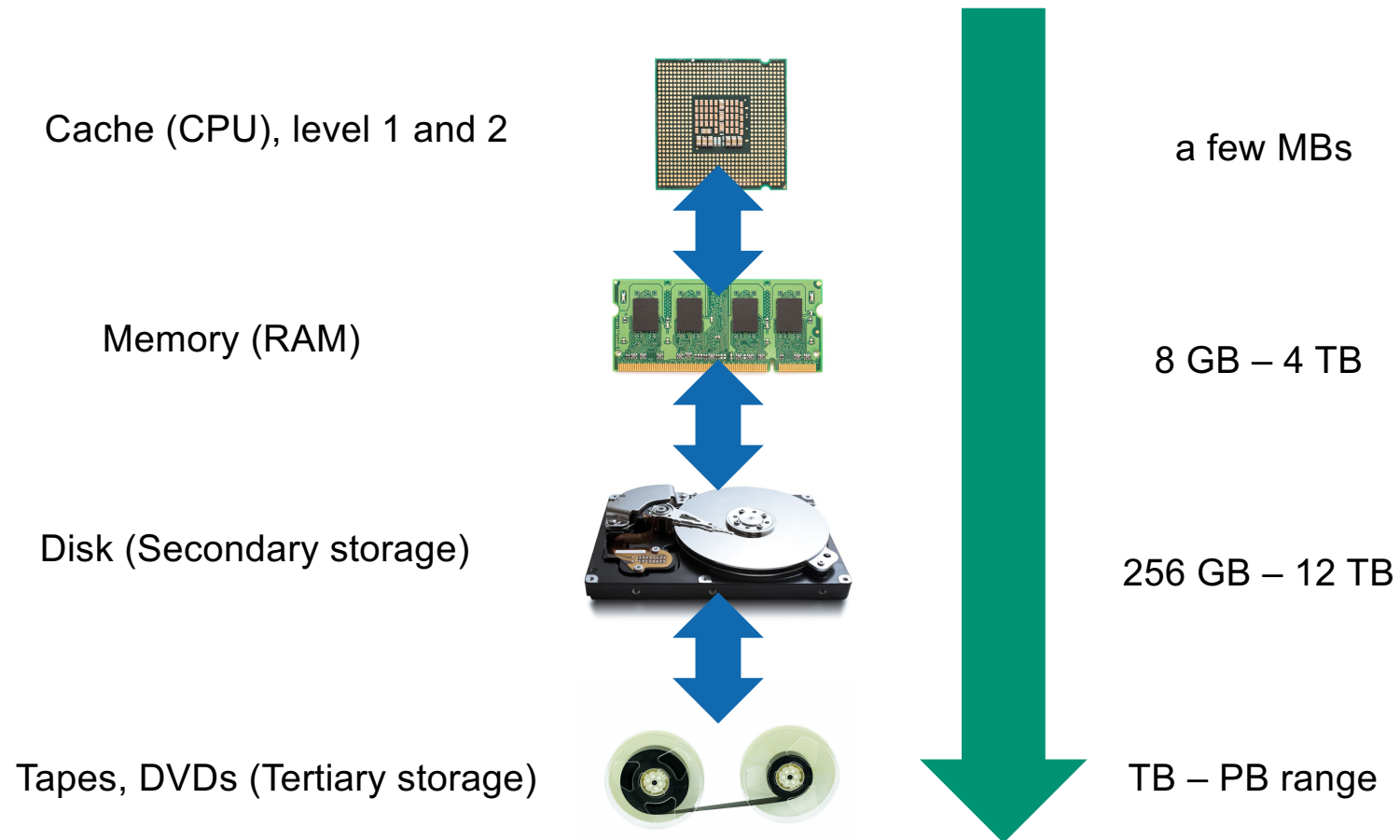
~ 1 minute to
stand up,
go to the shelf,
pick the book,
find the page.

The progress made (1956-2010)

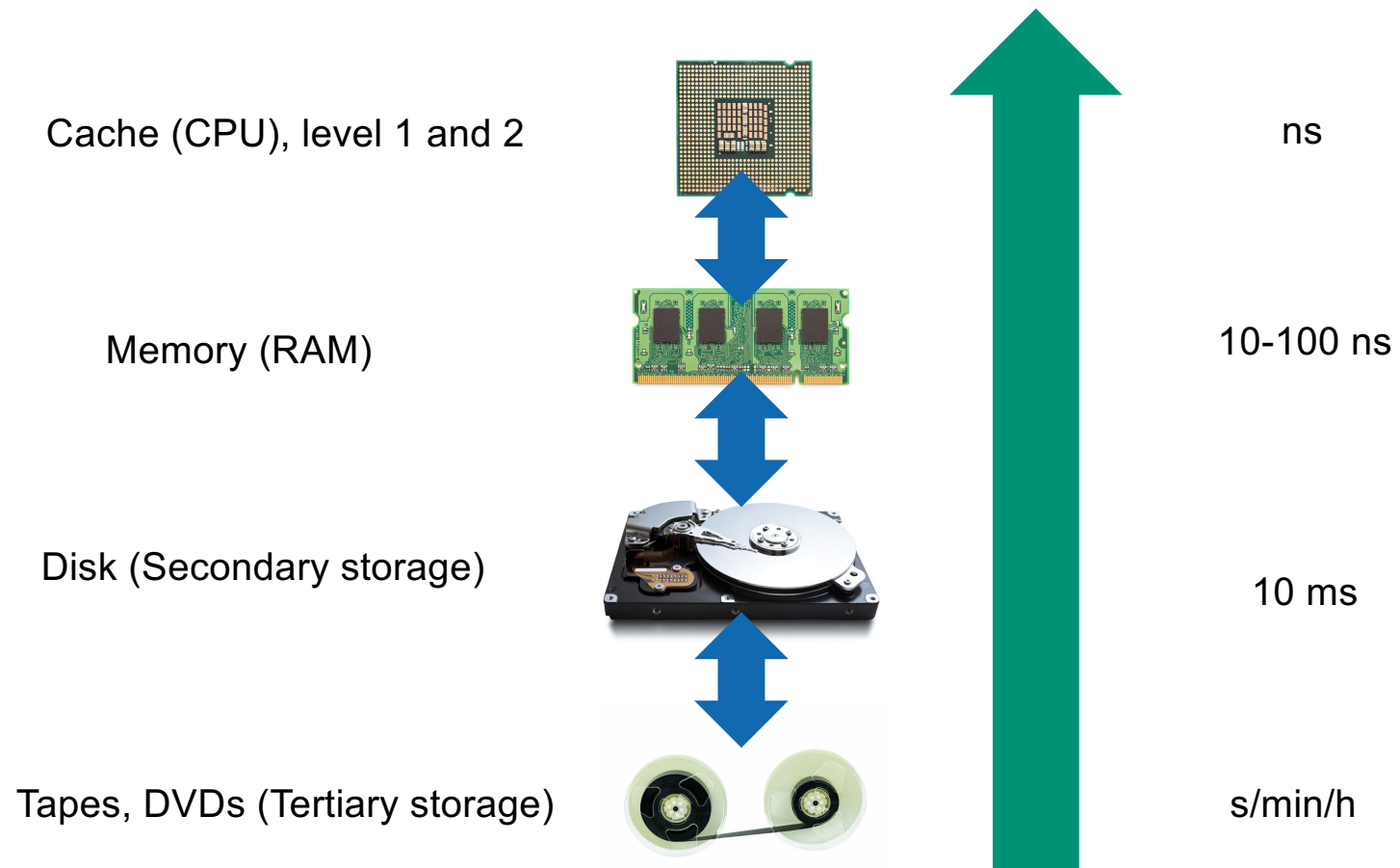


Source: *Michael E. Friske, Claus Mikkelsen, The History of Storage*, SHARE 2014

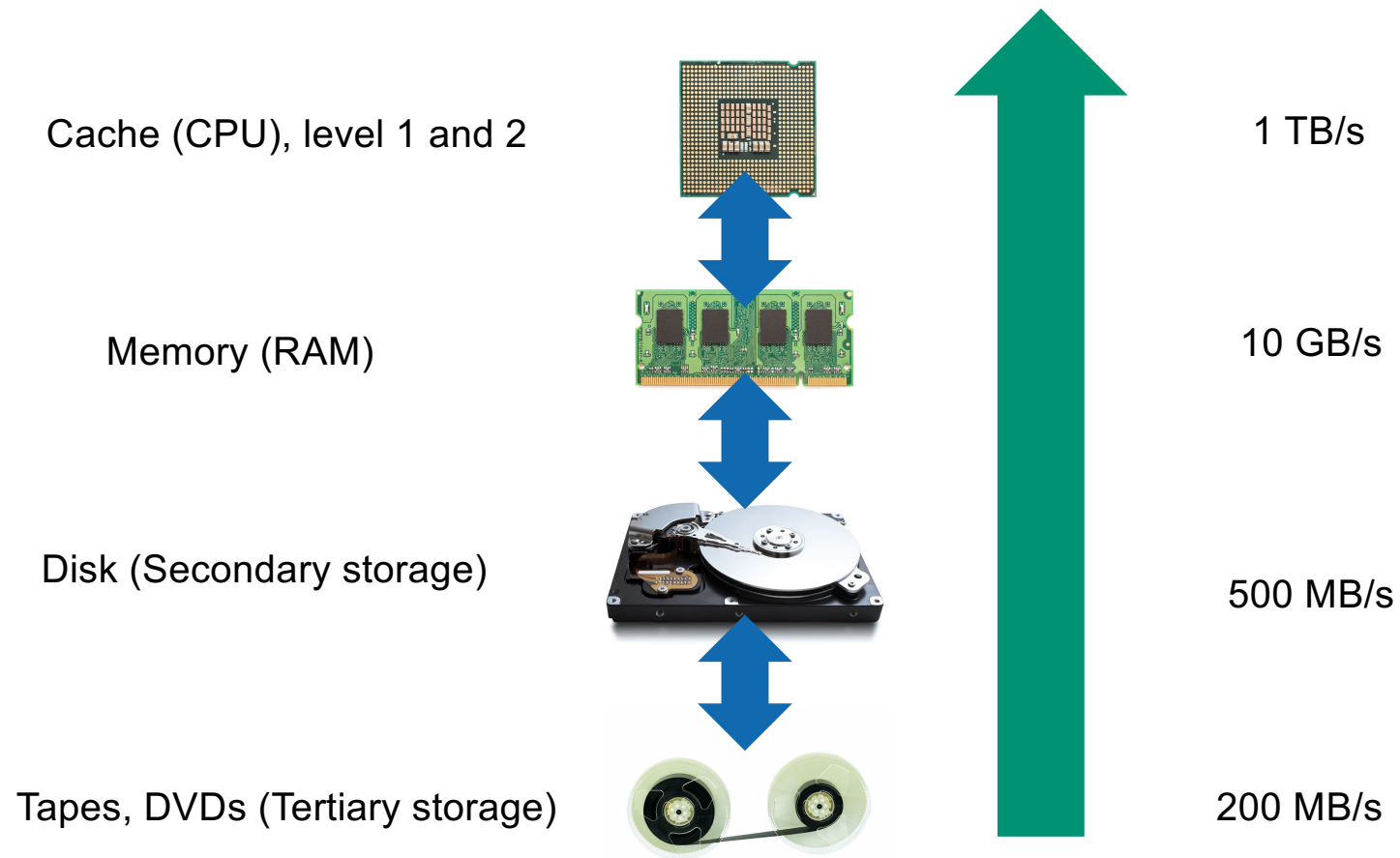
Capacity



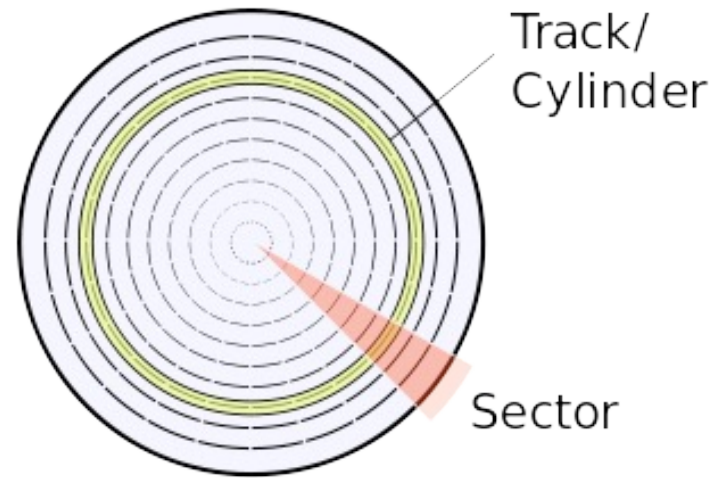
Latency



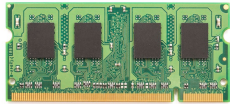
Throughput



Disks



Fetching one byte



Latency (seek)

Transfer



Disks

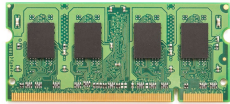


Disks store data in

blocks

~64kb

Fetching one block

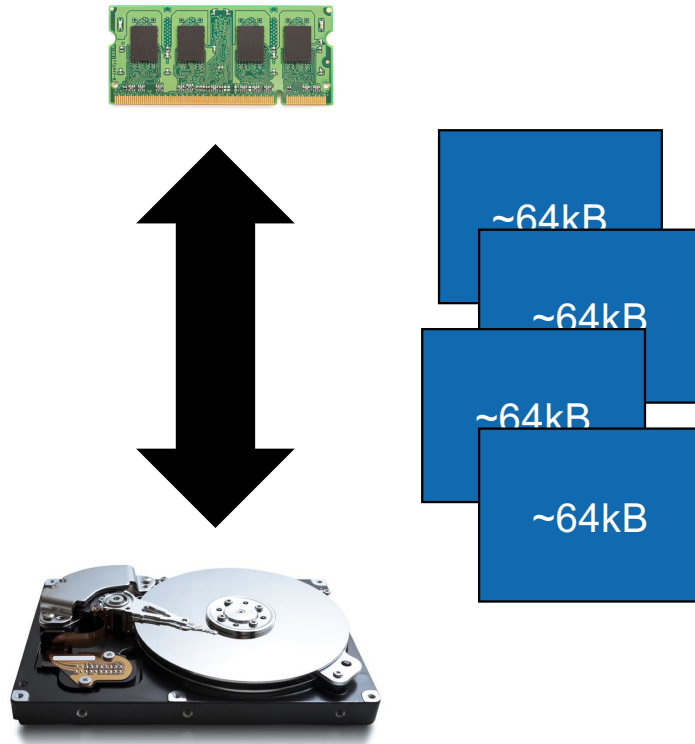


Latency (seek)

Block transfer



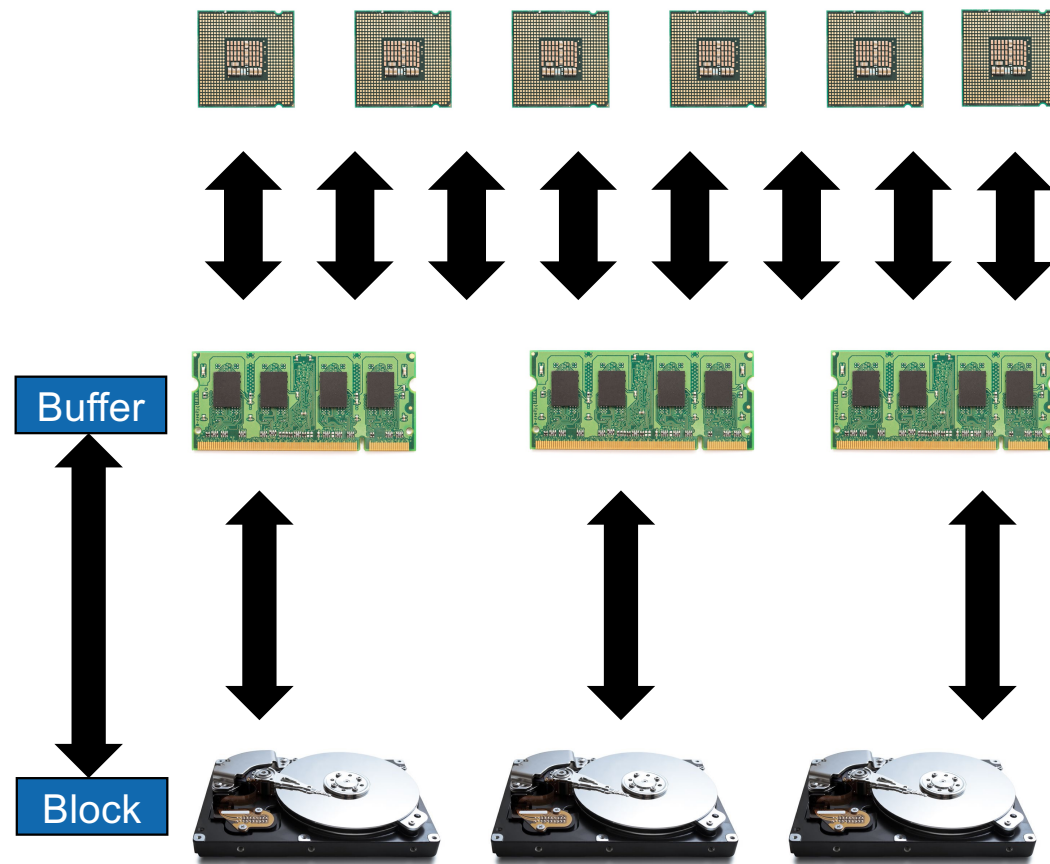
Disks



Data is sent to and from memory

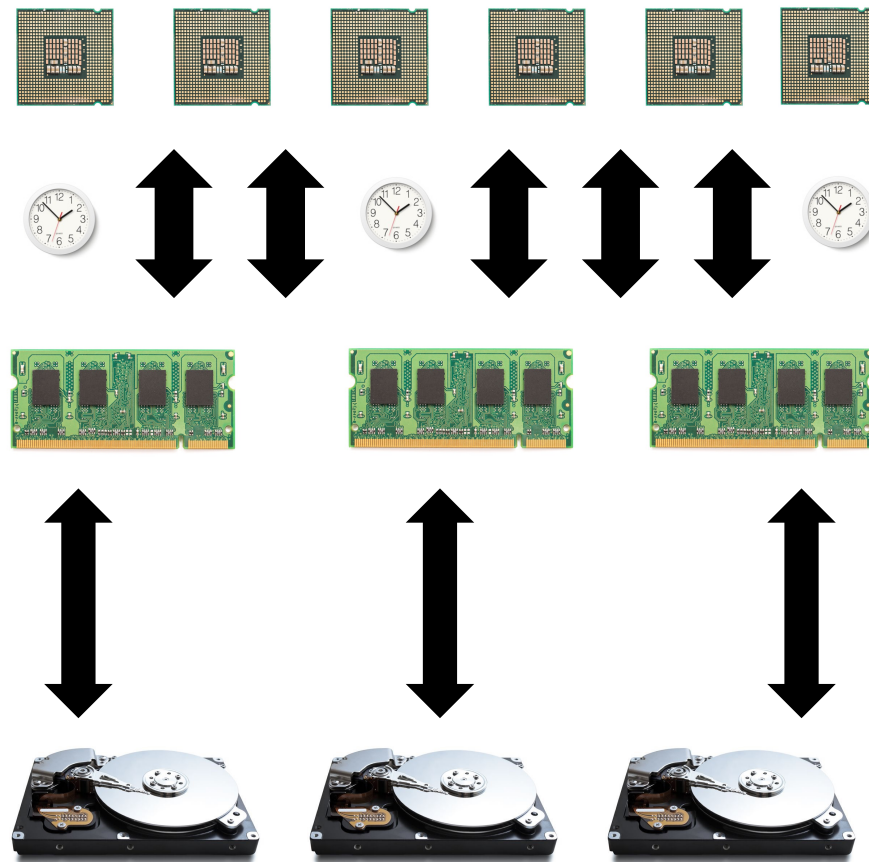
in blocks

Caching

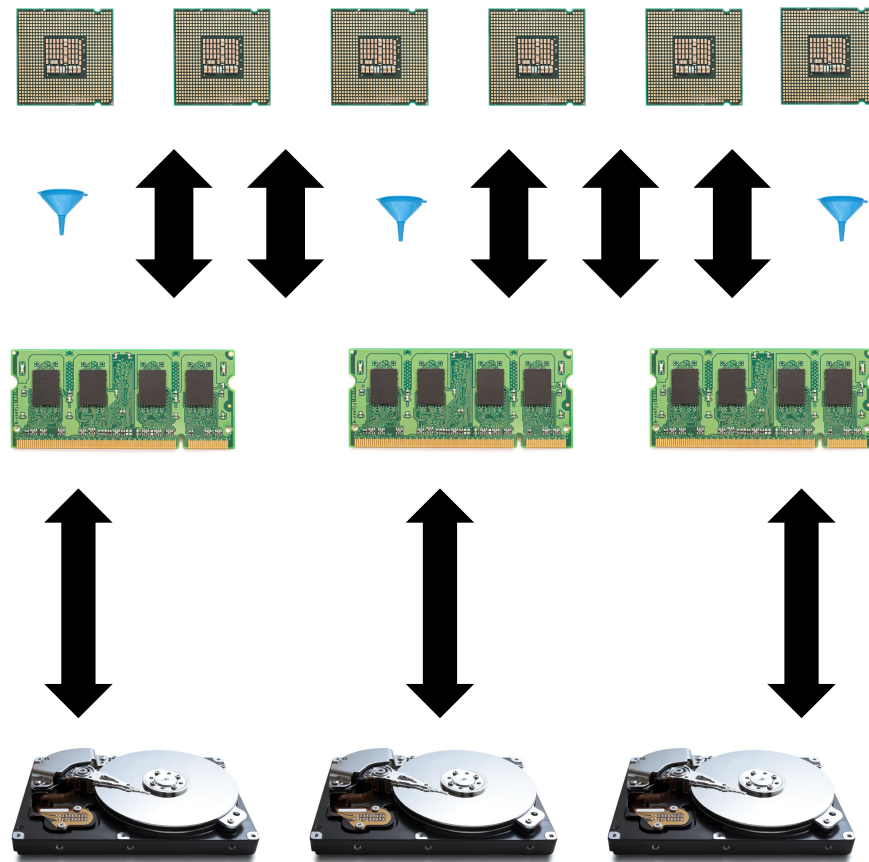


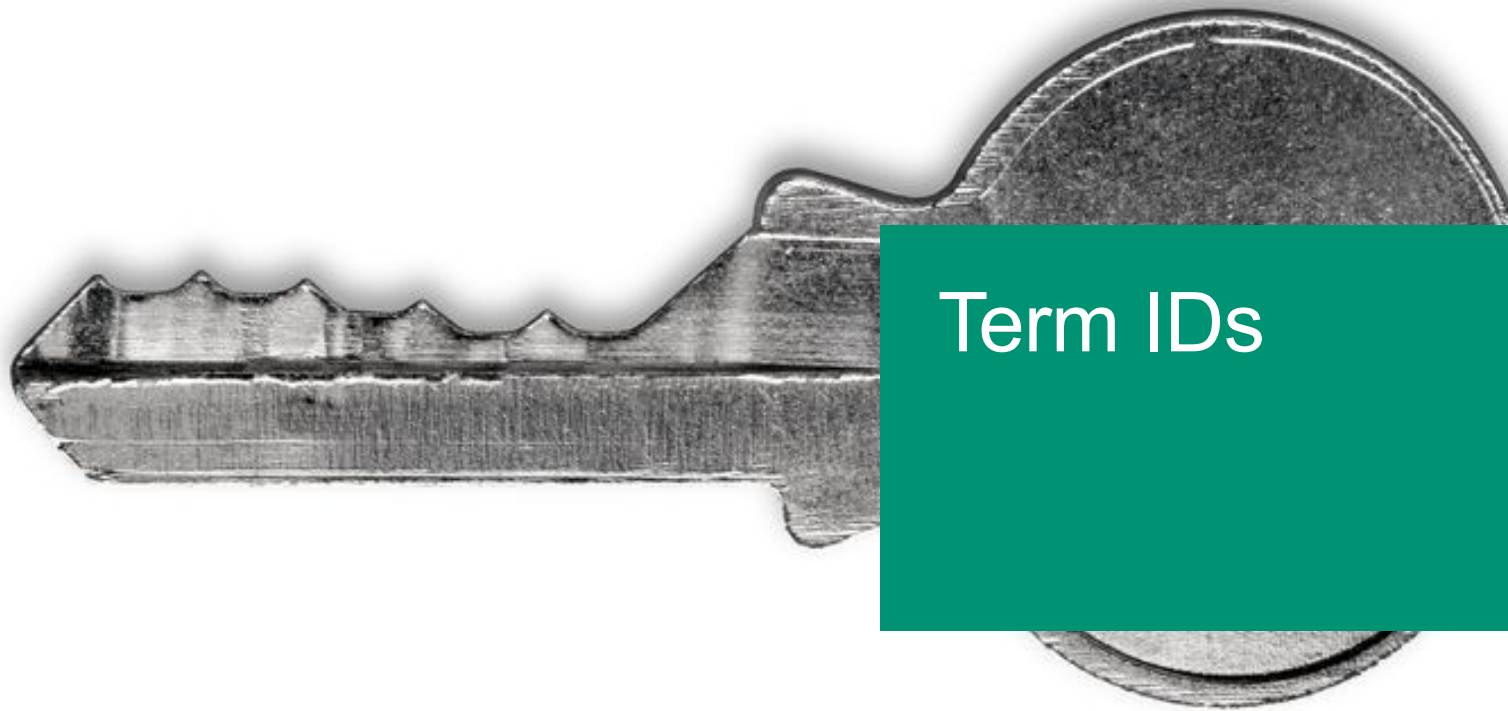
Idle time

Pure I/O
(CPU not needed)



Compression/decompression





Term IDs

Standard inverted index

ETH	➡	6	1	2	3	5	6	8
Zürich	➡	5	3	4	7	8	9	
computer	➡	5	1	2	4	5	7	
data	➡	5	1	3	5	8	9	
CPU	➡	4	2	3	4	7		
information	➡	6	1	2	4	5	8	9
retrieval	➡	4	3	5	7	8		

Logical representation of postings

ETH	➔	ETH	1	ETH	2	ETH	3	...
Zürich	➔	Zürich	3	Zürich	4	Zürich	7	...
computer	➔	computer	1	computer	2	computer	4	...
data	➔	data	1	data	3	data	5	...
CPU	➔	CPU	2	CPU	3	CPU	4	...
information	➔	information	1	information	2	information	4	...
retrieval	➔	retrieval	3	retrieval	5	retrieval	7	...

Let's rewind...

ETH	1	ETH	2	ETH	3	...
Zürich	3	Zürich	4	Zürich	7	...
computer	1	computer	2	computer	4	...
data	1	data	3	data	5	...
CPU	2	CPU	3	CPU	4	...
information	1	information	2	information	4	...
retrieval	3	retrieval	5	retrieval	7	...

Let's rewind...

ETH	1
computer	1
data	1
information	1

CPU	2
ETH	2
information	2
computer	2

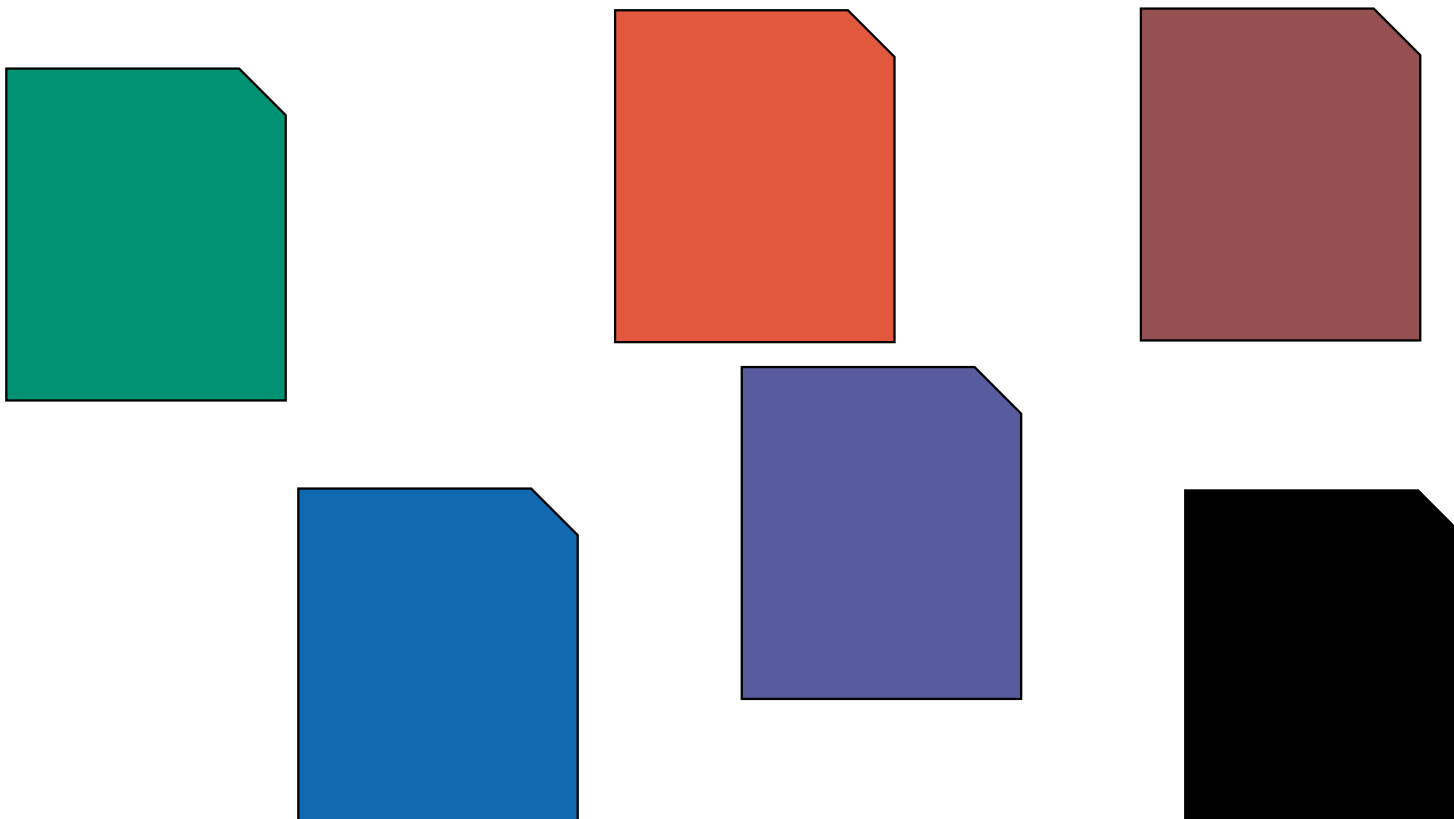
retrieval	7
Zürich	7

Zürich	3
ETH	3
CPU	3
data	3
retrieval	3

Zürich	4
CPU	4
information	4
computer	4

data	5
retrieval	5

Let's rewind...



This intermediate representation is very space consuming!

ETH	1	ETH	2	ETH	3	...
Zürich	3	Zürich	4	Zürich	7	...
computer	1	computer	2	computer	4	...
data	1	data	3	data	5	...
CPU	2	CPU	3	CPU	4	...
information	1	information	2	information	4	...
retrieval	3	retrieval	5	retrieval	7	...

Reuters-RCV1 collection: example

1GB of text
800,000 documents
100 million tokens

Reuters-RCV1 collection: example

token in memory

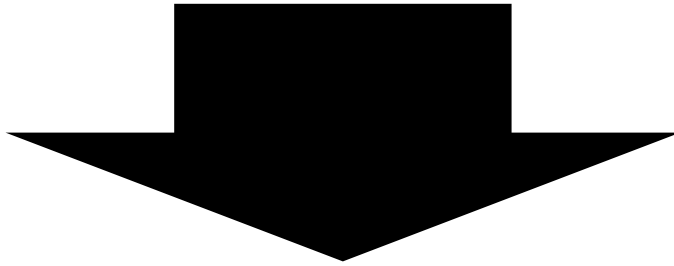


let's take 10 bytes in average 4 bytes

$$14 \text{ bytes} \times 100,000,000 = 1.4 \text{ GB}$$

Reuters-RCV1 collection: example

Let's say we have 16 GB



1,100,000,000 tokens
11 GB of text

This intermediate representation is very space consuming!

ETH	1	ETH	2	ETH	3	...
Zürich	3	Zürich	4	Zürich	7	...
computer	1	computer	2	computer	4	...
data	1	data	3	data	5	...
CPU	2	CPU	3	CPU	4	...
information	1	information	2	information	4	...
retrieval	3	retrieval	5	retrieval	7	...

TermIDs

t1	1	t1	2	t1	3	...
t2	3	t2	4	t2	7	...
t3	1	t3	2	t3	4	...
t4	1	t4	3	t4	5	...
t5	2	t5	3	t5	4	...
t6	1	t6	2	t6	4	...
t7	3	t7	5	t7	7	...

Term/TermID mapping

t1	ETH
t2	Zürich
t3	computer
t4	data
t5	CPU
t6	information
t7	retrieval

Reuters-RCV1 collection: example

token in memory

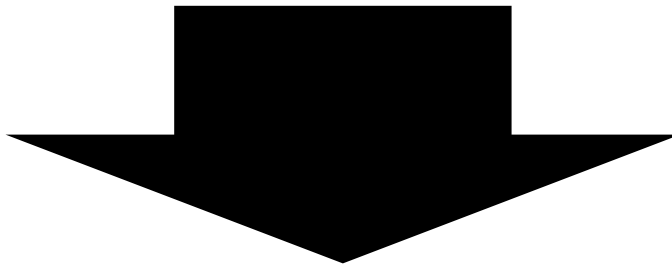


4 bytes (always) 4 byte

$8 \text{ bytes} \times 100,000,000 = 0.8 \text{ GB}$

Reuters-RCV1 collection: example

Let's say we have 16 GB

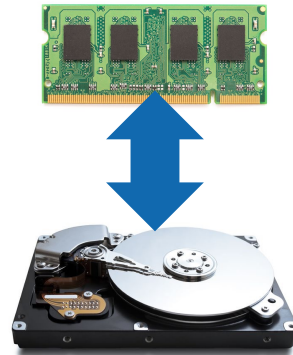


2,000,000,000 tokens
20 GB of text

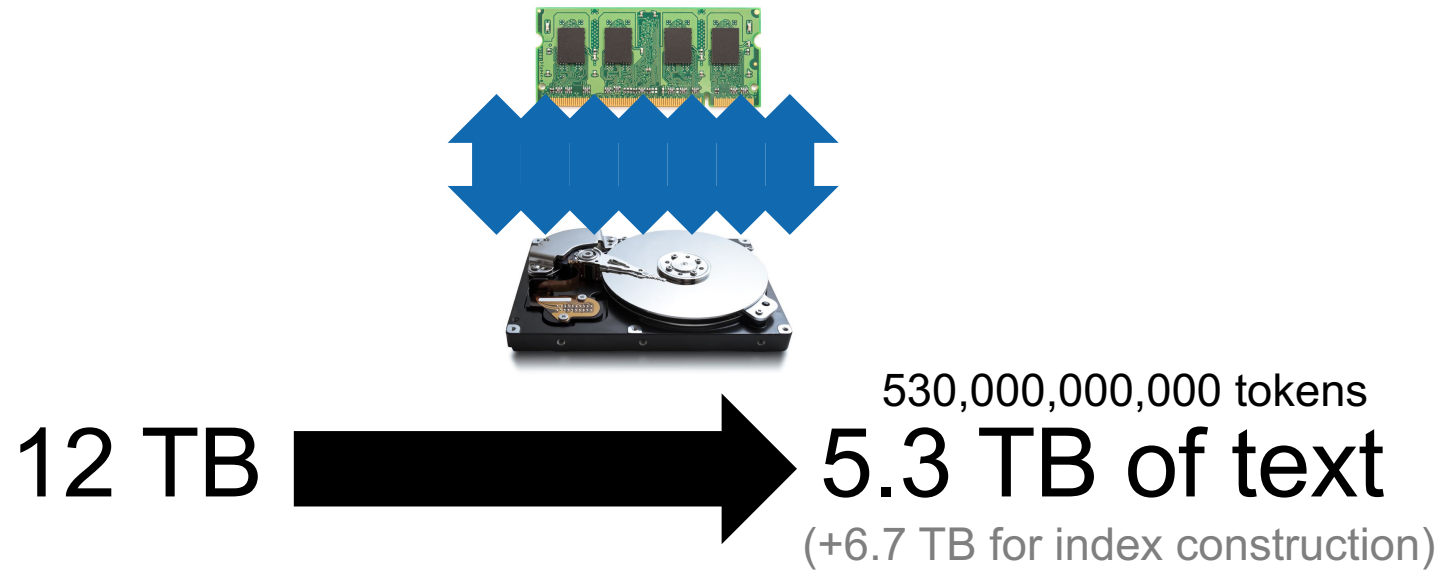


Blocked Sort-Based Indexing

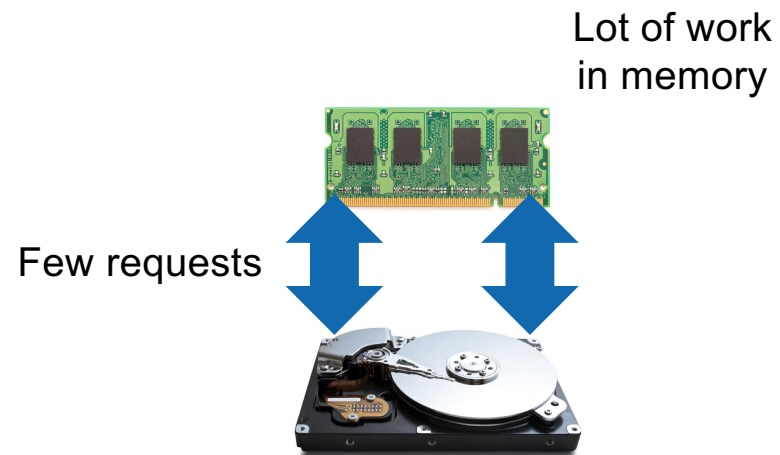
Could we leverage the space on disk?



If we didn't care about speed...

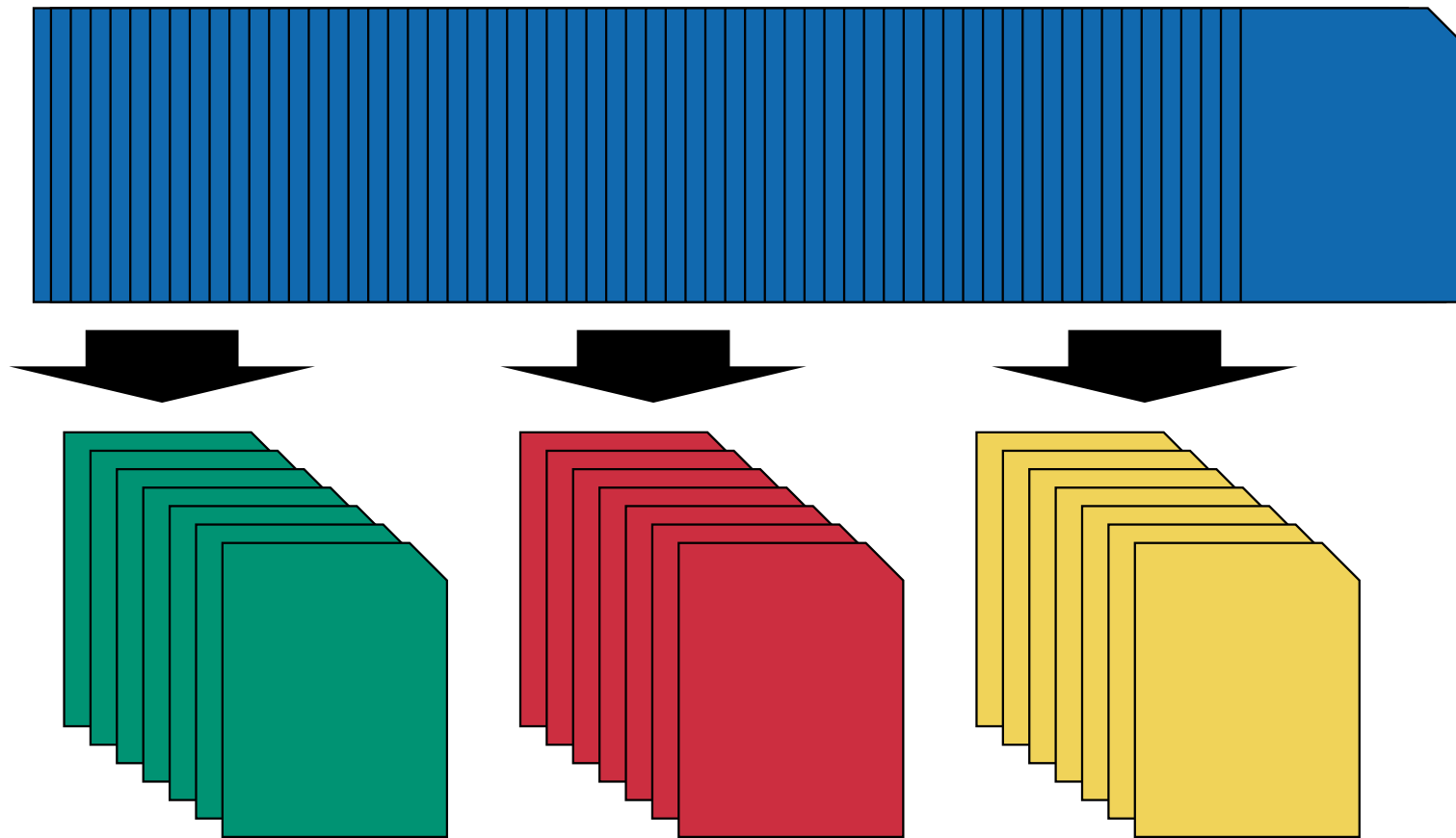


But we DO care about speed.

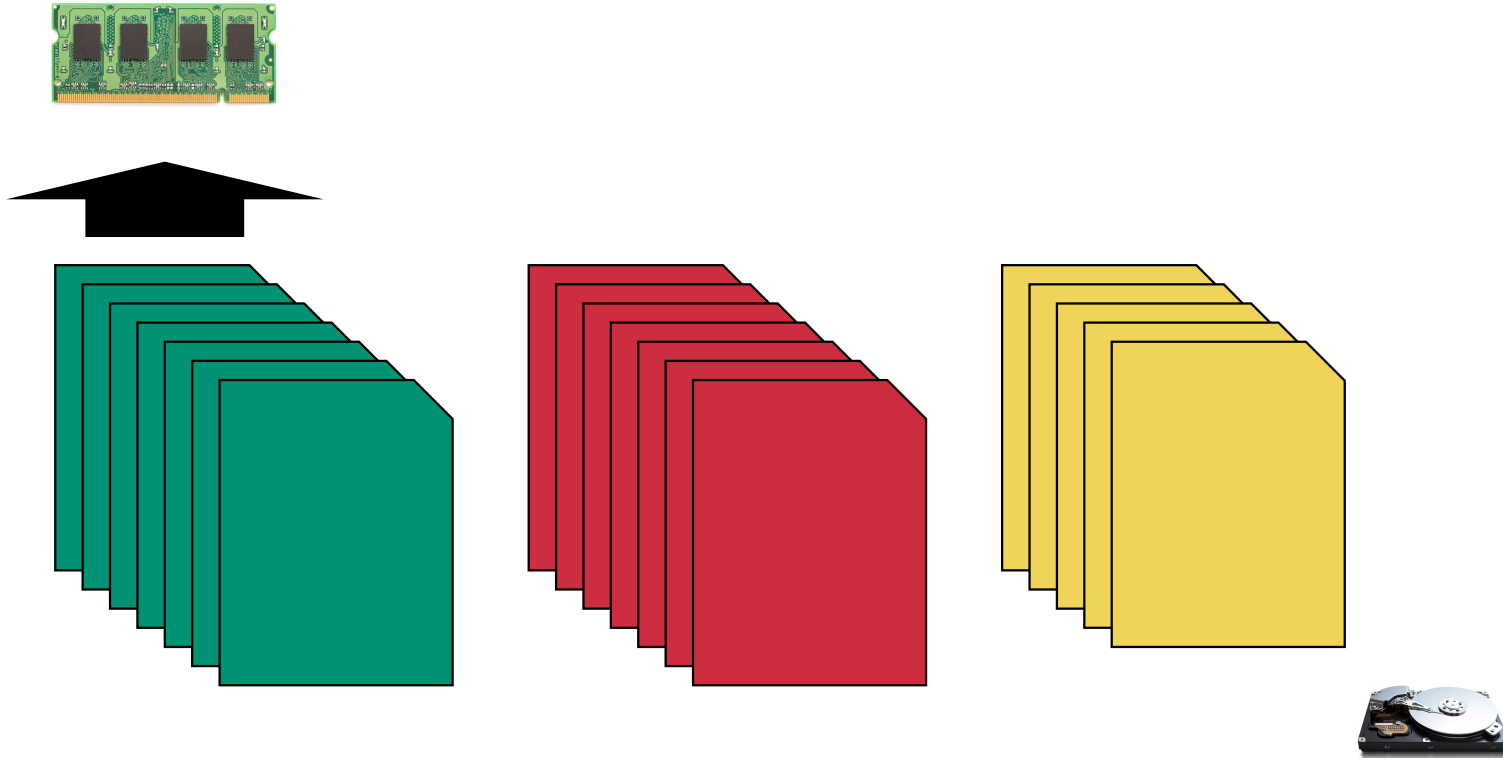


Batch processing

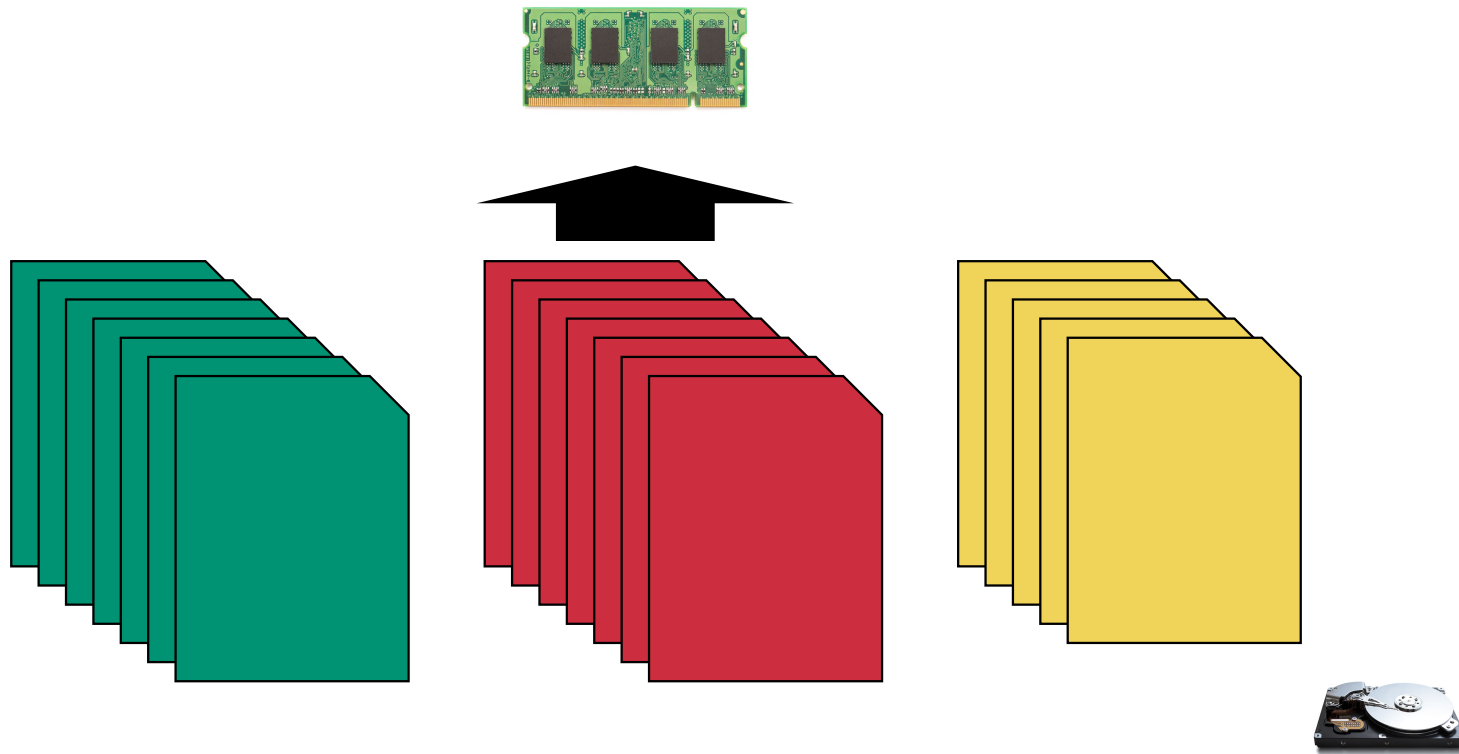
Shard the collection of documents



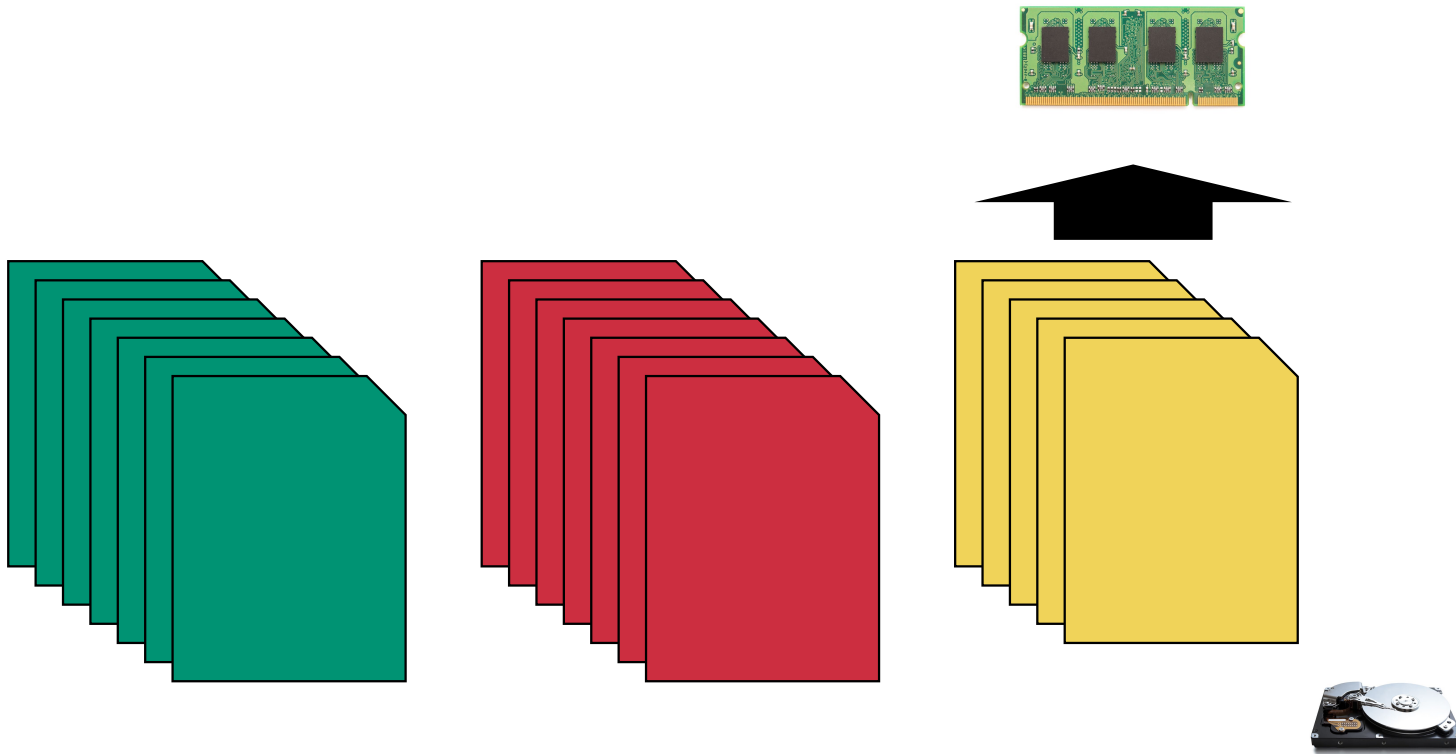
Process them one by one in memory



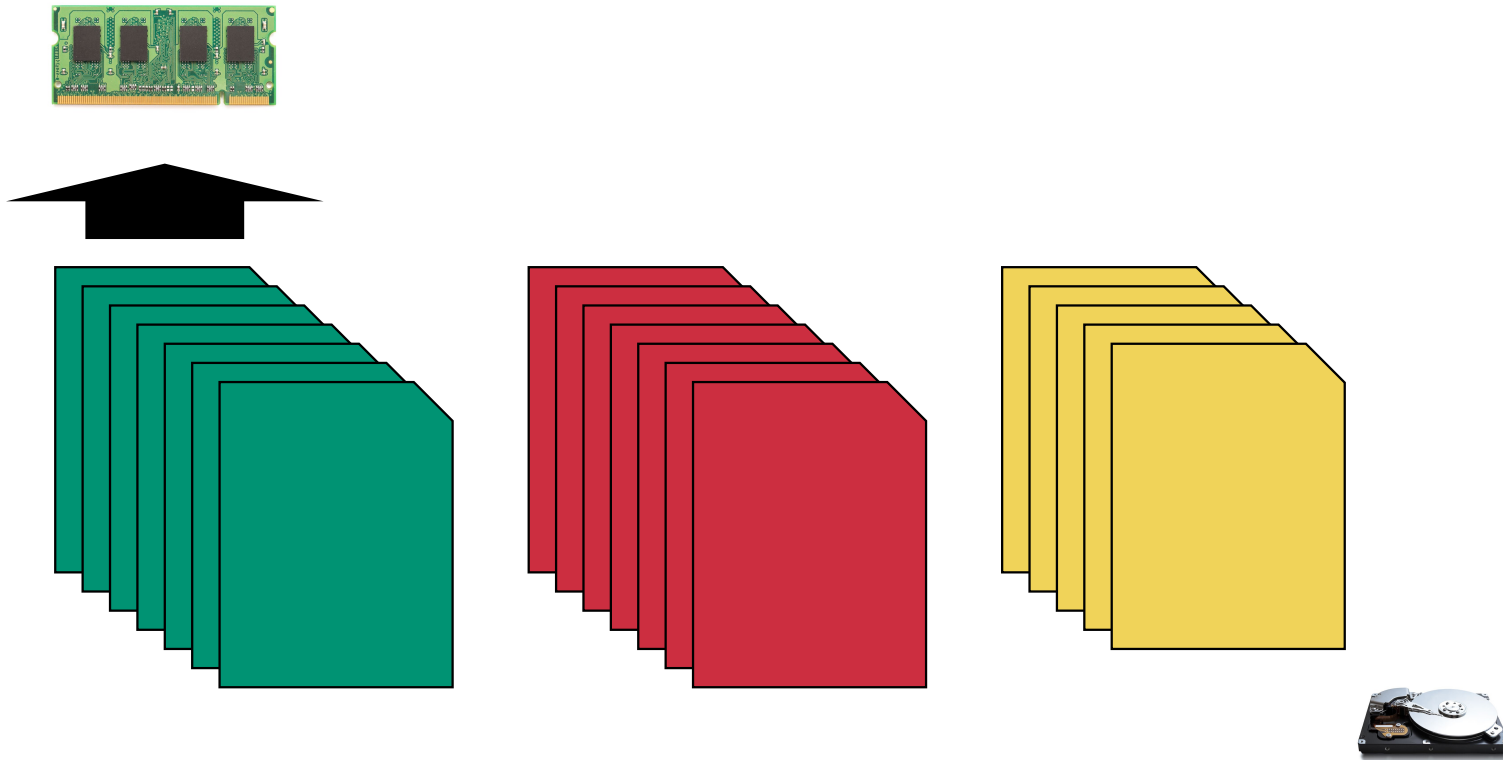
Process them one by one in memory



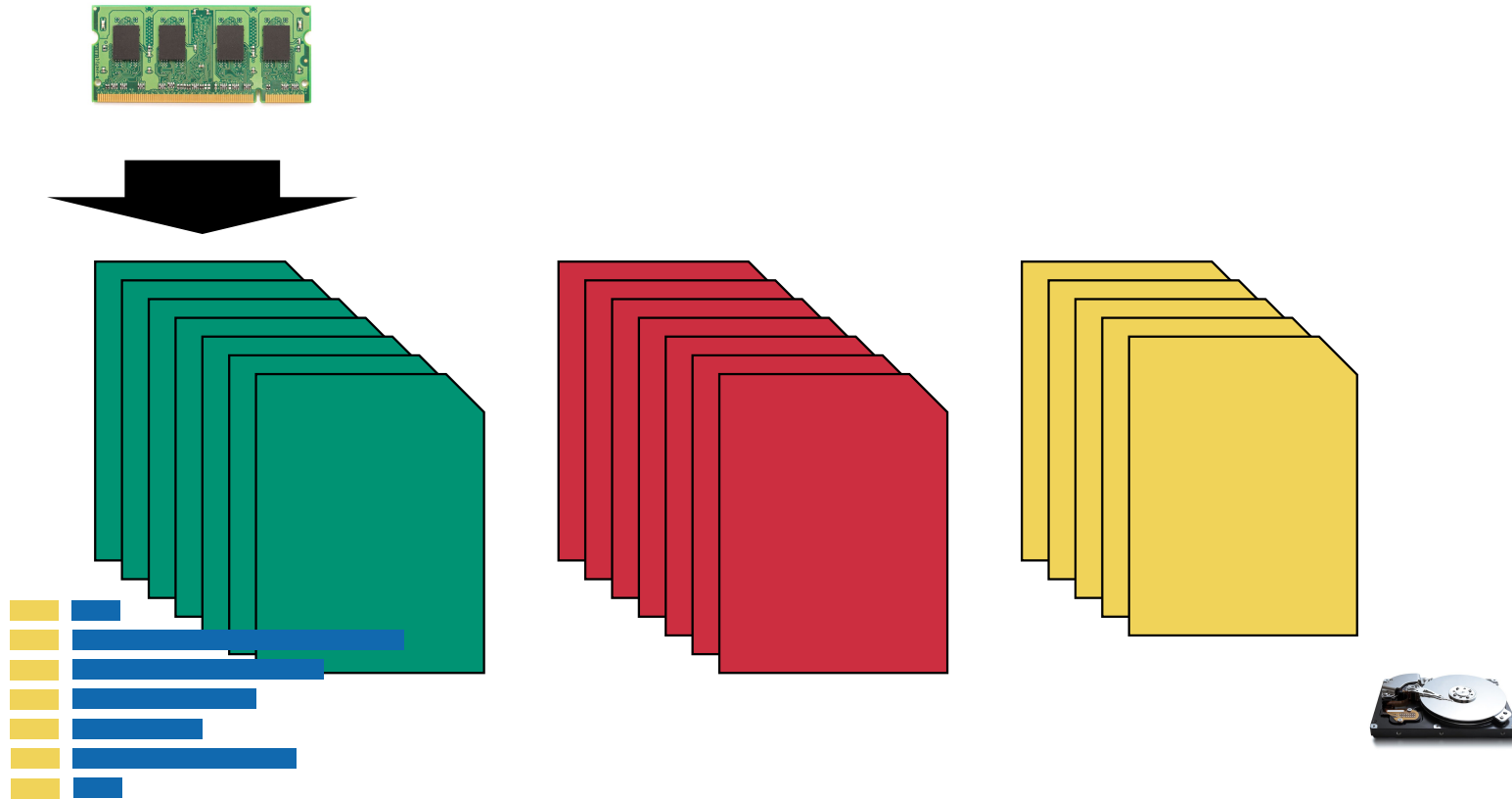
Process them one by one in memory



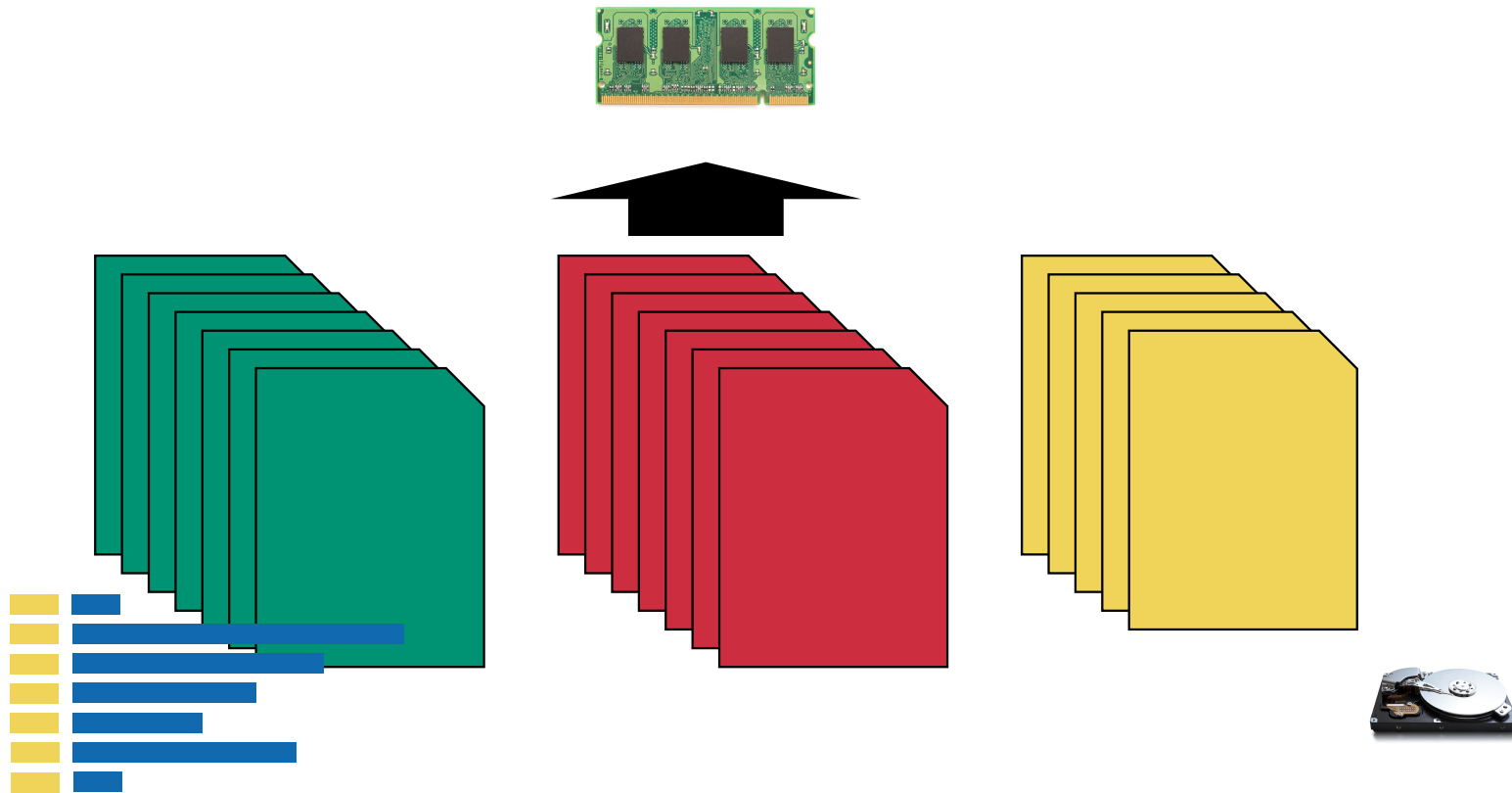
Write back intermediate results



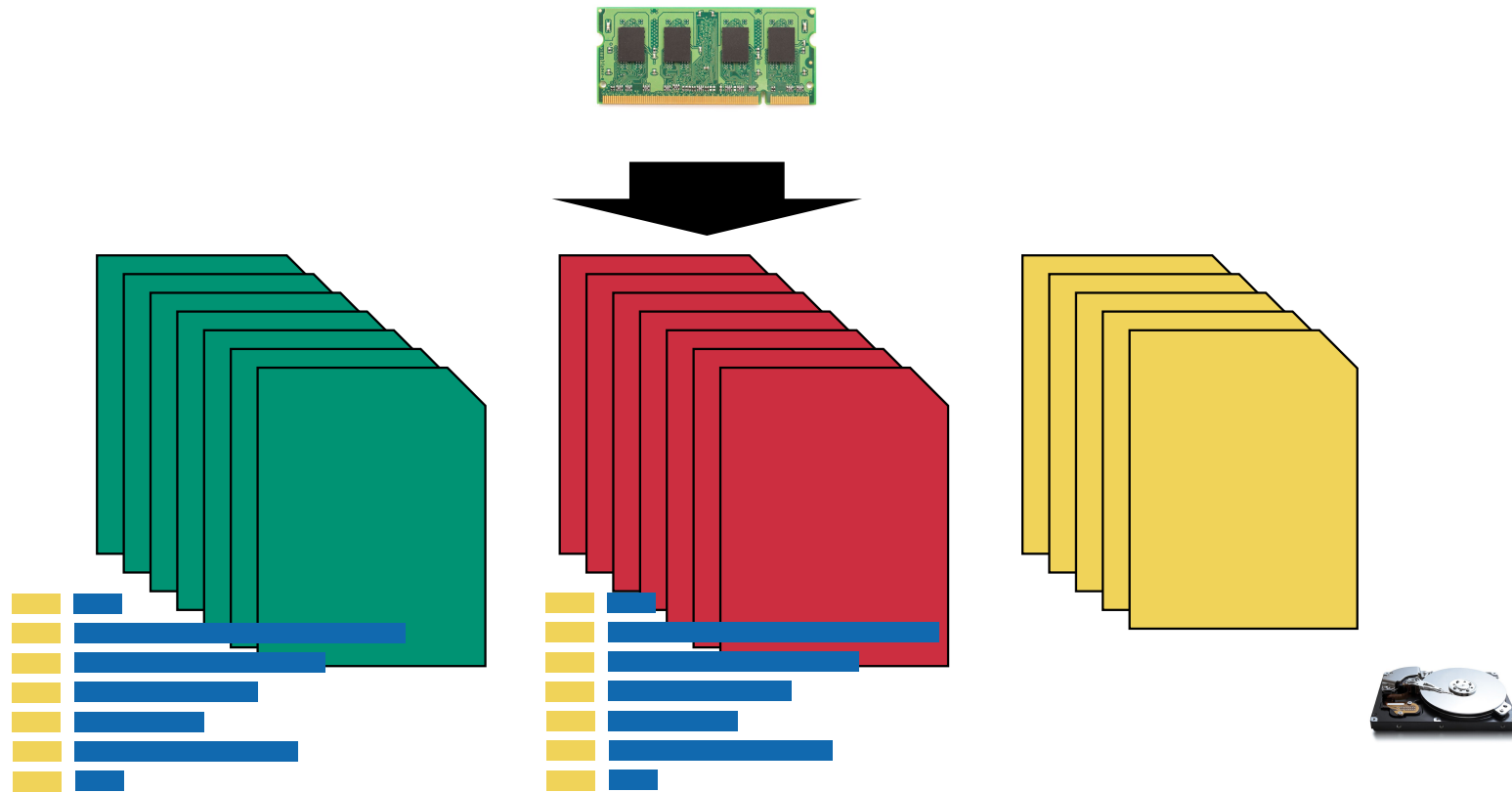
Write back intermediate results



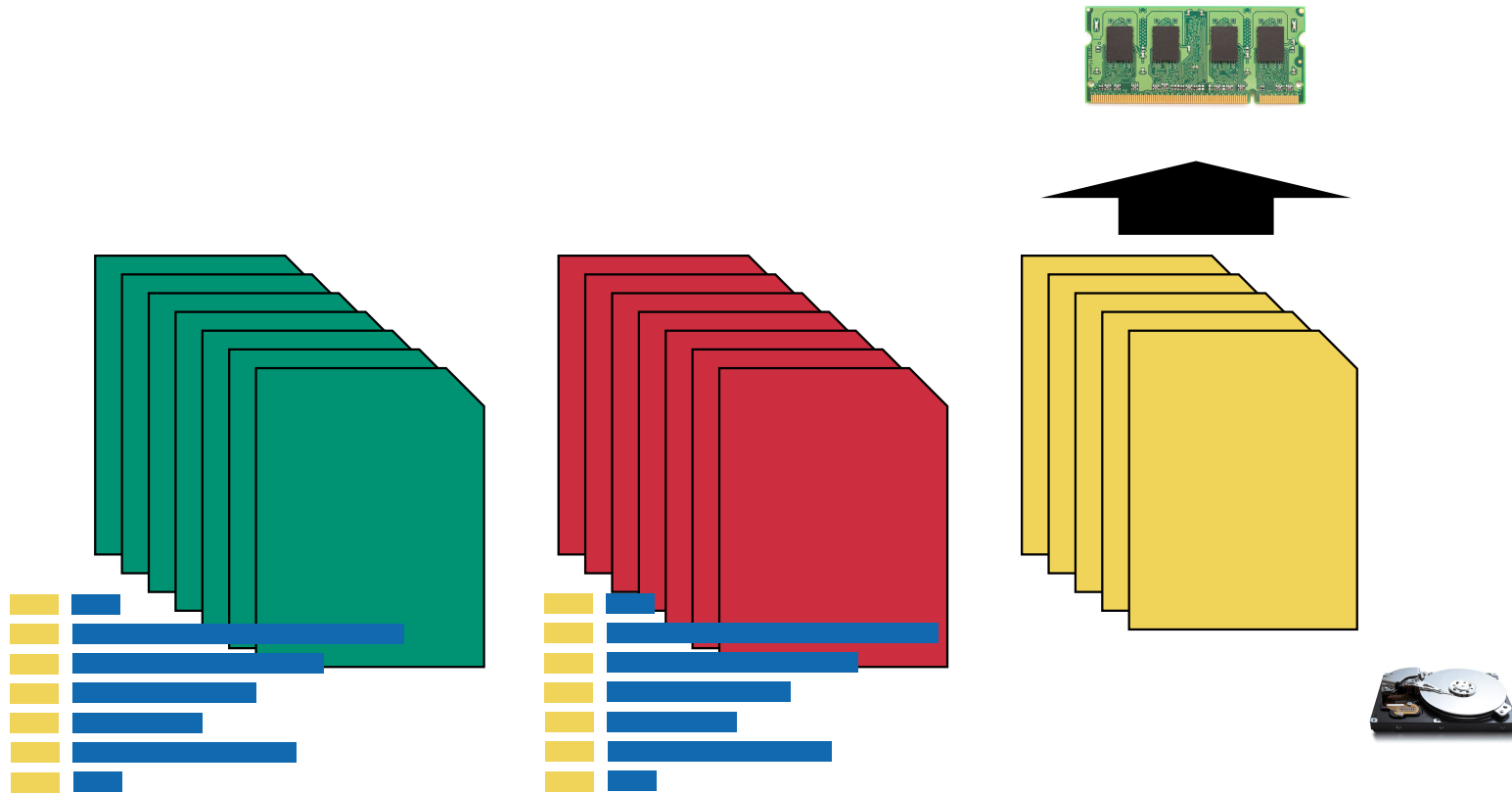
Write back intermediate results



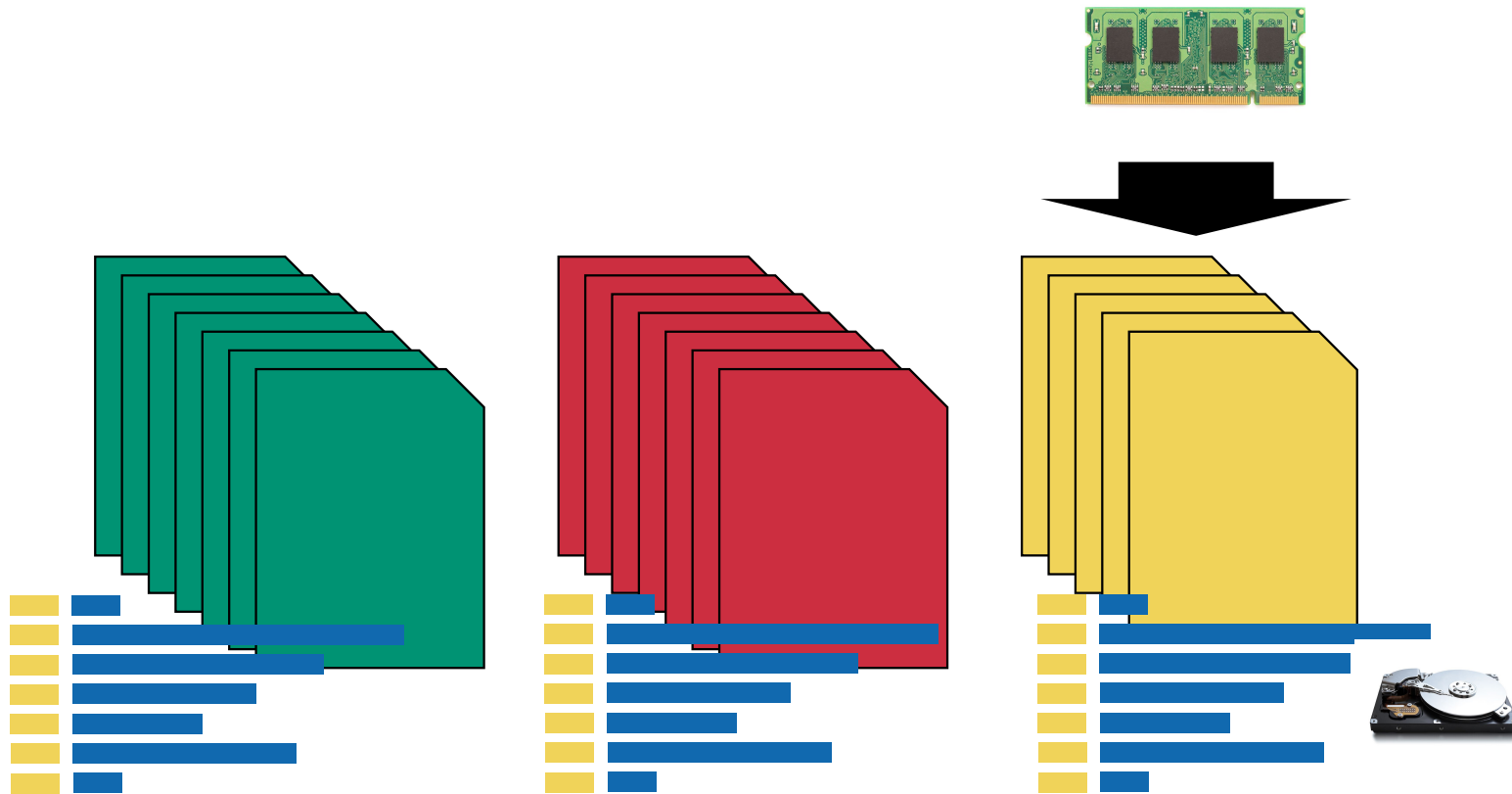
Write back intermediate results



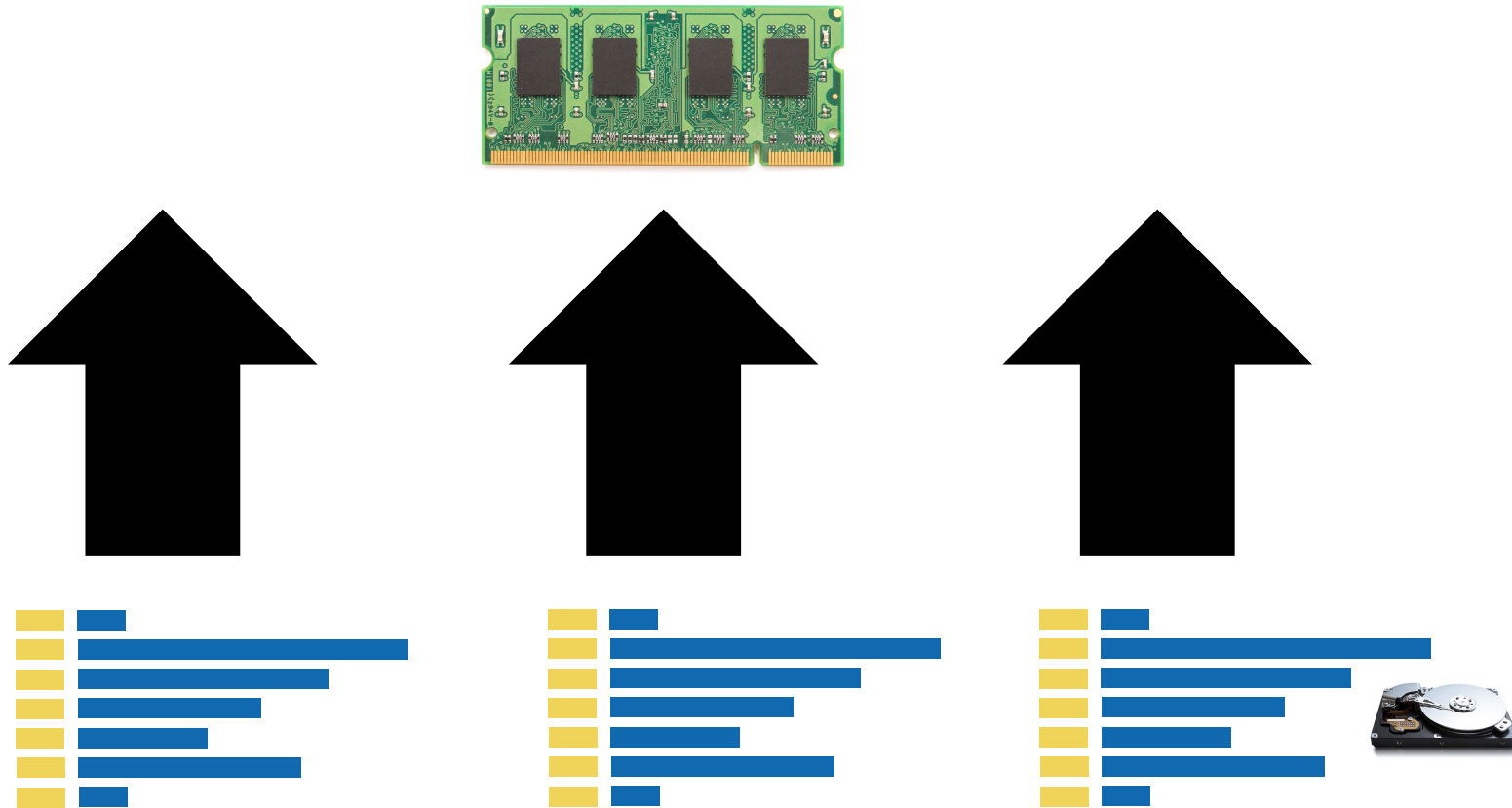
Write back intermediate results



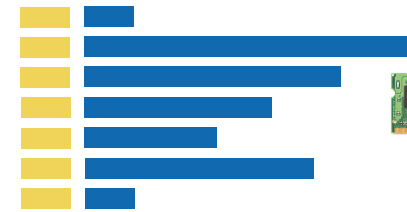
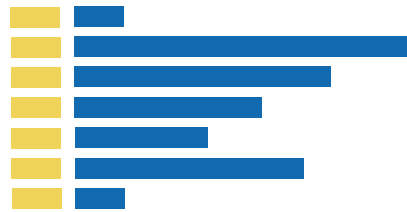
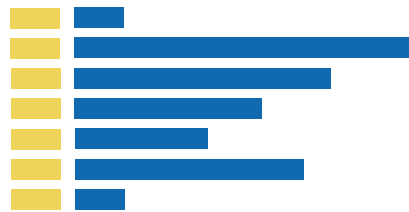
Write back intermediate results



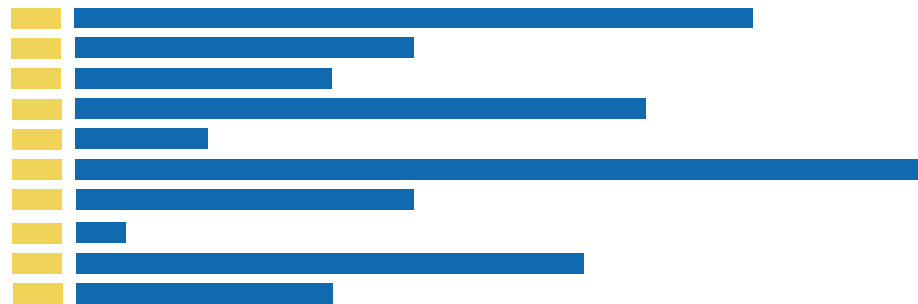
Merge intermediate results into final index



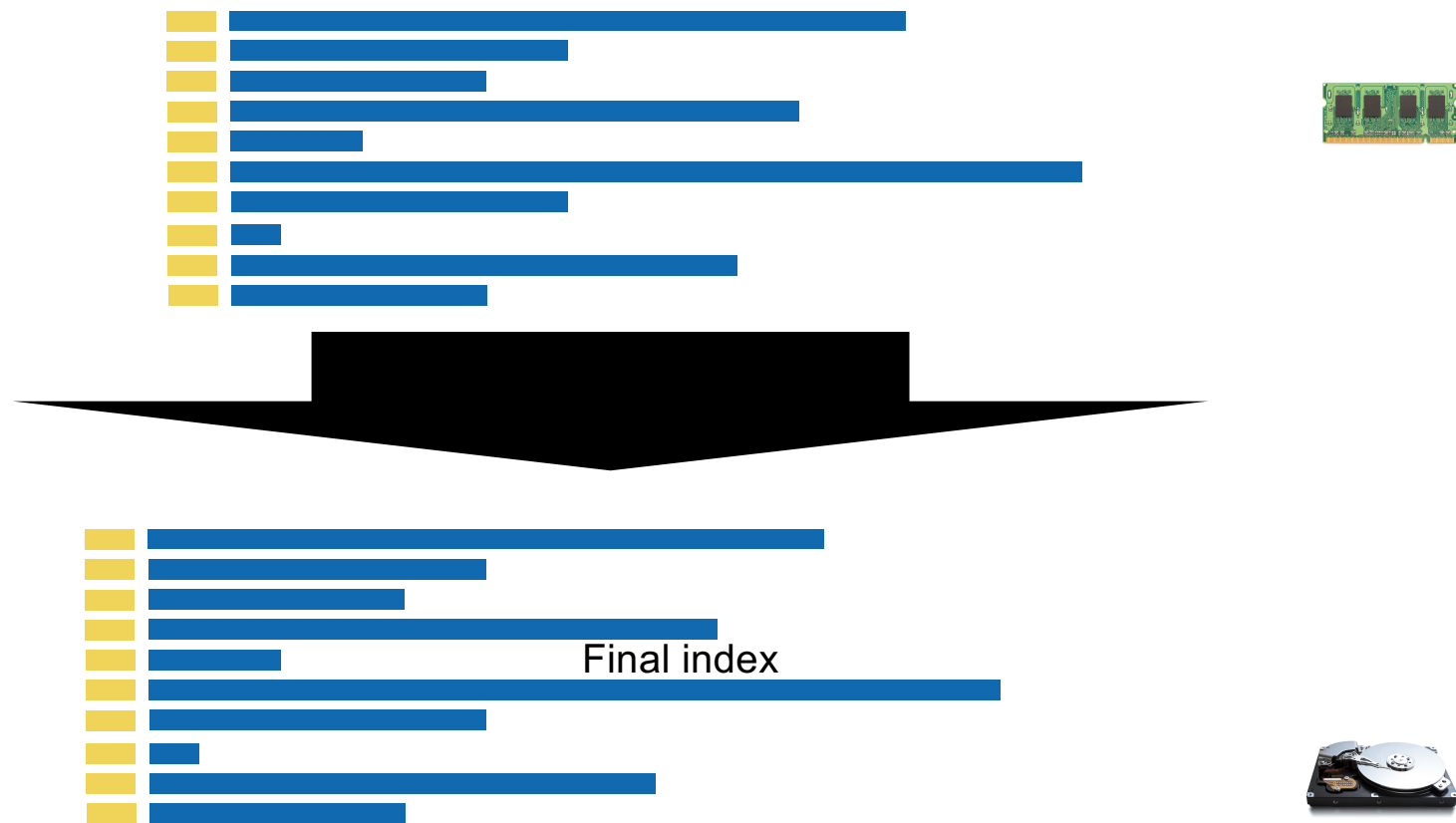
Merge intermediate results into final index



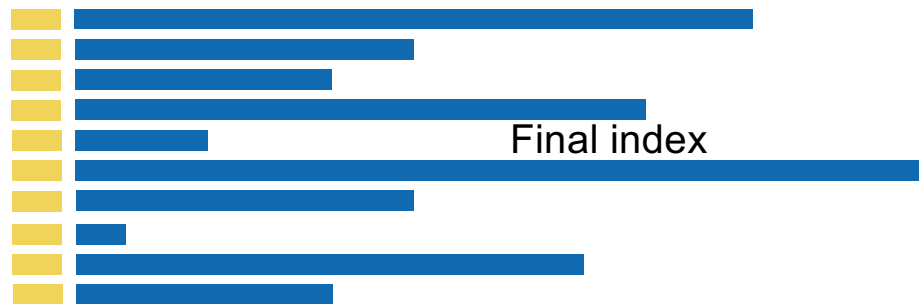
Merge intermediate results into final index



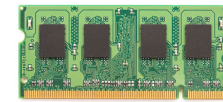
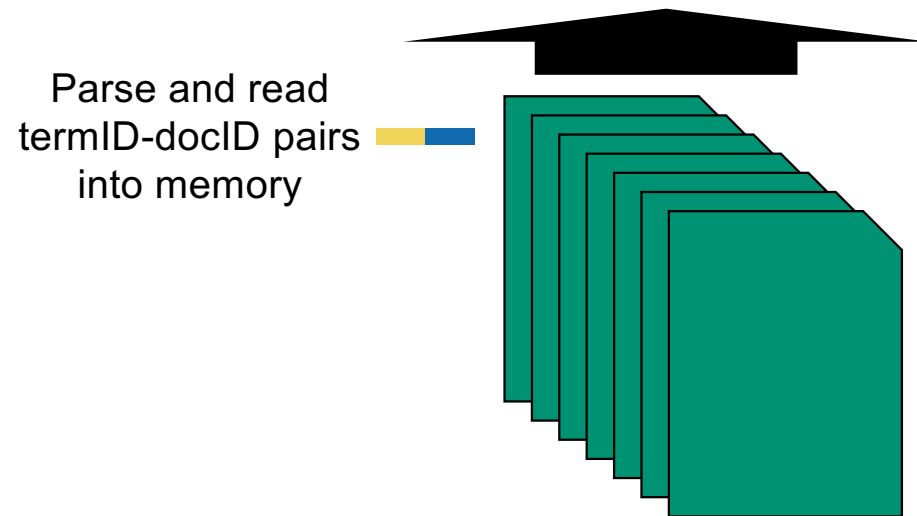
Merge intermediate results into final index



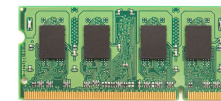
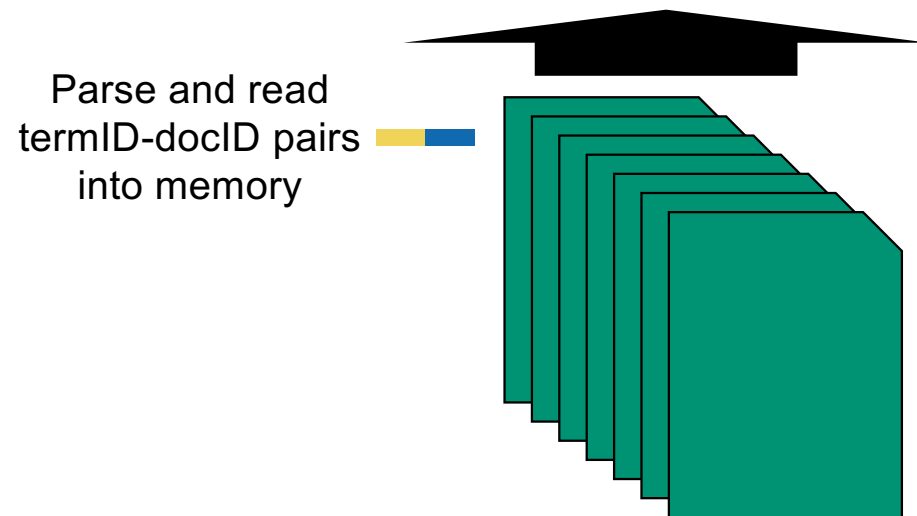
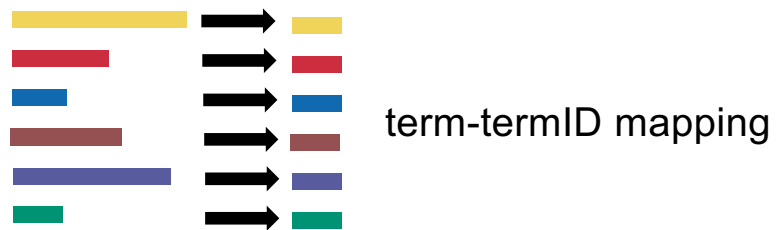
Merge intermediate results into final index



Zooming in: processing a block



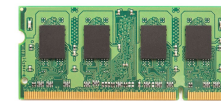
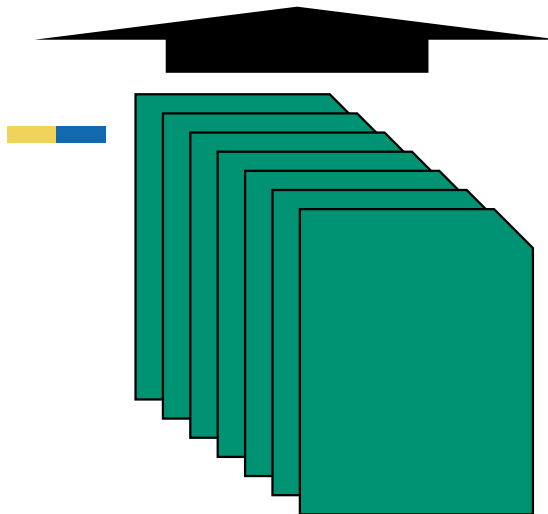
Zooming in: processing a block



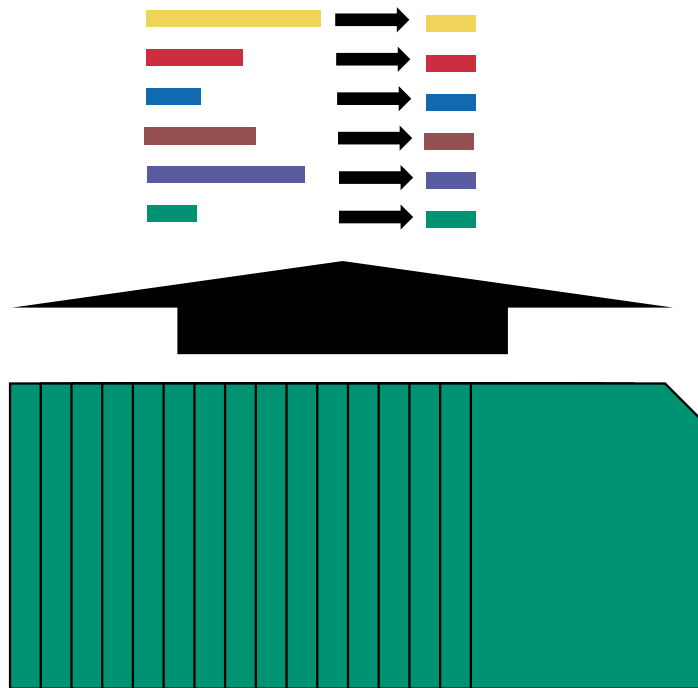
Zooming in: processing a block



Parse and read
termID-docID pairs
into memory

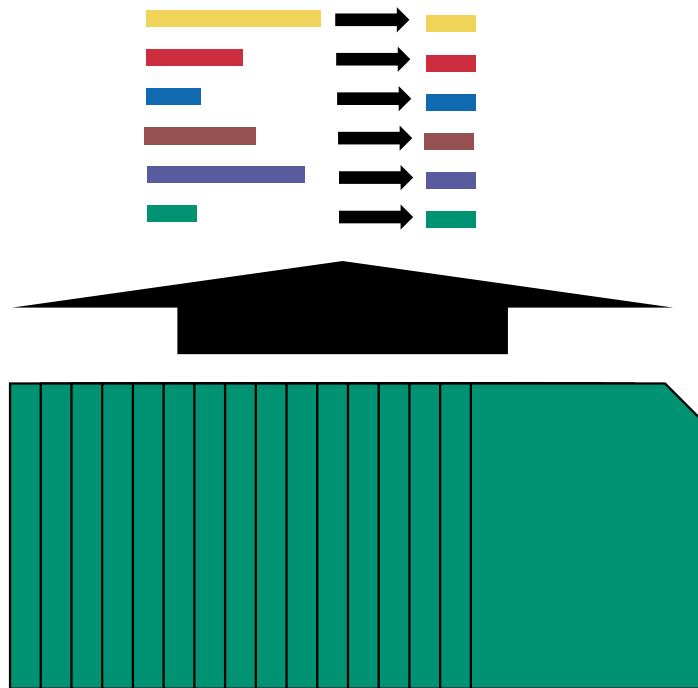


Generating termIDs

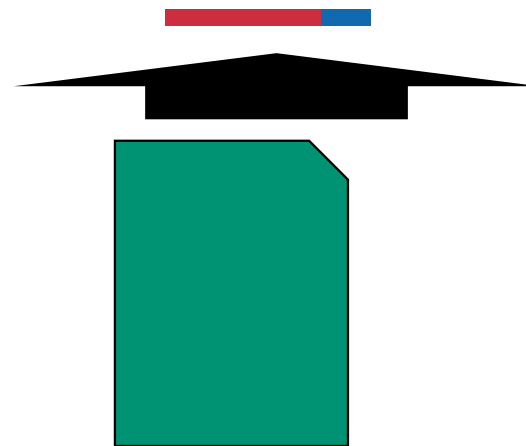


Strategy 1
Do a first pass to
build term-termID mapping

Generating termIDs

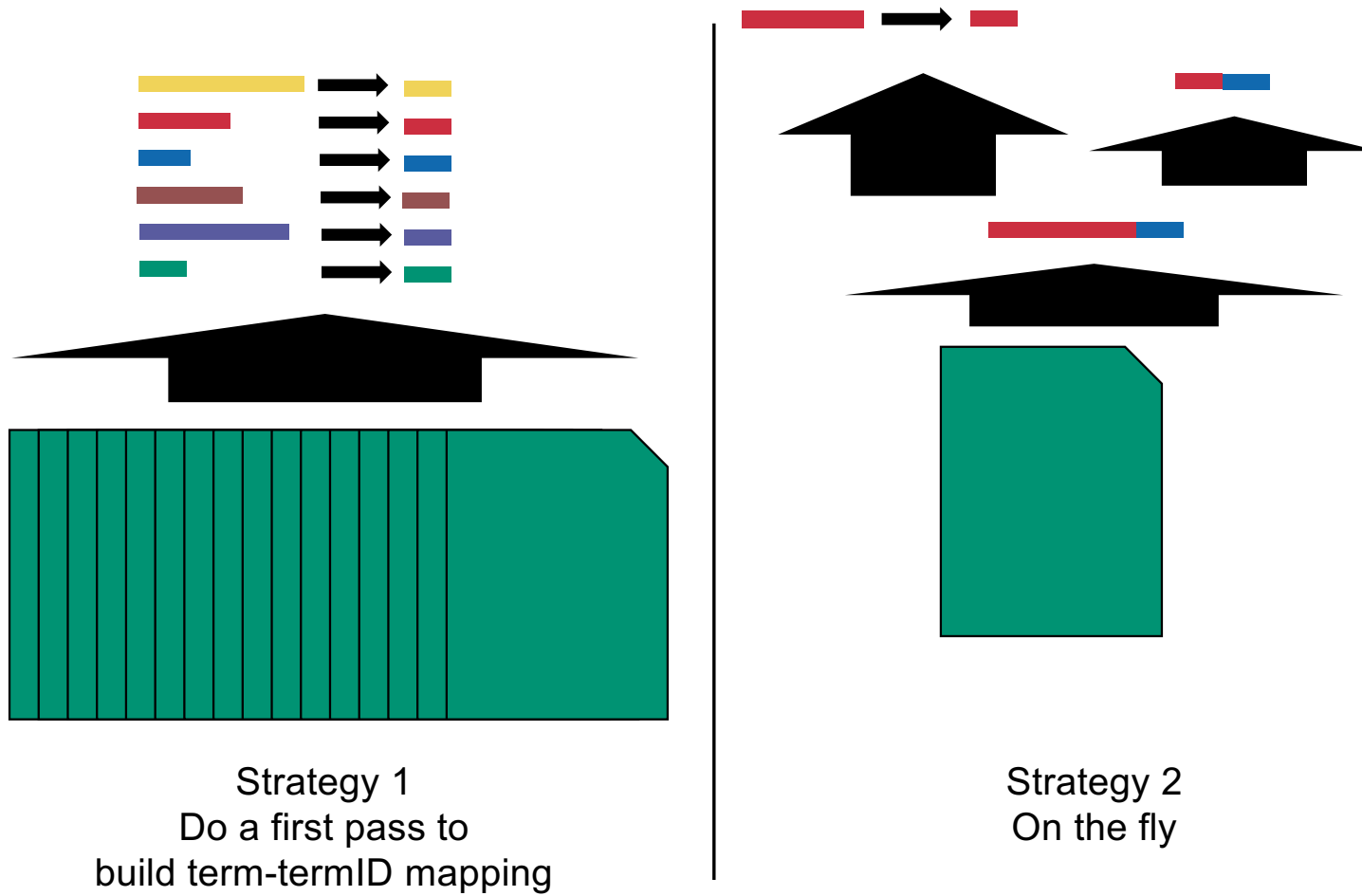


Strategy 1
Do a first pass to
build term-termID mapping

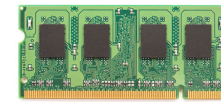
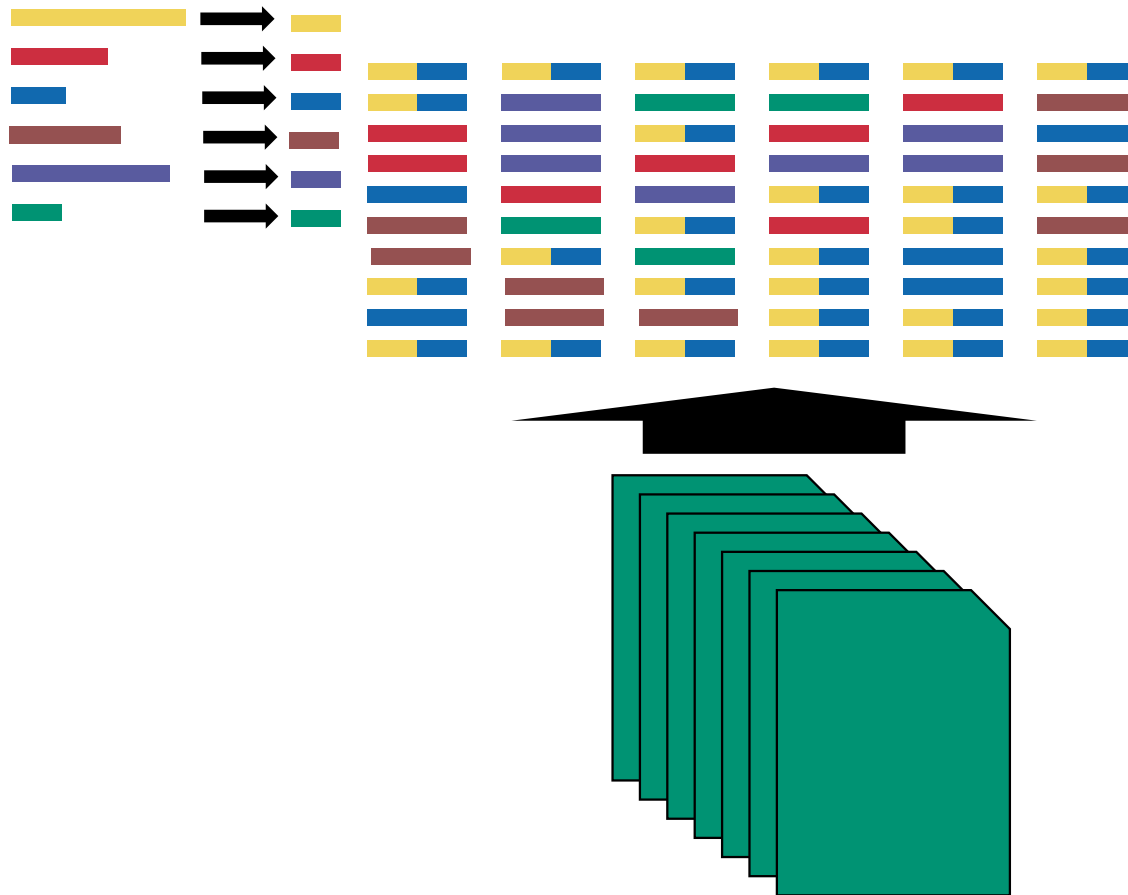


Strategy 2
On the fly

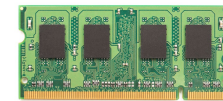
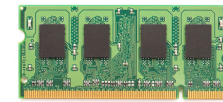
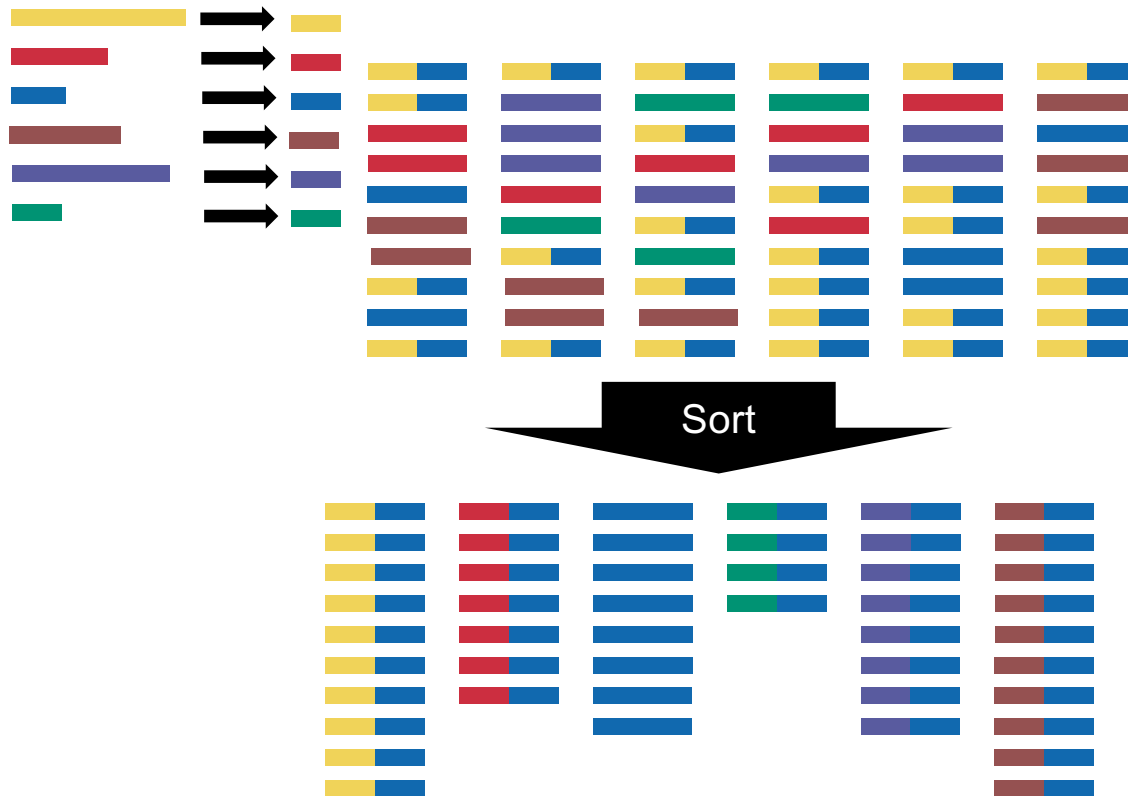
Generating termIDs



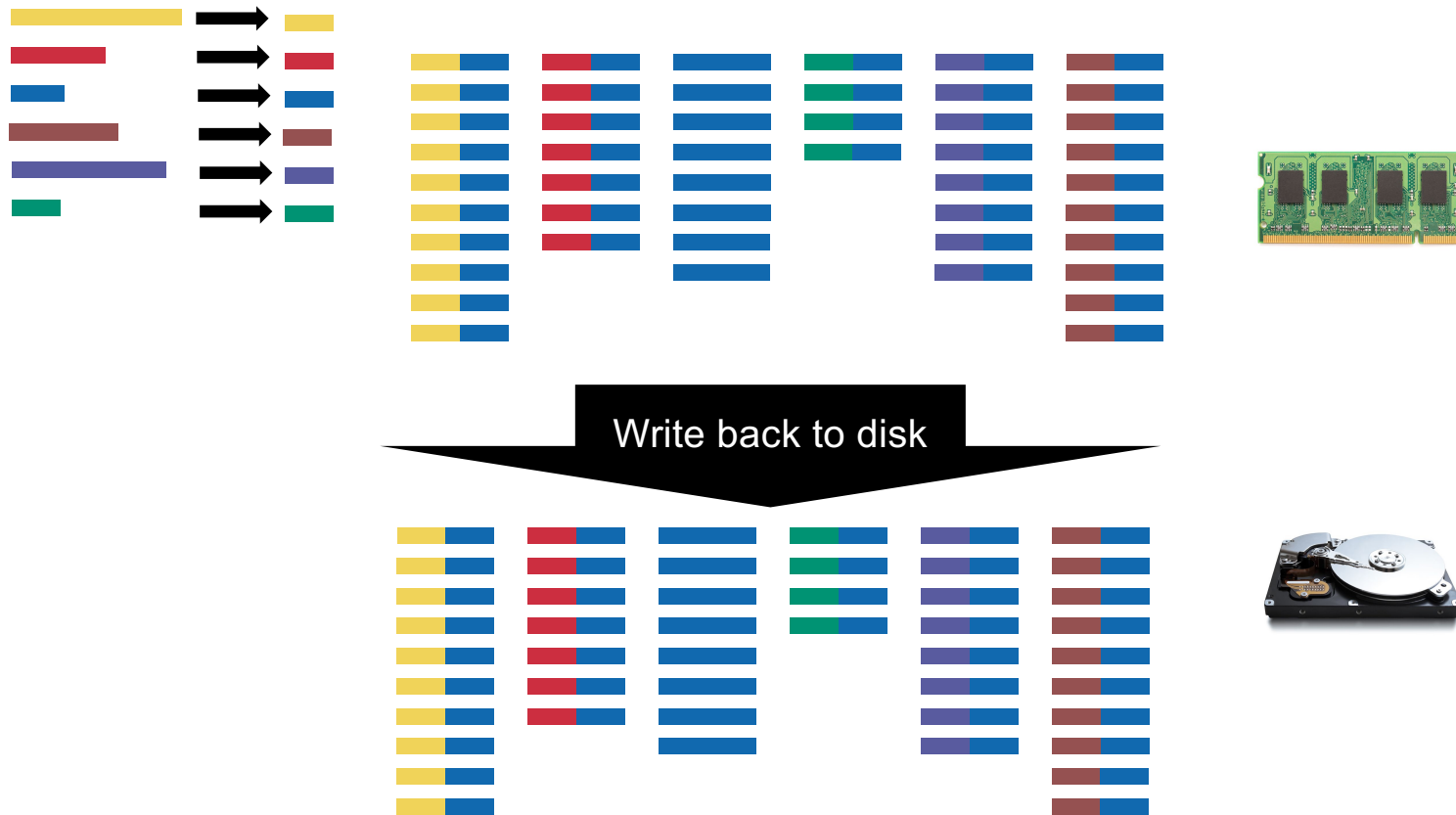
Zooming in: processing a block



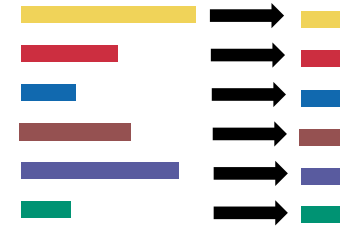
Zooming in: processing a block



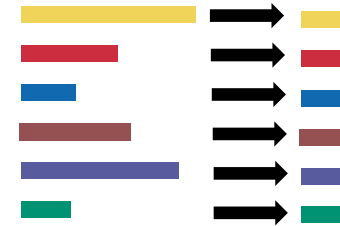
Zooming in: processing a block



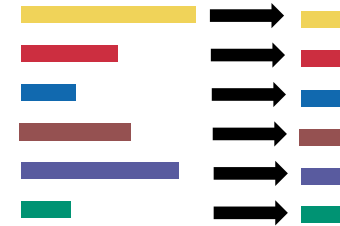
Zooming in: processing a block



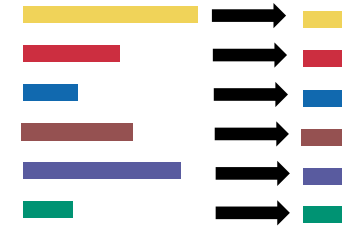
Zooming in: processing a block



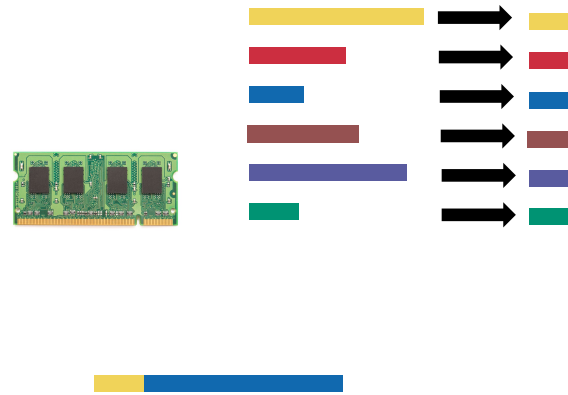
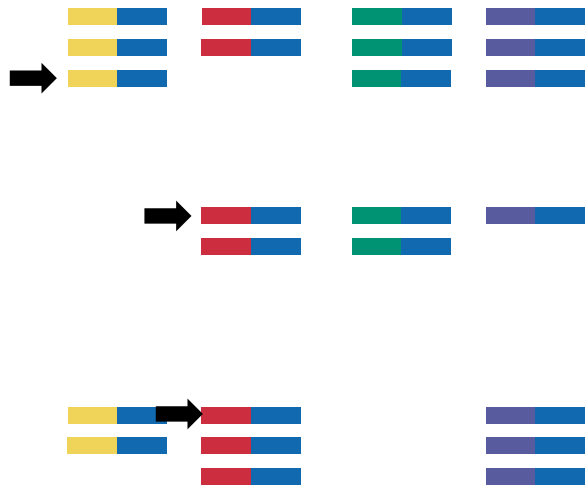
Zooming in: processing a block



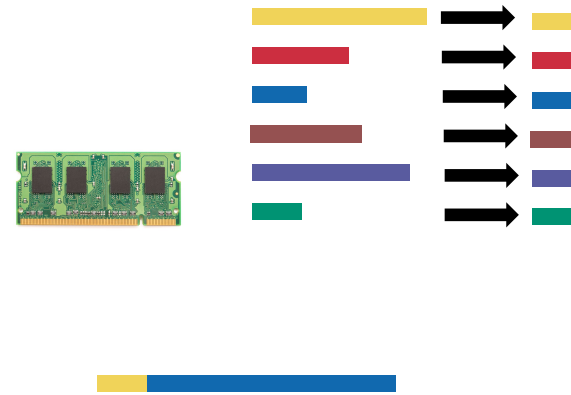
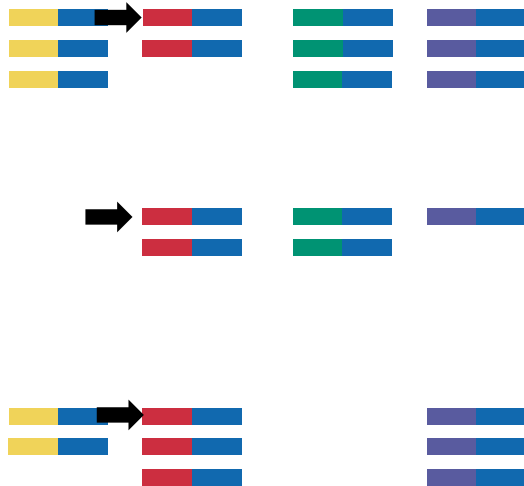
Zooming in: processing a block



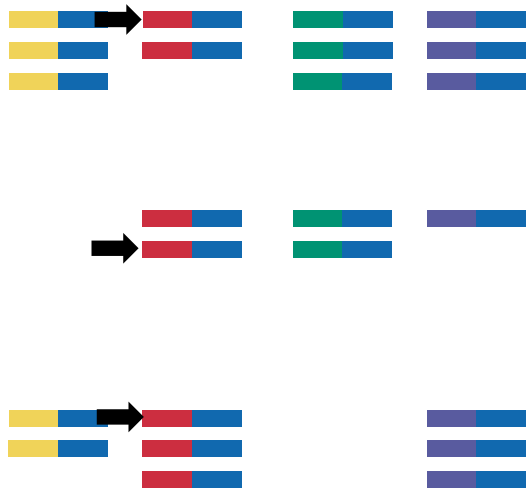
Zooming in: processing a block



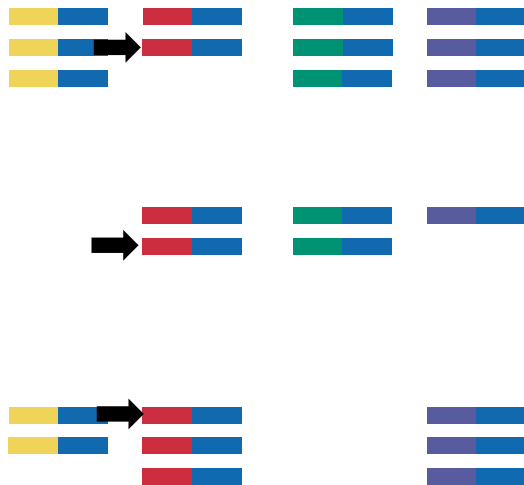
Zooming in: processing a block



Zooming in: processing a block



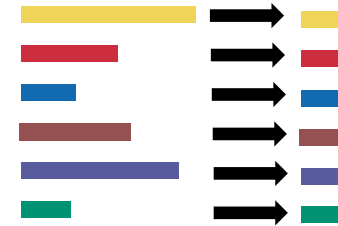
Zooming in: processing a block



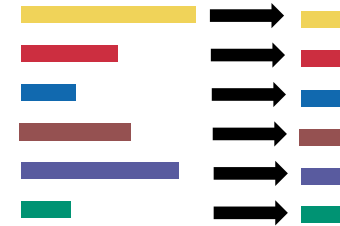
Zooming in: processing a block



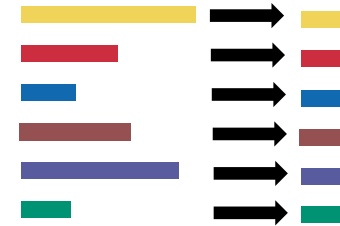
Zooming in: processing a block



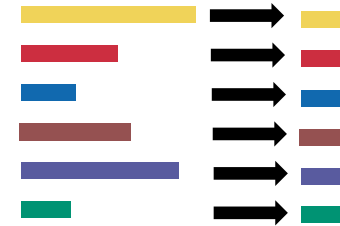
Zooming in: processing a block



Zooming in: processing a block



Zooming in: processing a block



Zooming in: processing a block



Zooming in: processing a block



Complexity

First step

Complexity

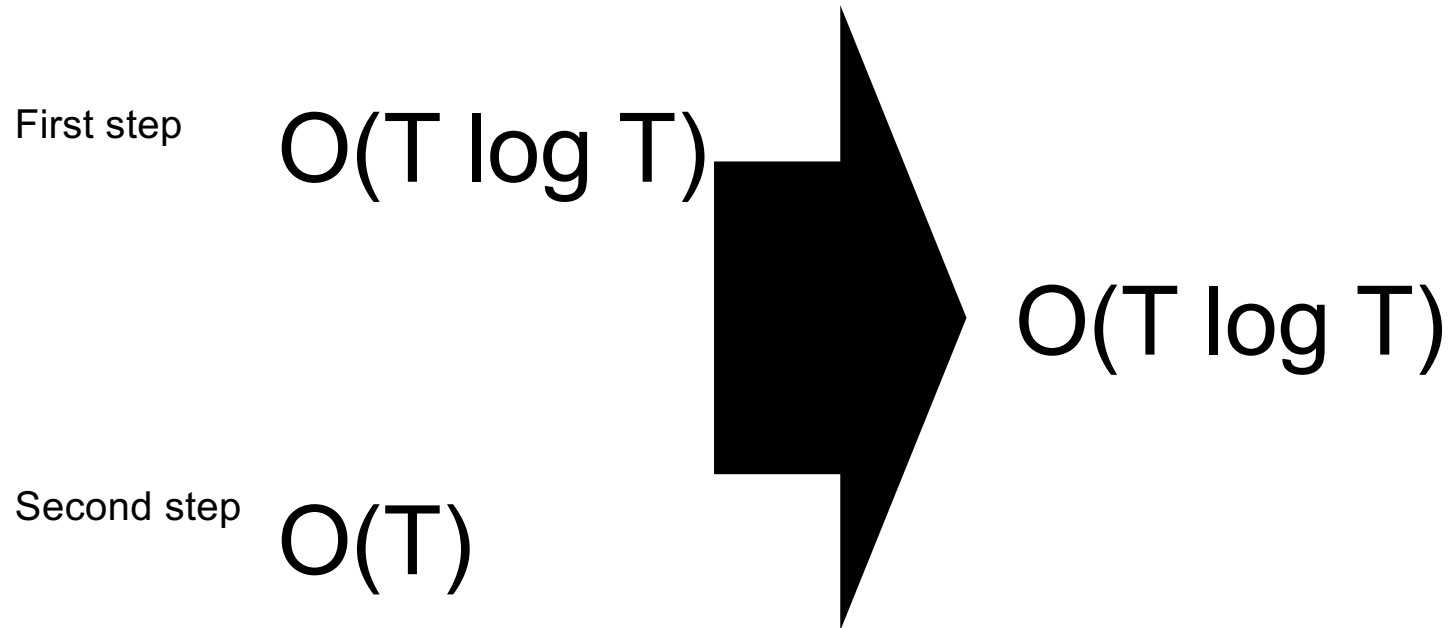
First step $O(T \log T)$

Complexity

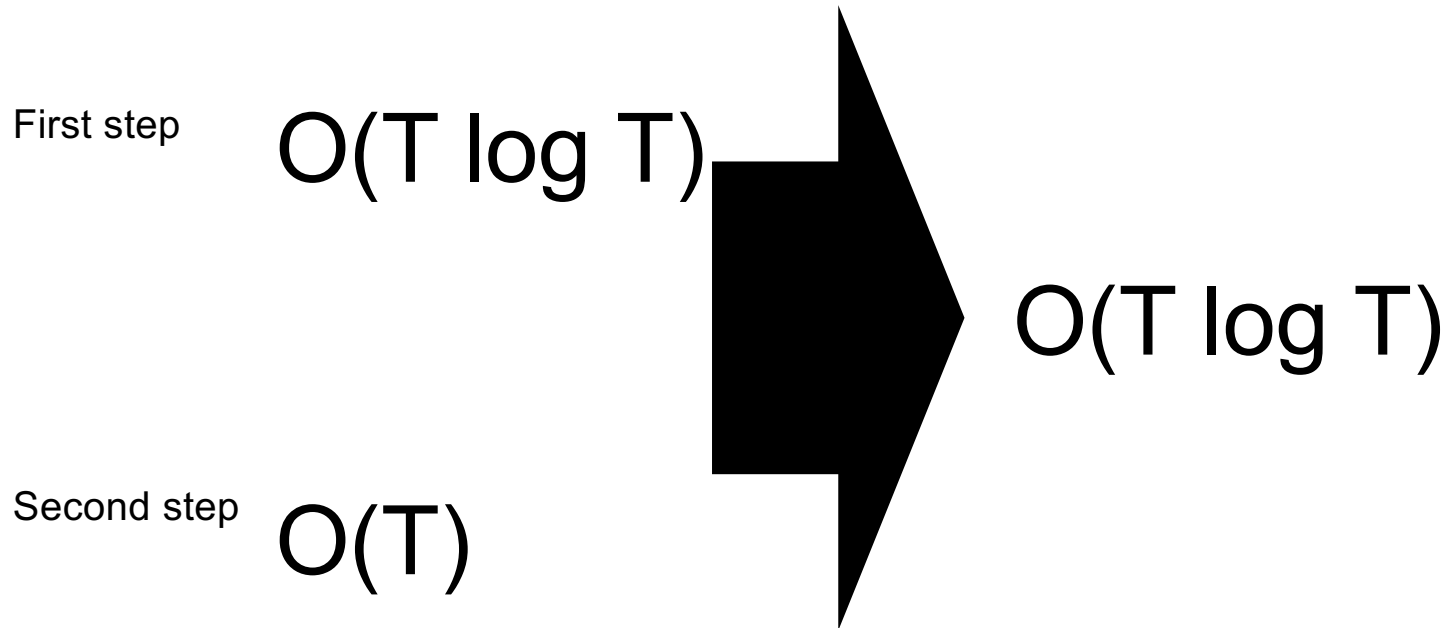
First step $O(T \log T)$

Second step $O(T)$

Complexity



Complexity

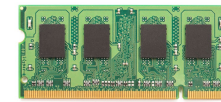
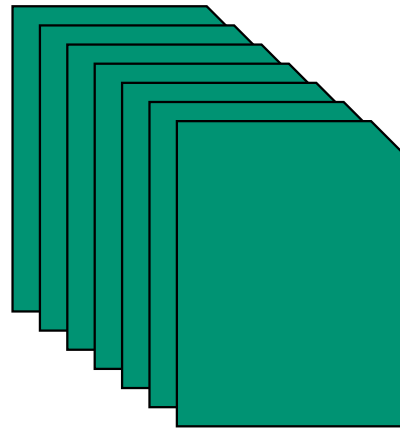


in practice "feels" like $O(T)$

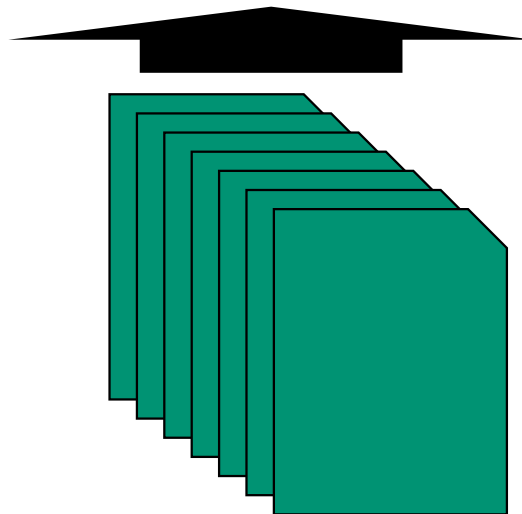
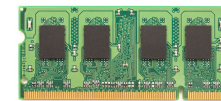


Single-Pass In-Memory Indexing

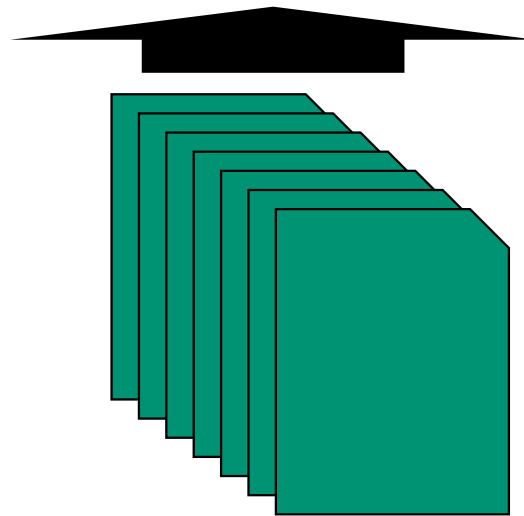
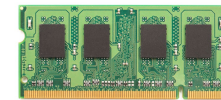
BSBI: processing a block



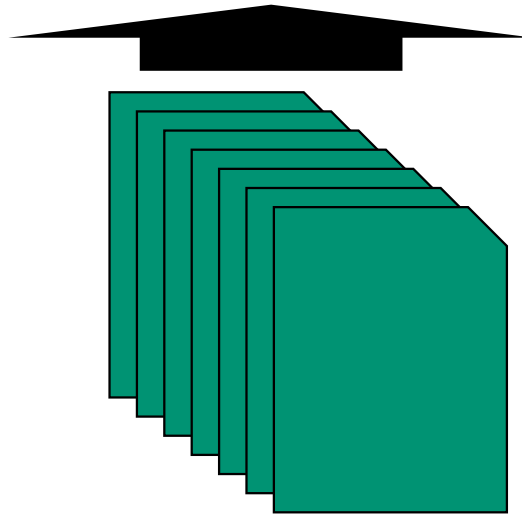
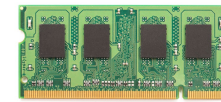
SPIMI: processing a block



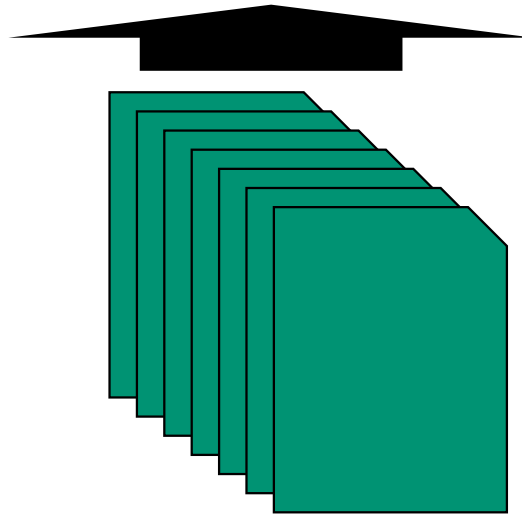
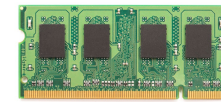
SPIMI: processing a block



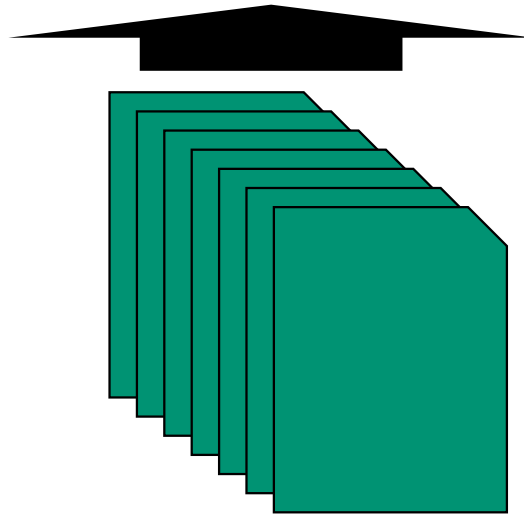
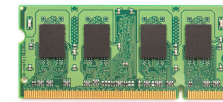
SPIMI: processing a block



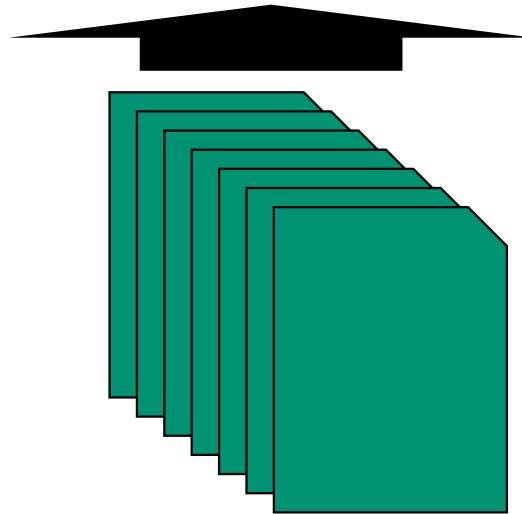
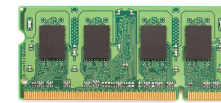
SPIMI: processing a block



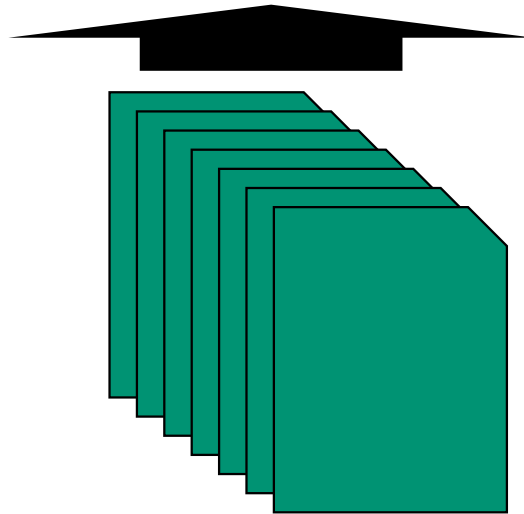
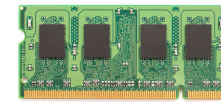
SPIMI: processing a block



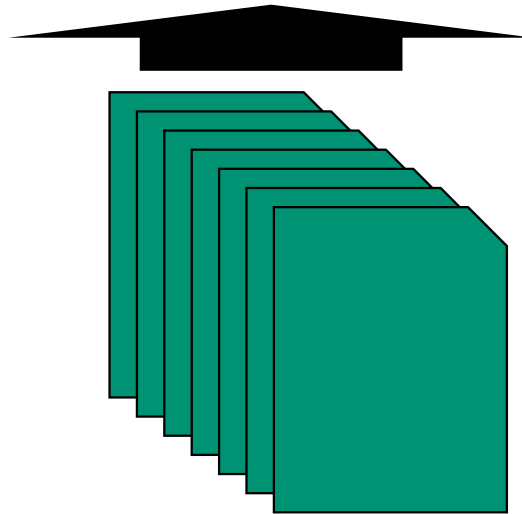
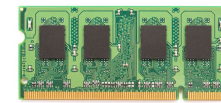
SPIMI: processing a block



SPIMI: processing a block



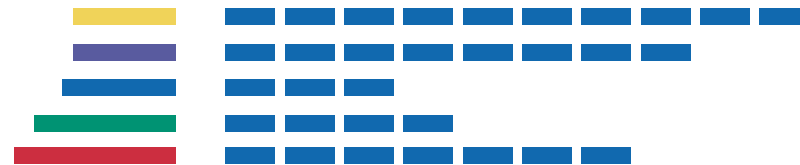
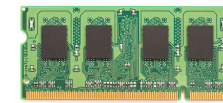
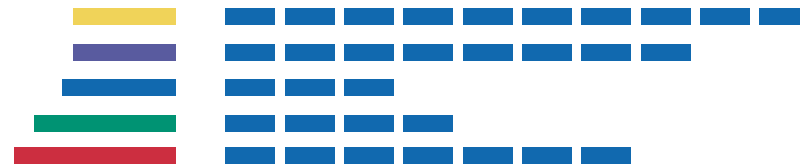
SPIMI: processing a block



Sort by term

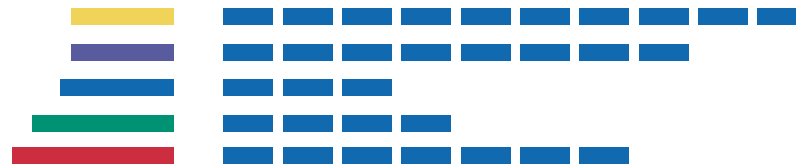
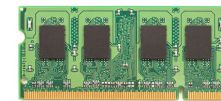


Write back to disk



Write back to disk

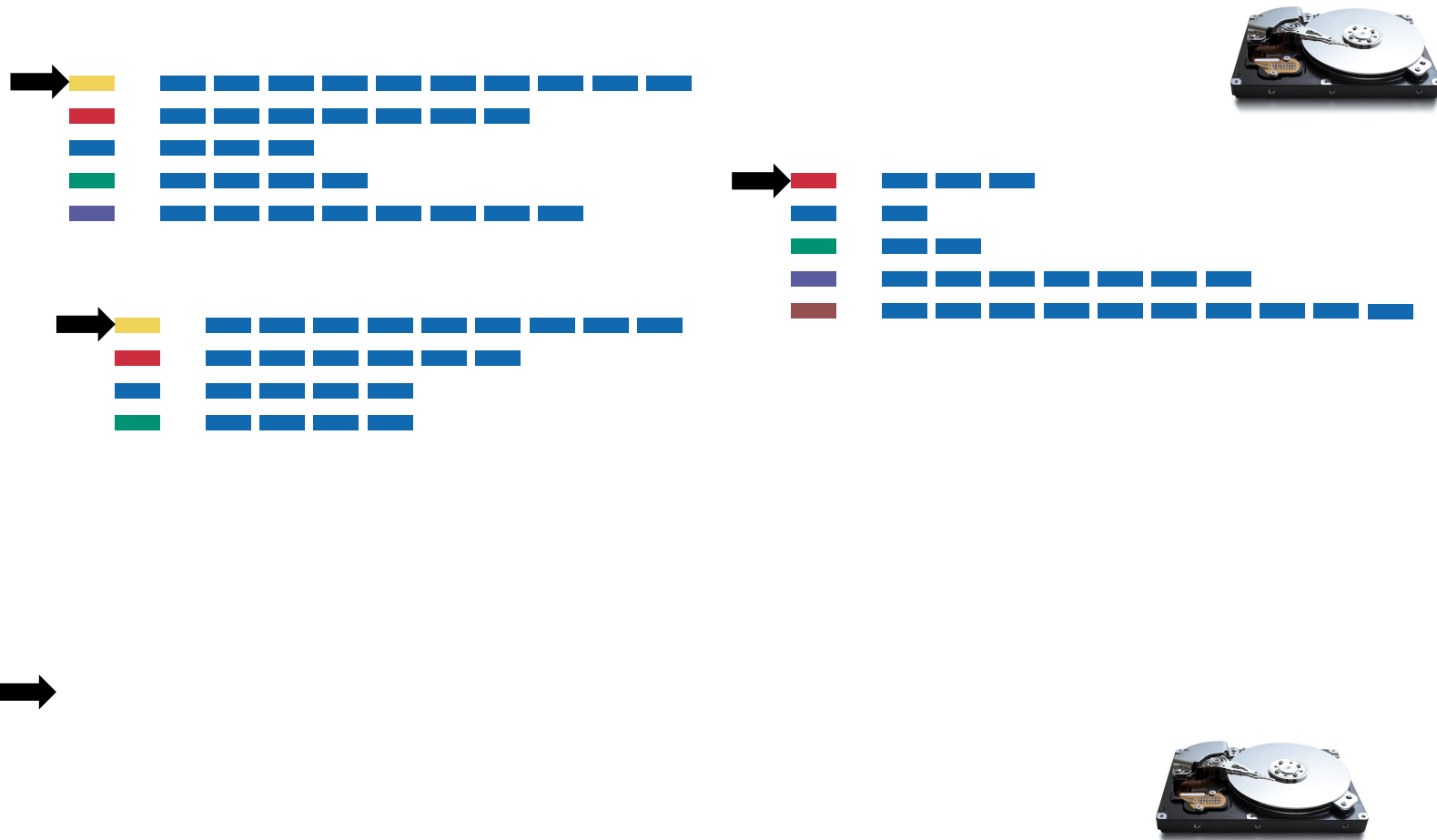
No term-termID mapping held in memory!



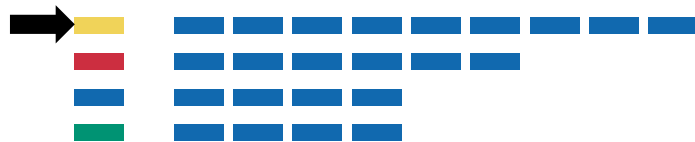
Zooming in: final merge



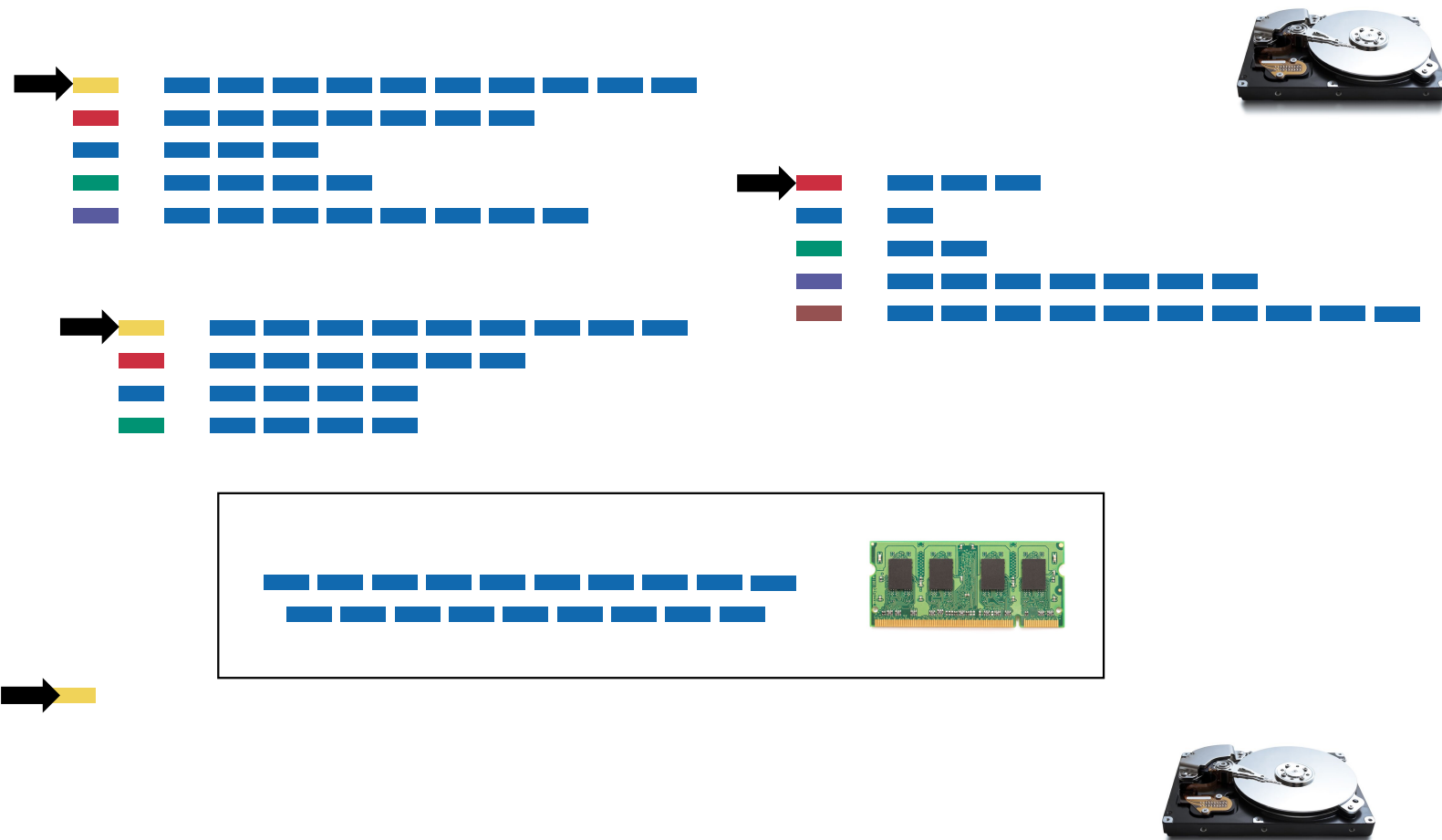
Zooming in: final merge



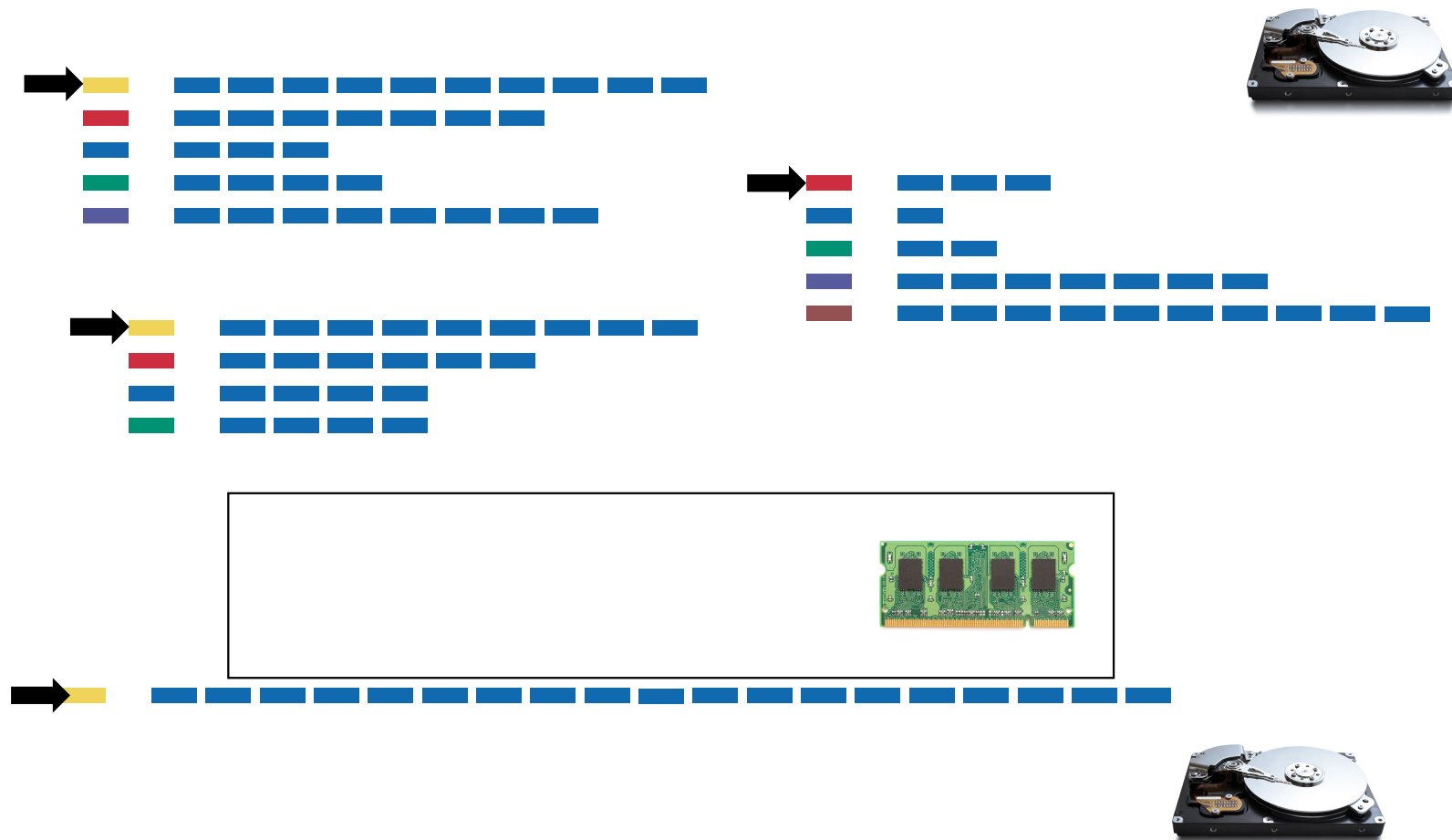
Zooming in: final merge



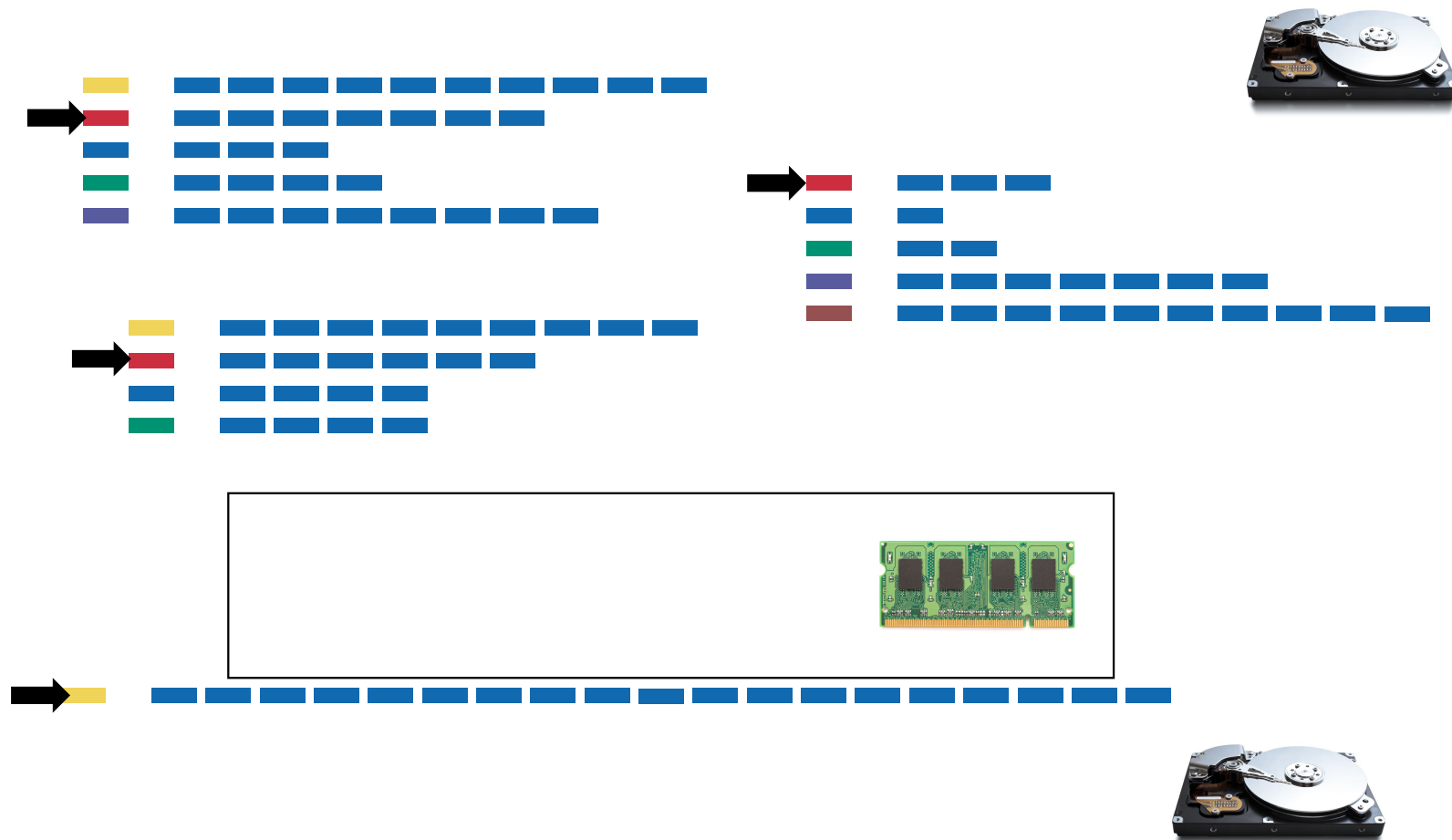
Zooming in: final merge



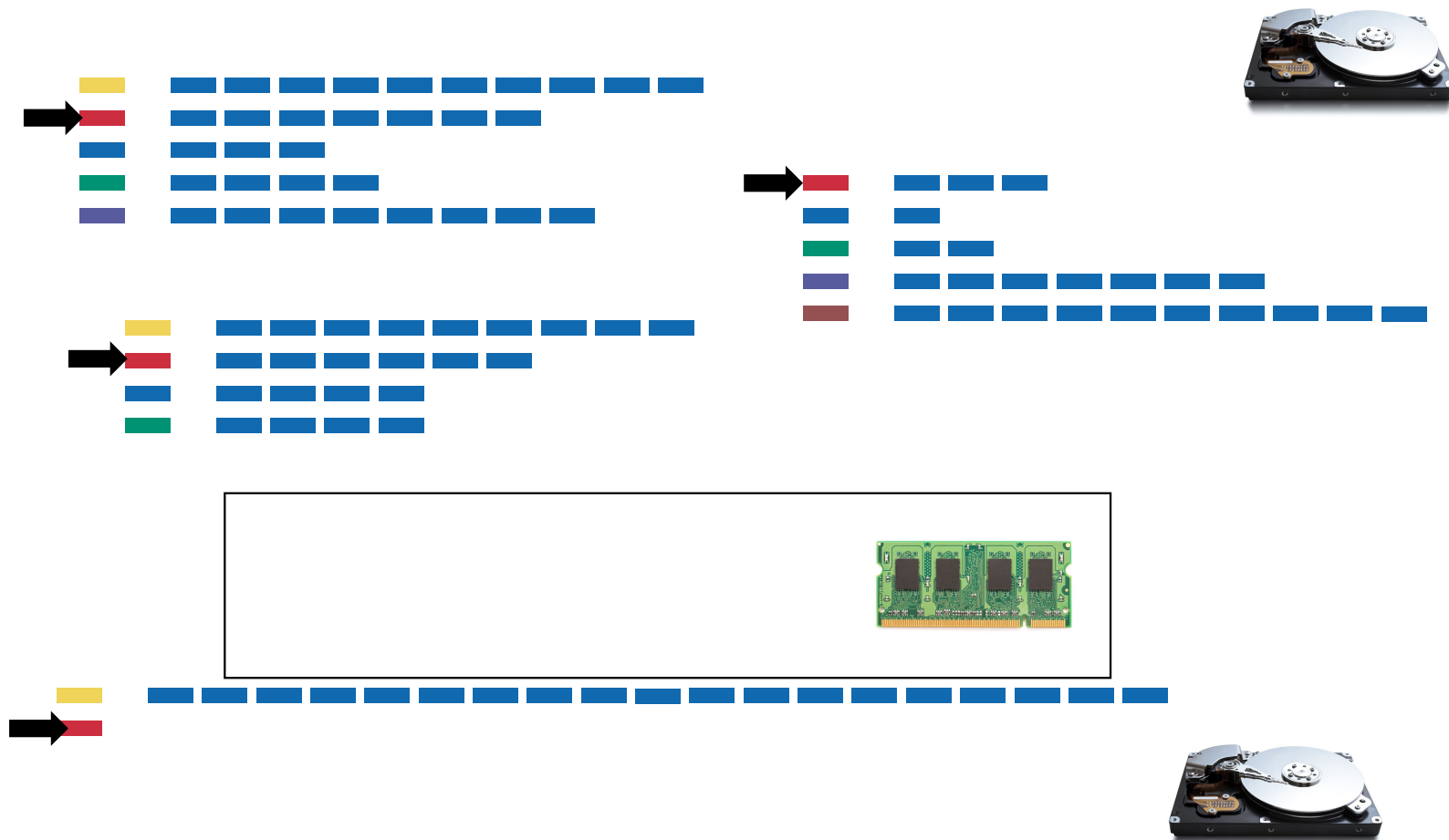
Zooming in: final merge



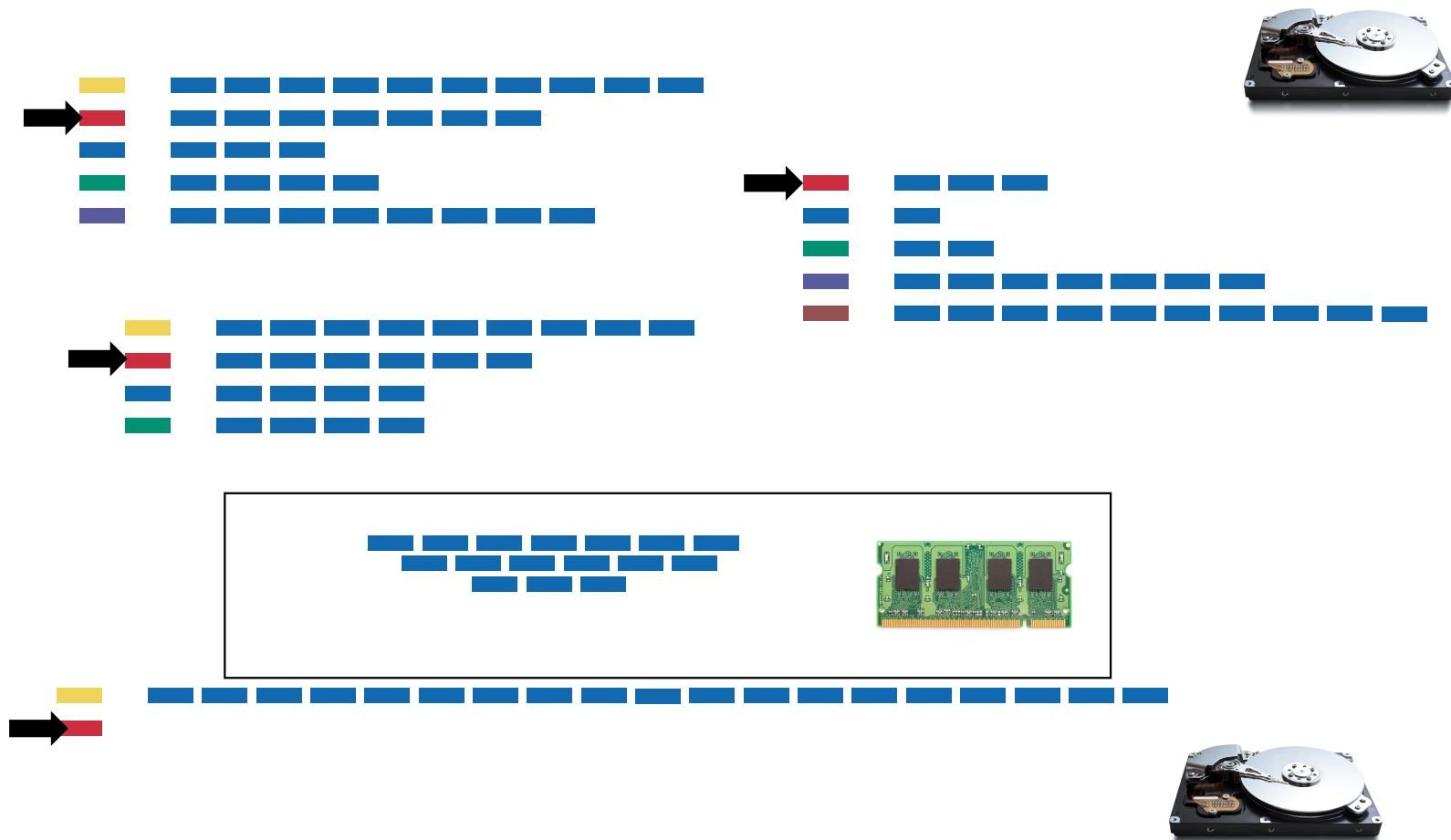
Zooming in: final merge



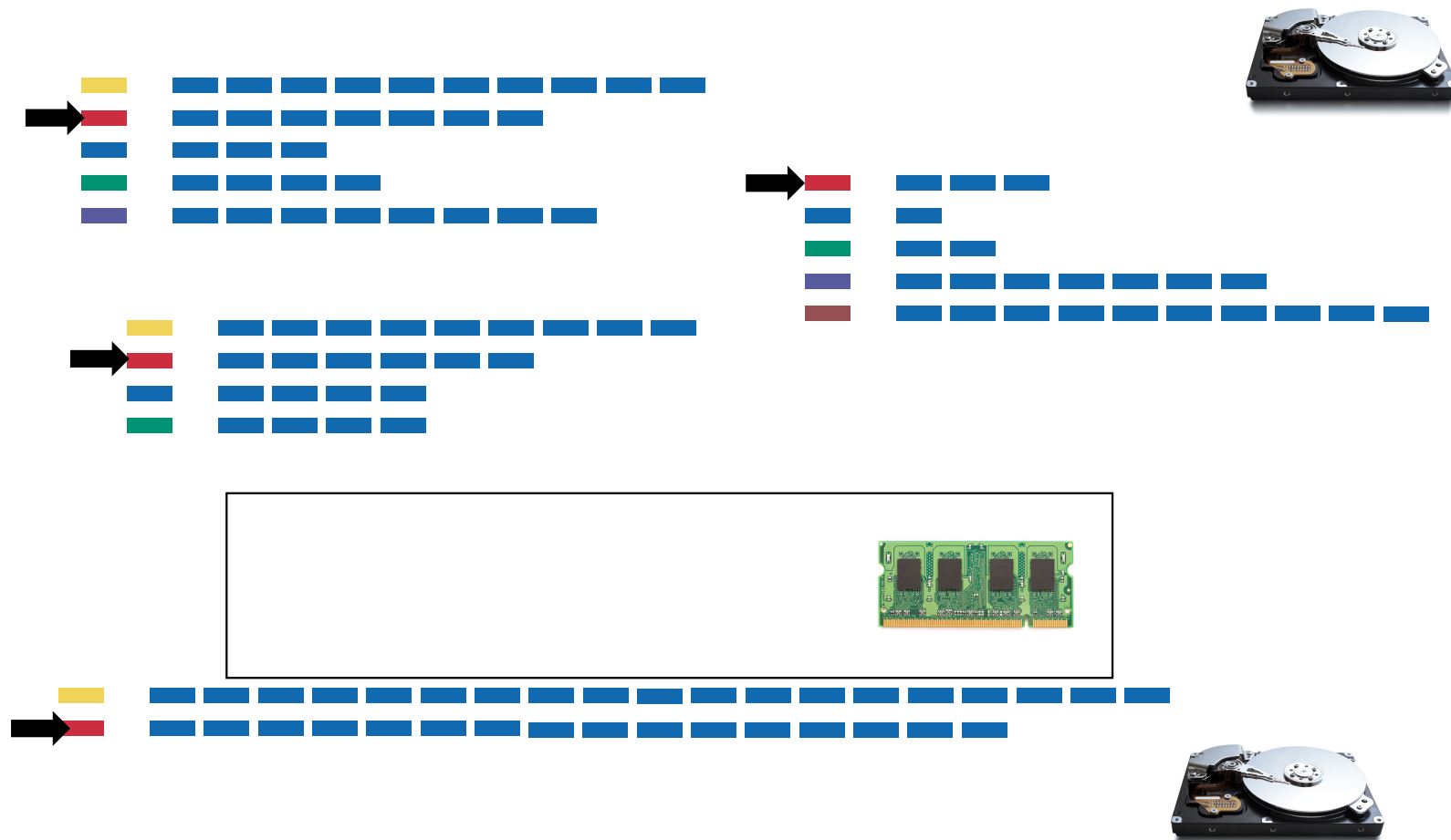
Zooming in: final merge



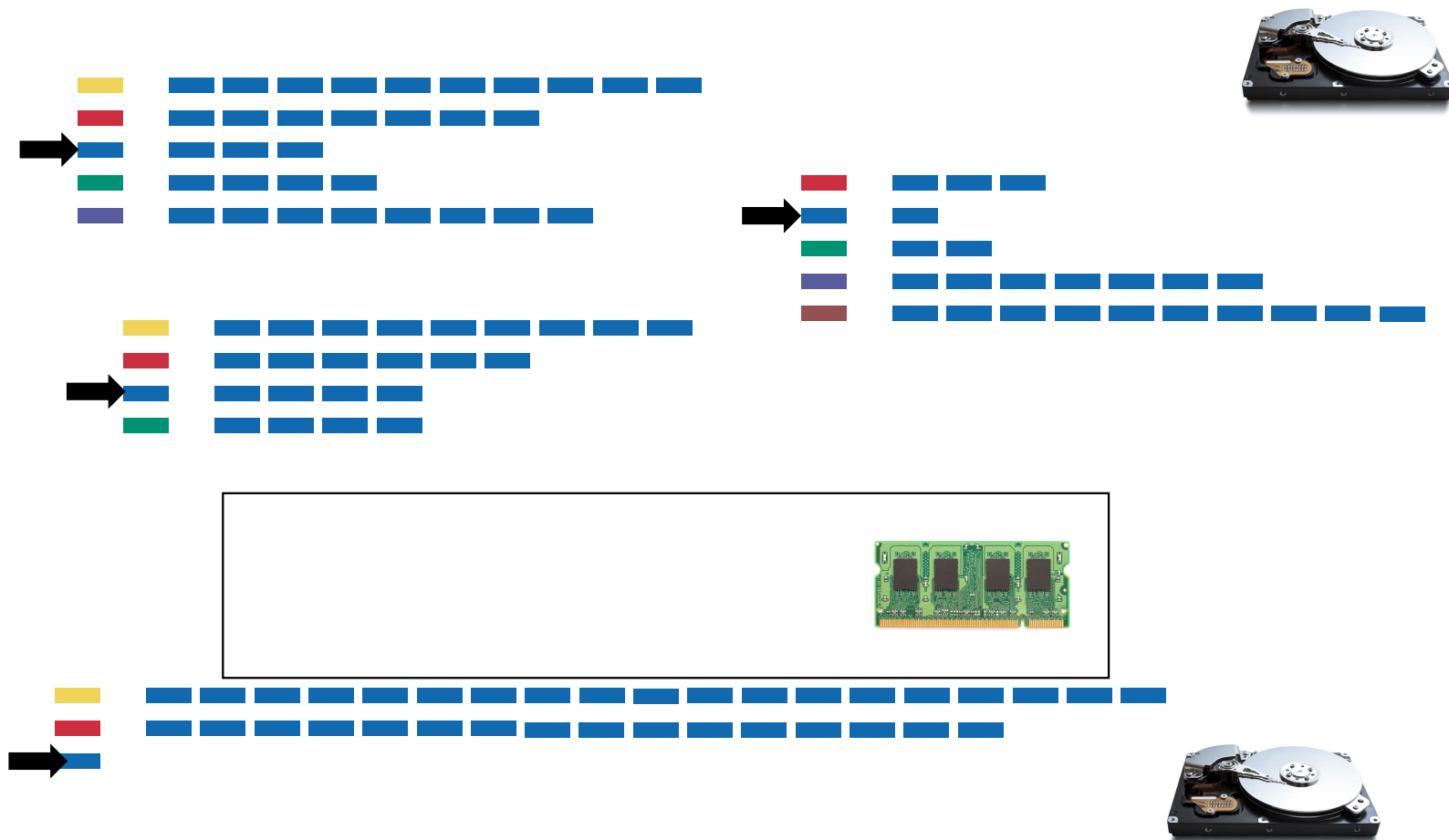
Zooming in: final merge



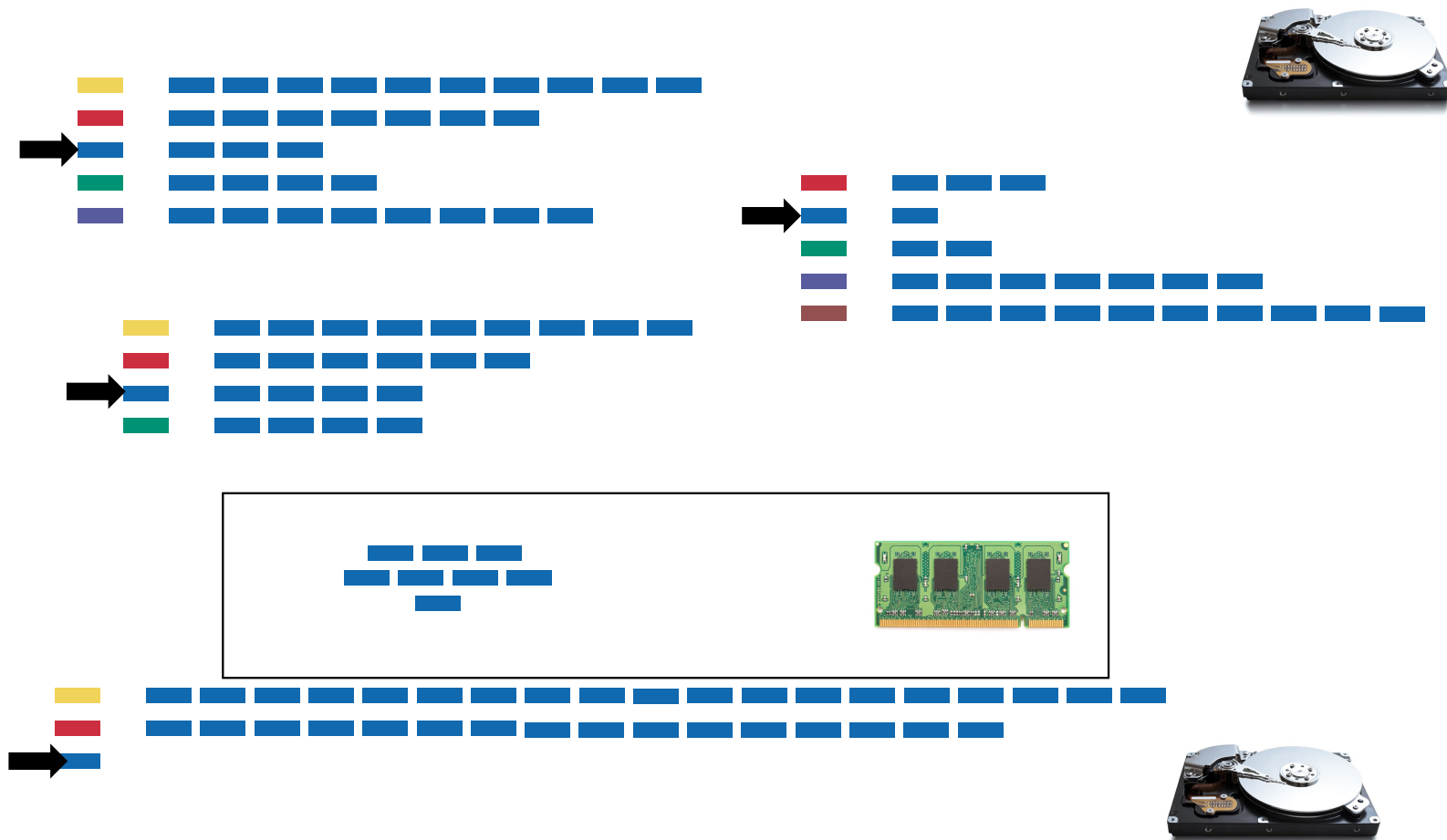
Zooming in: final merge



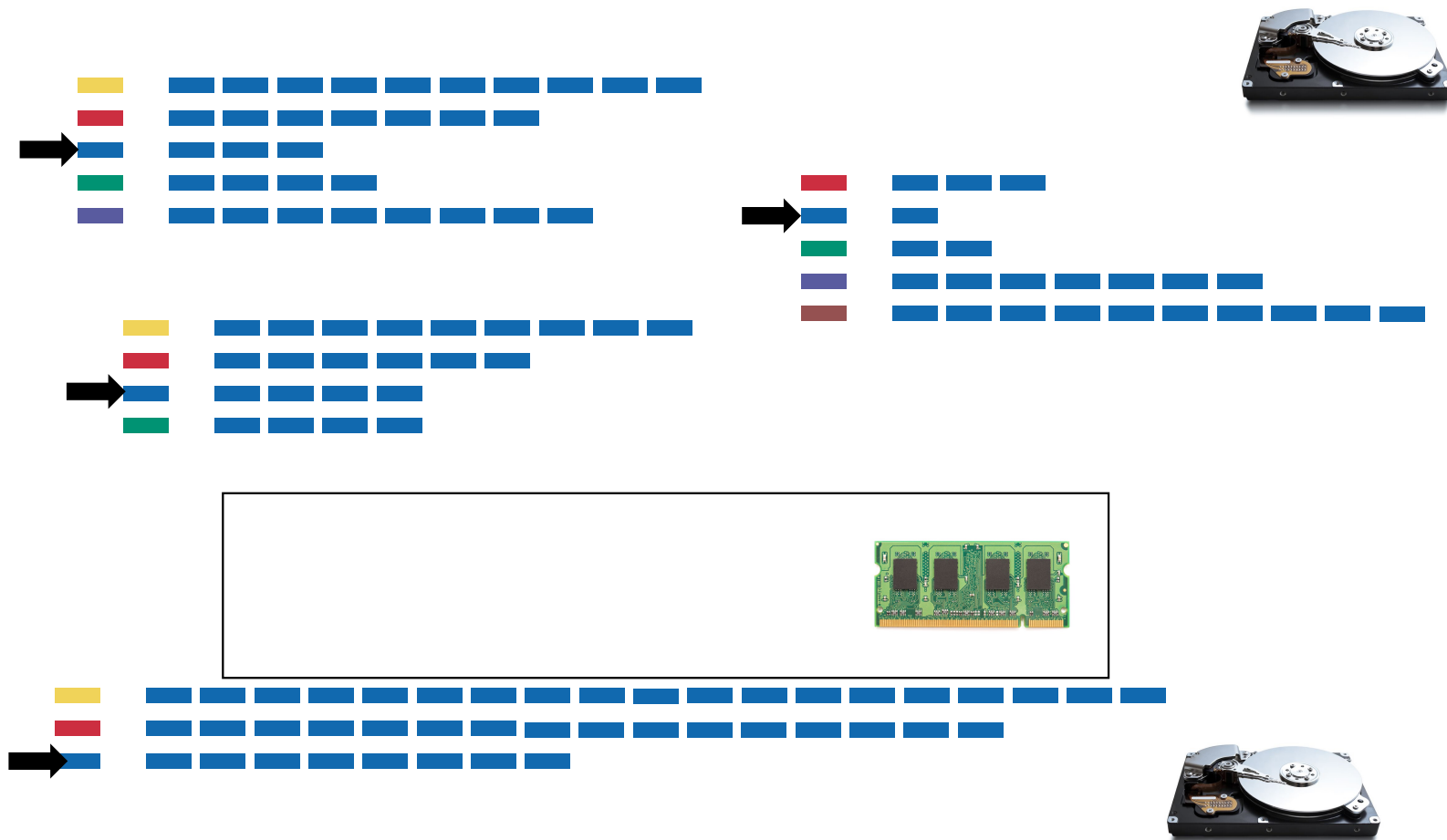
Zooming in: final merge



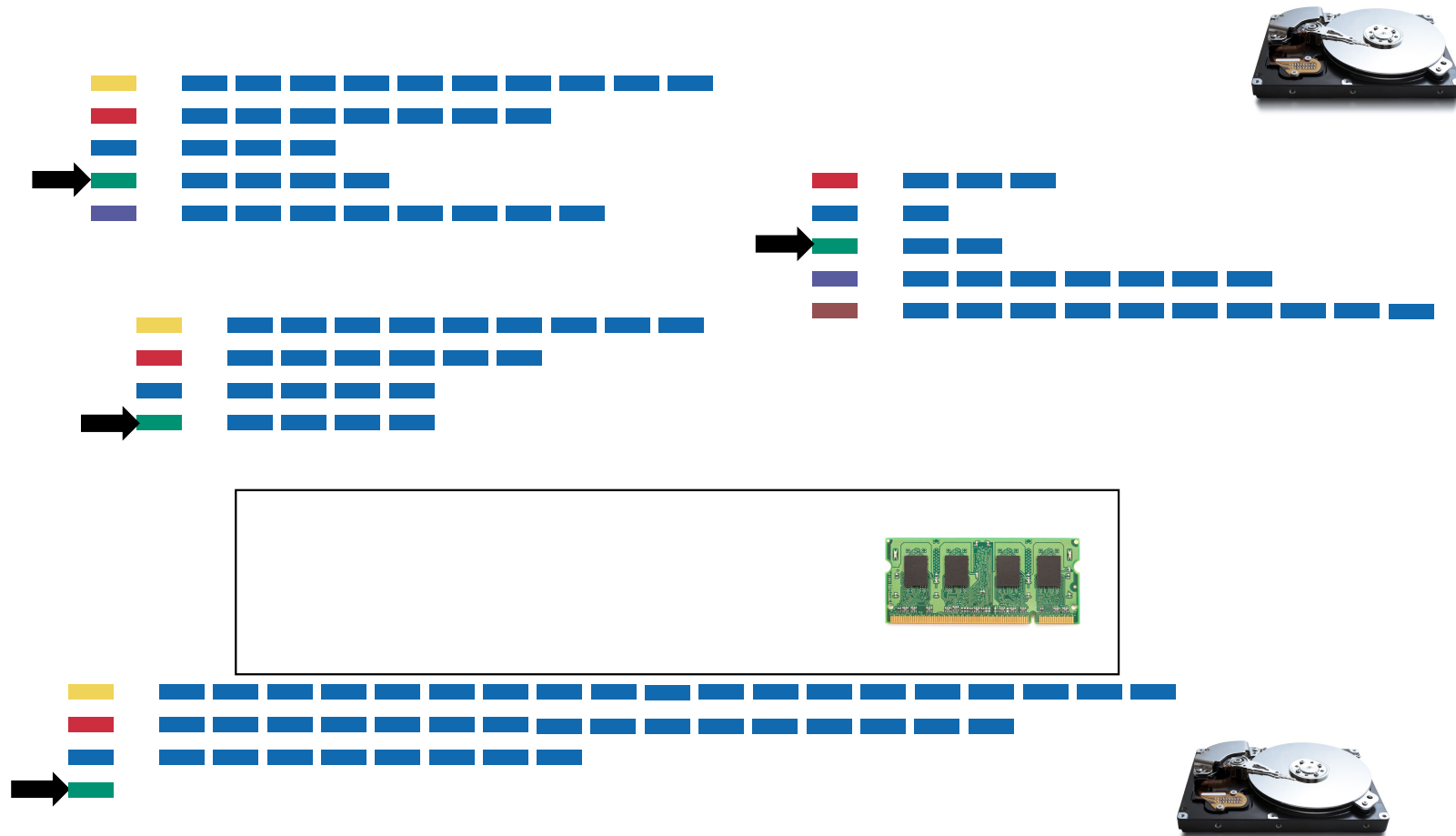
Zooming in: final merge



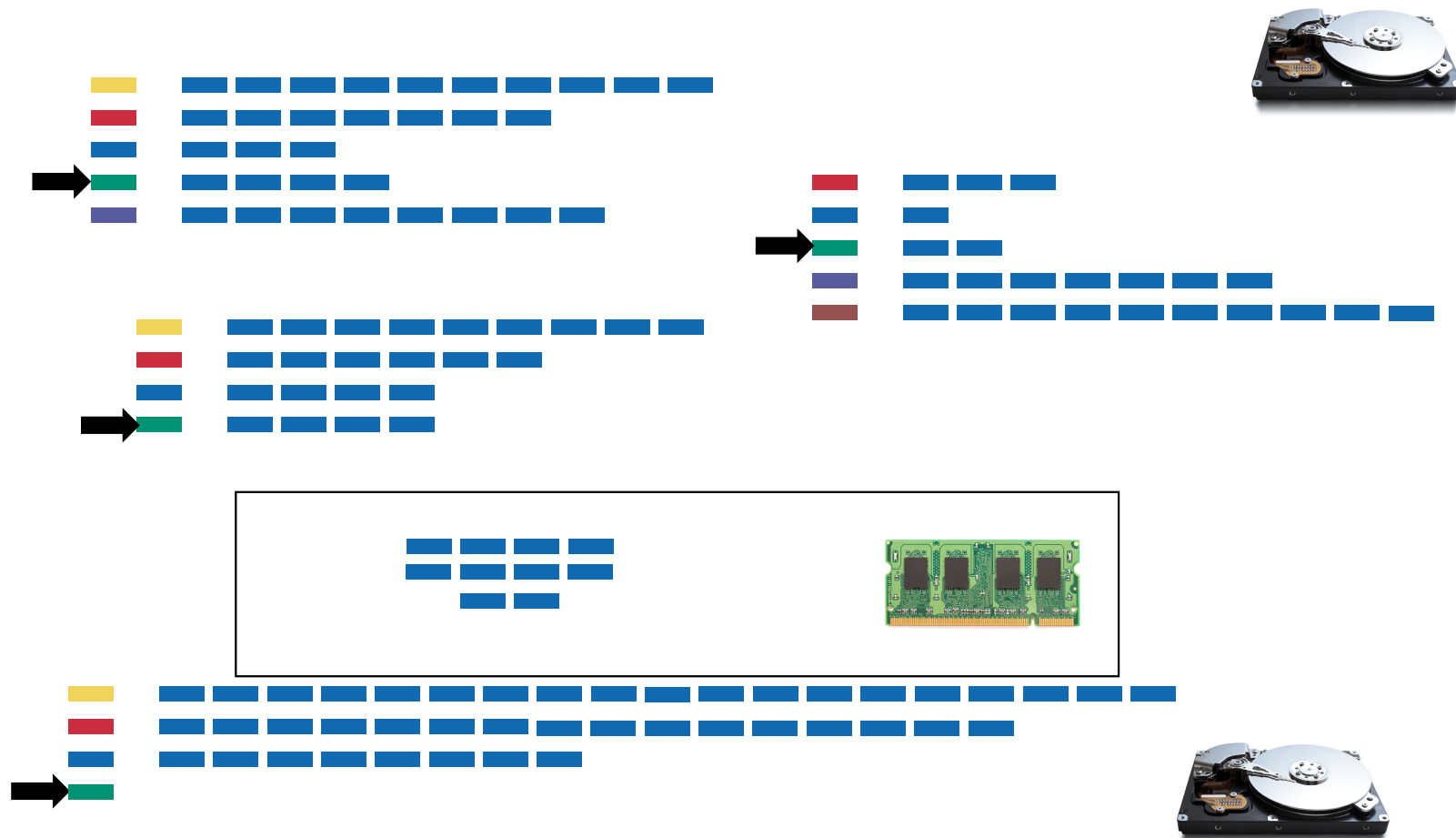
Zooming in: final merge



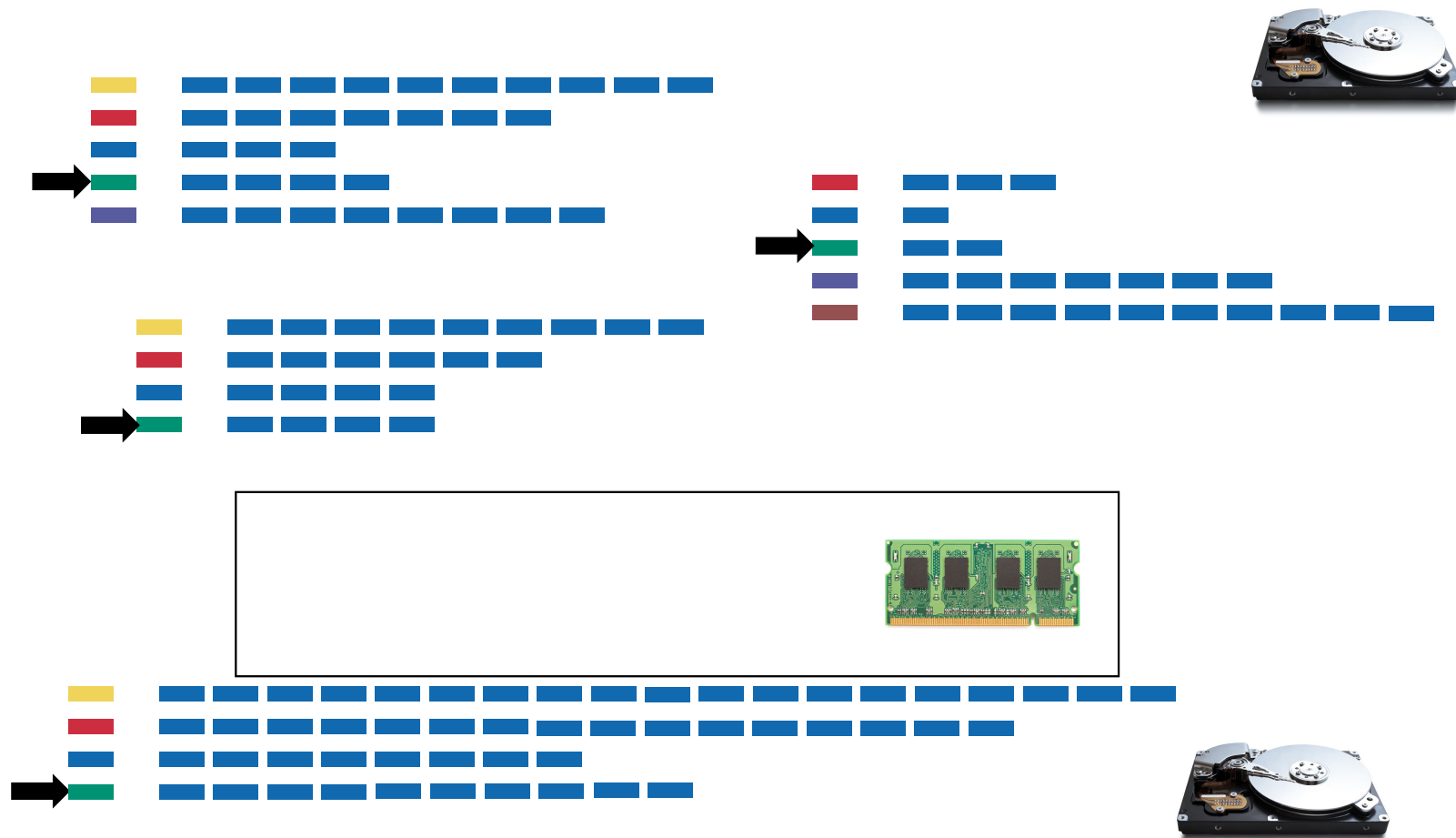
Zooming in: final merge



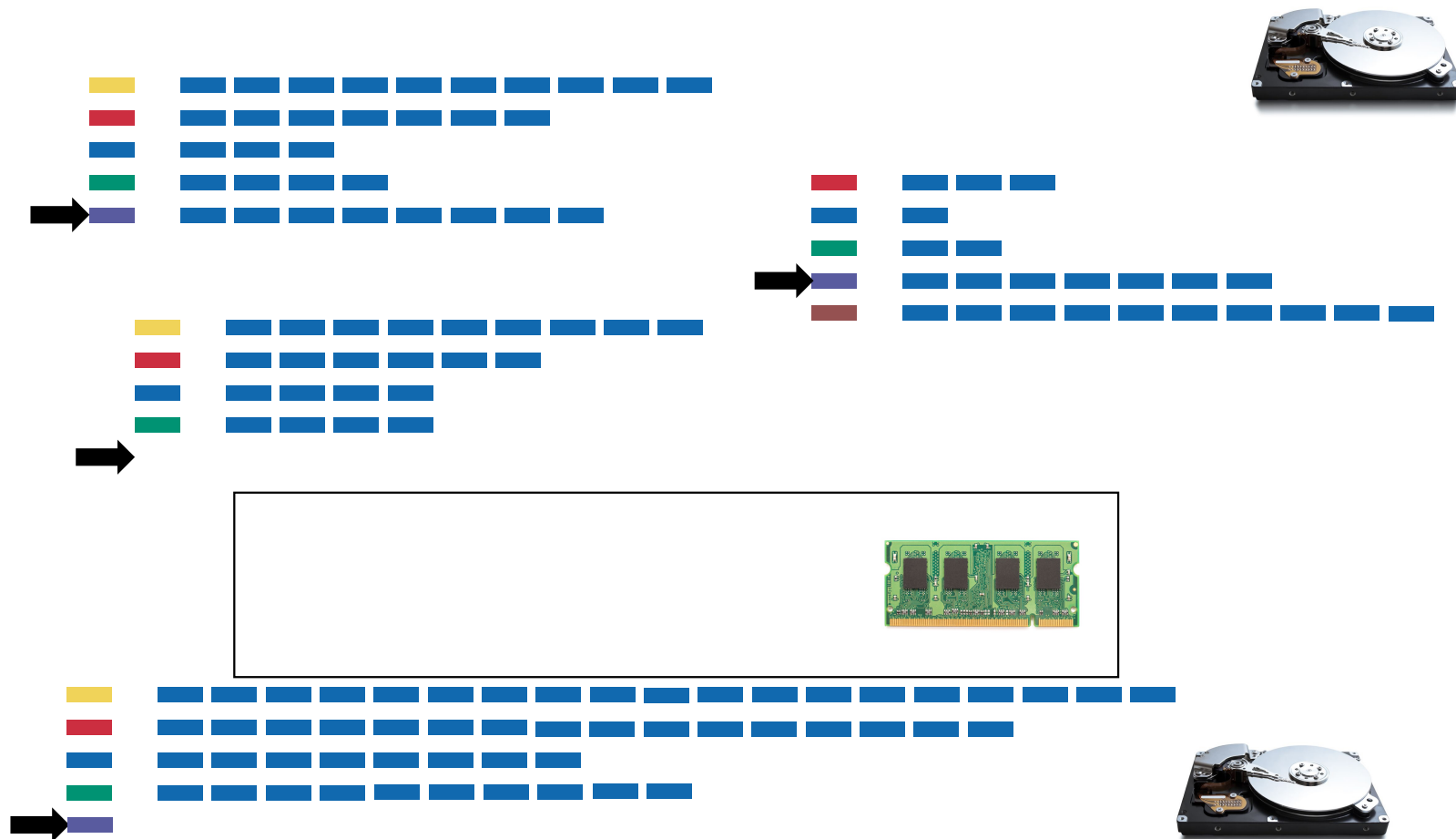
Zooming in: final merge



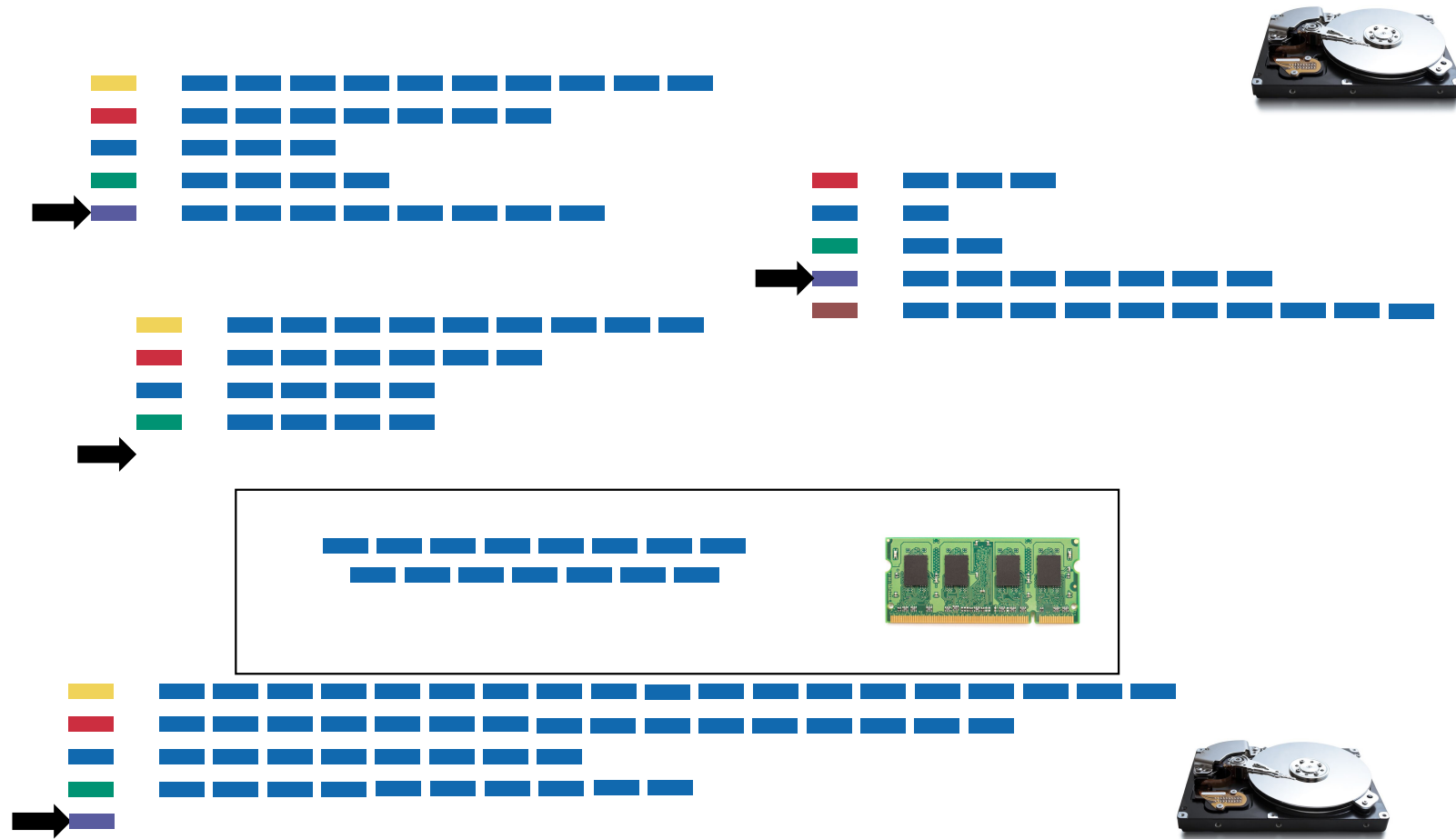
Zooming in: final merge



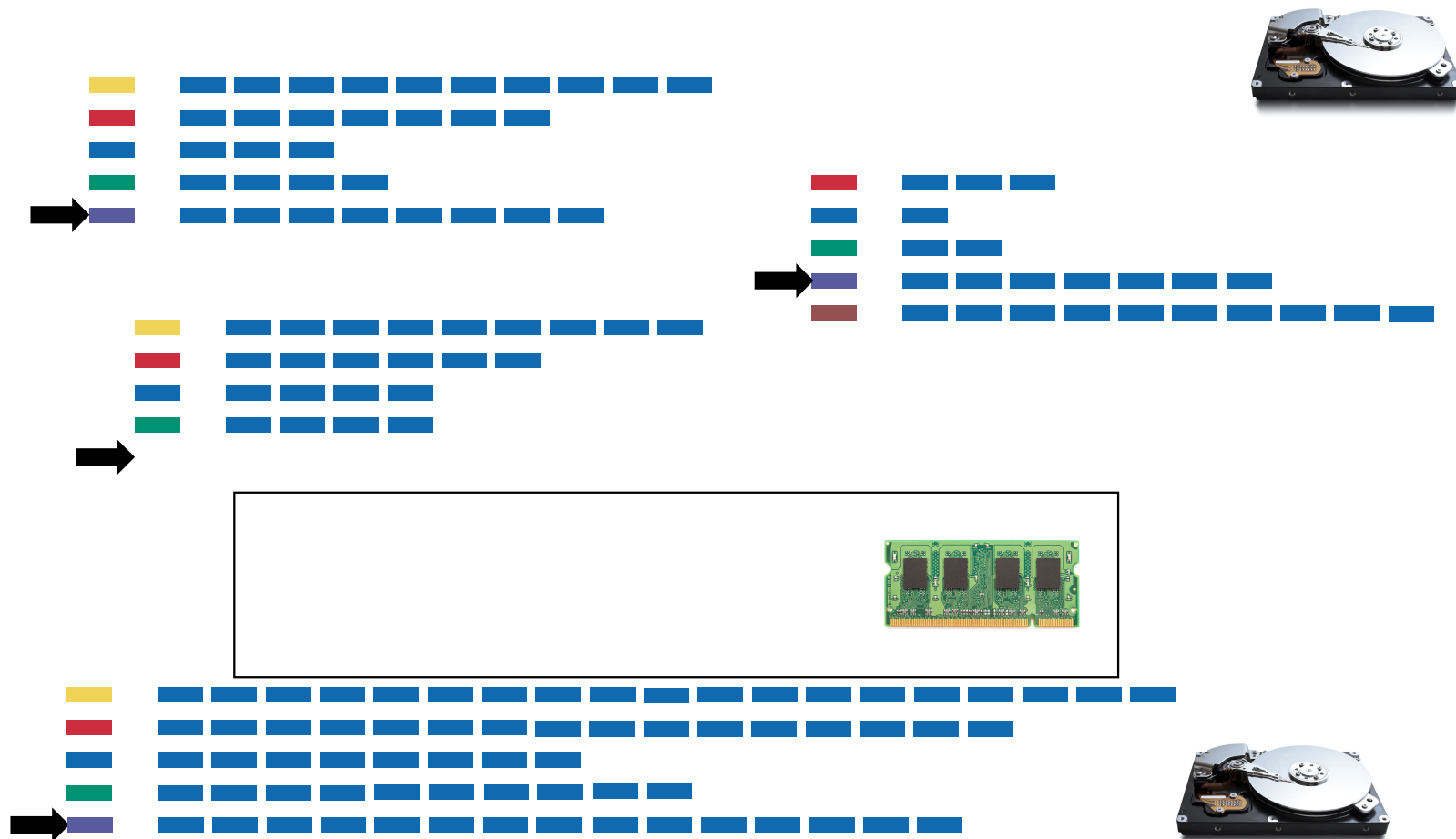
Zooming in: final merge



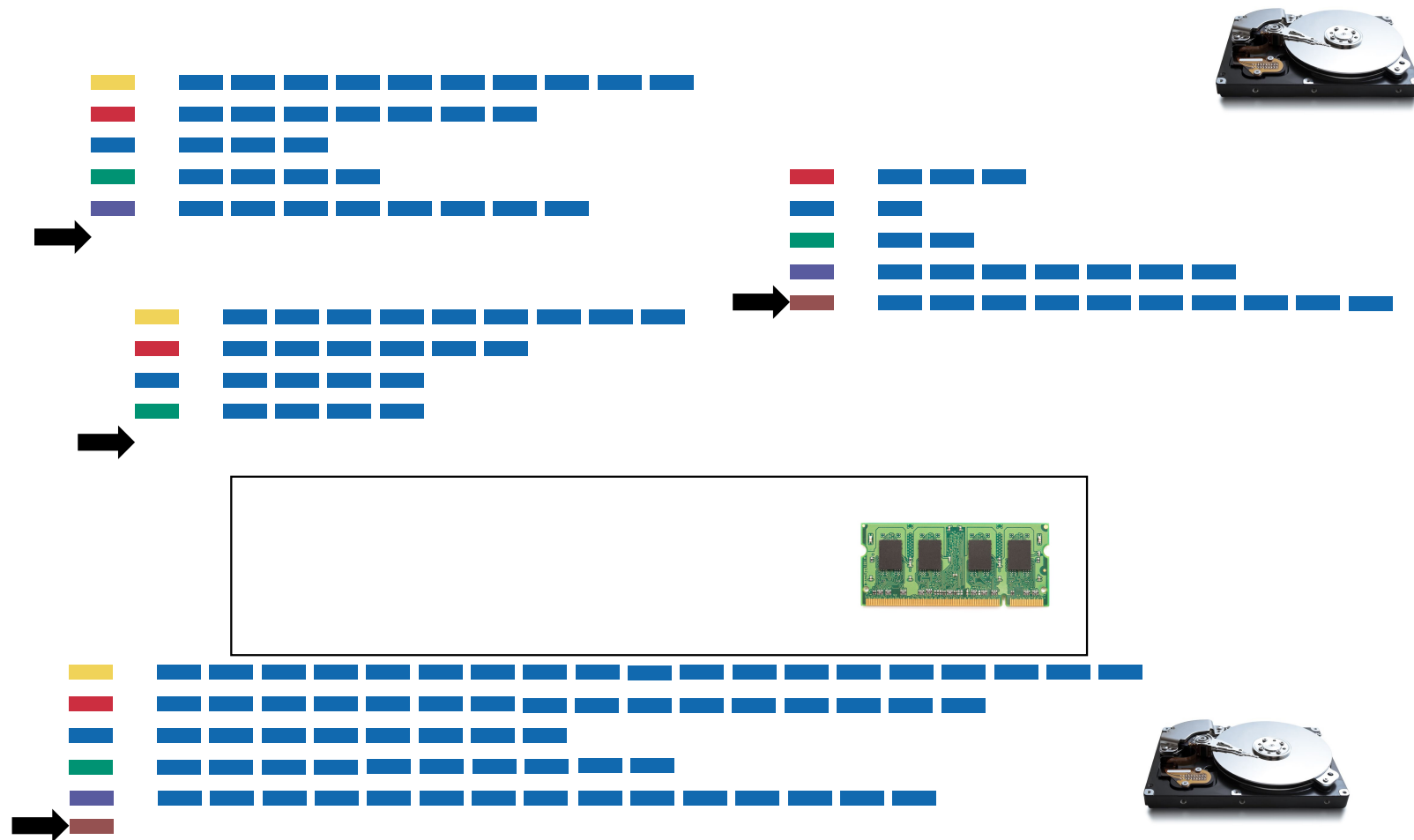
Zooming in: final merge



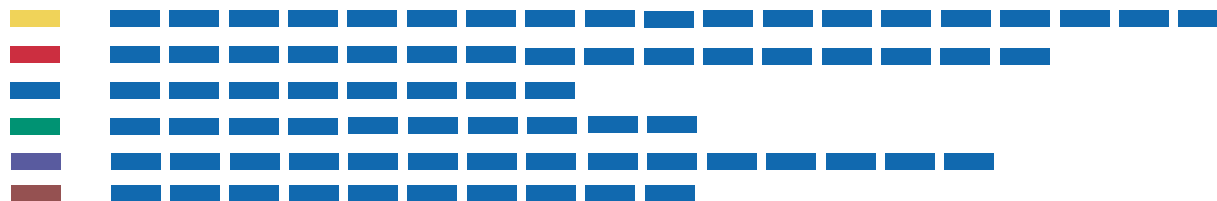
Zooming in: final merge



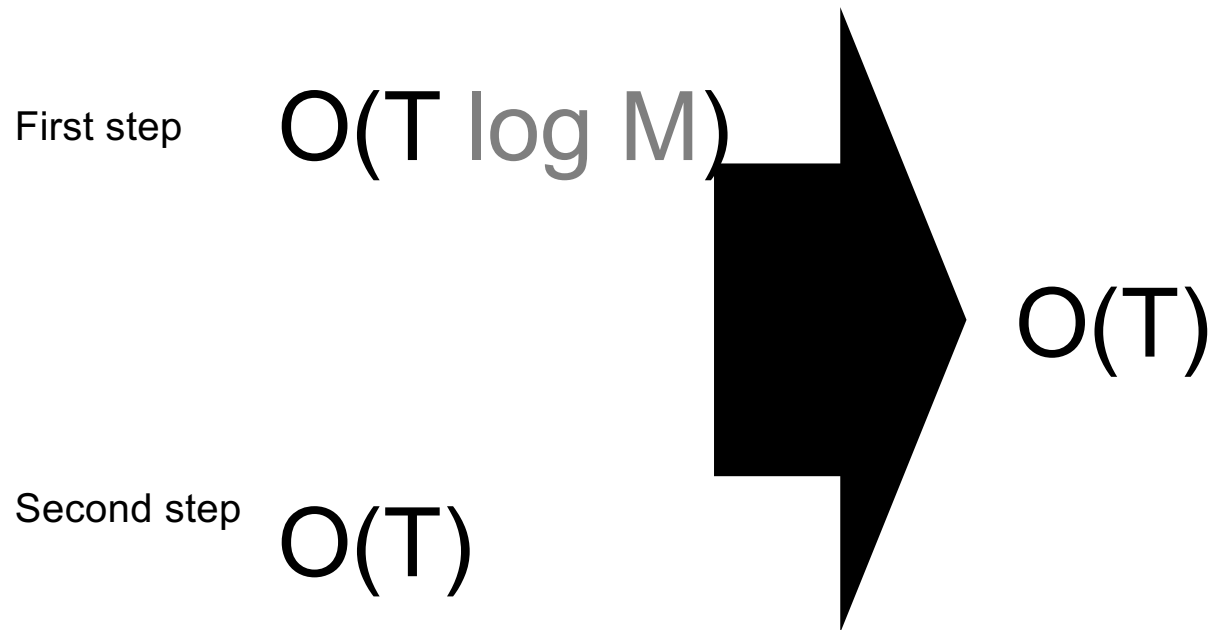
Zooming in: final merge



Zooming in: final merge



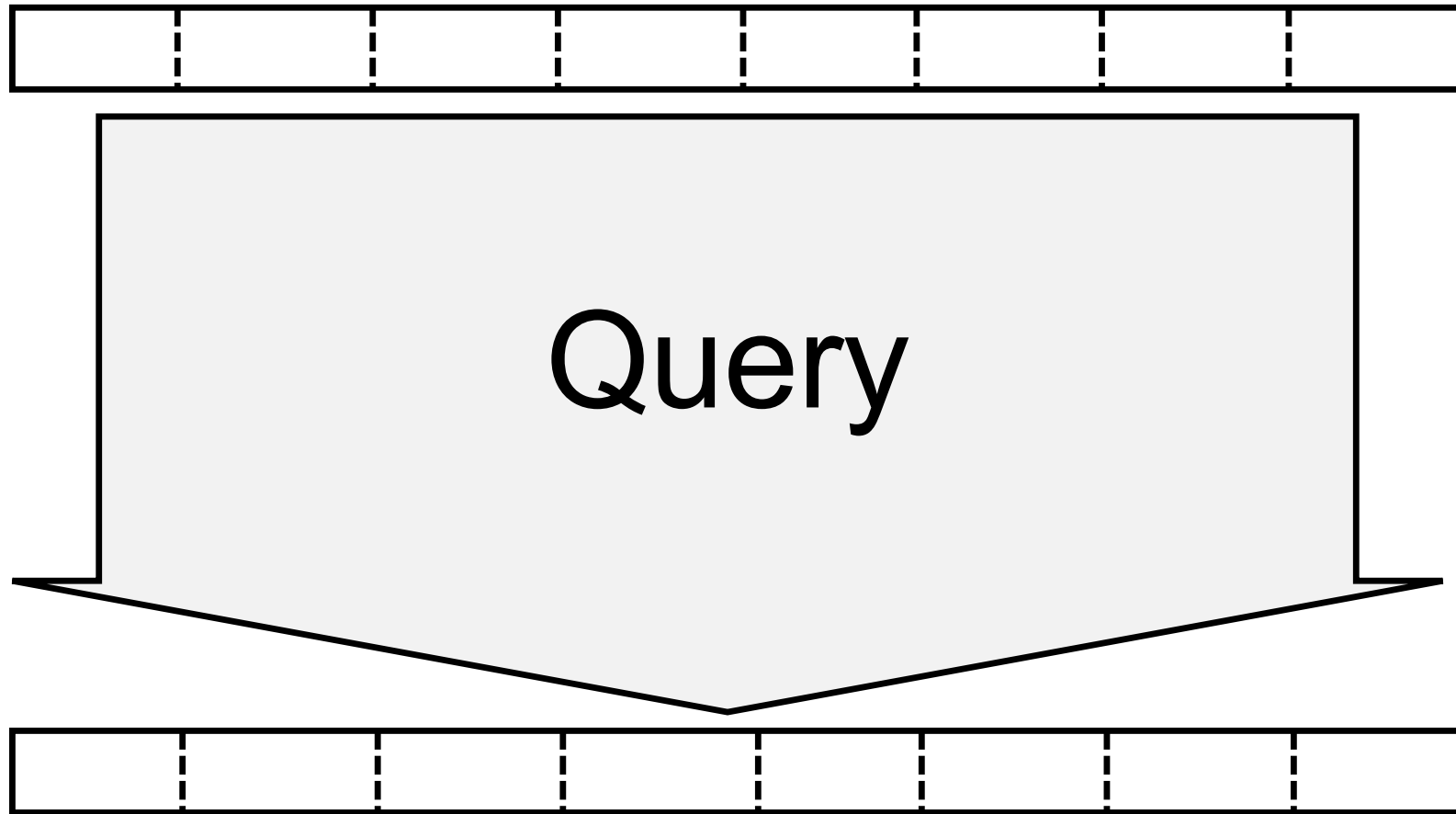
Complexity



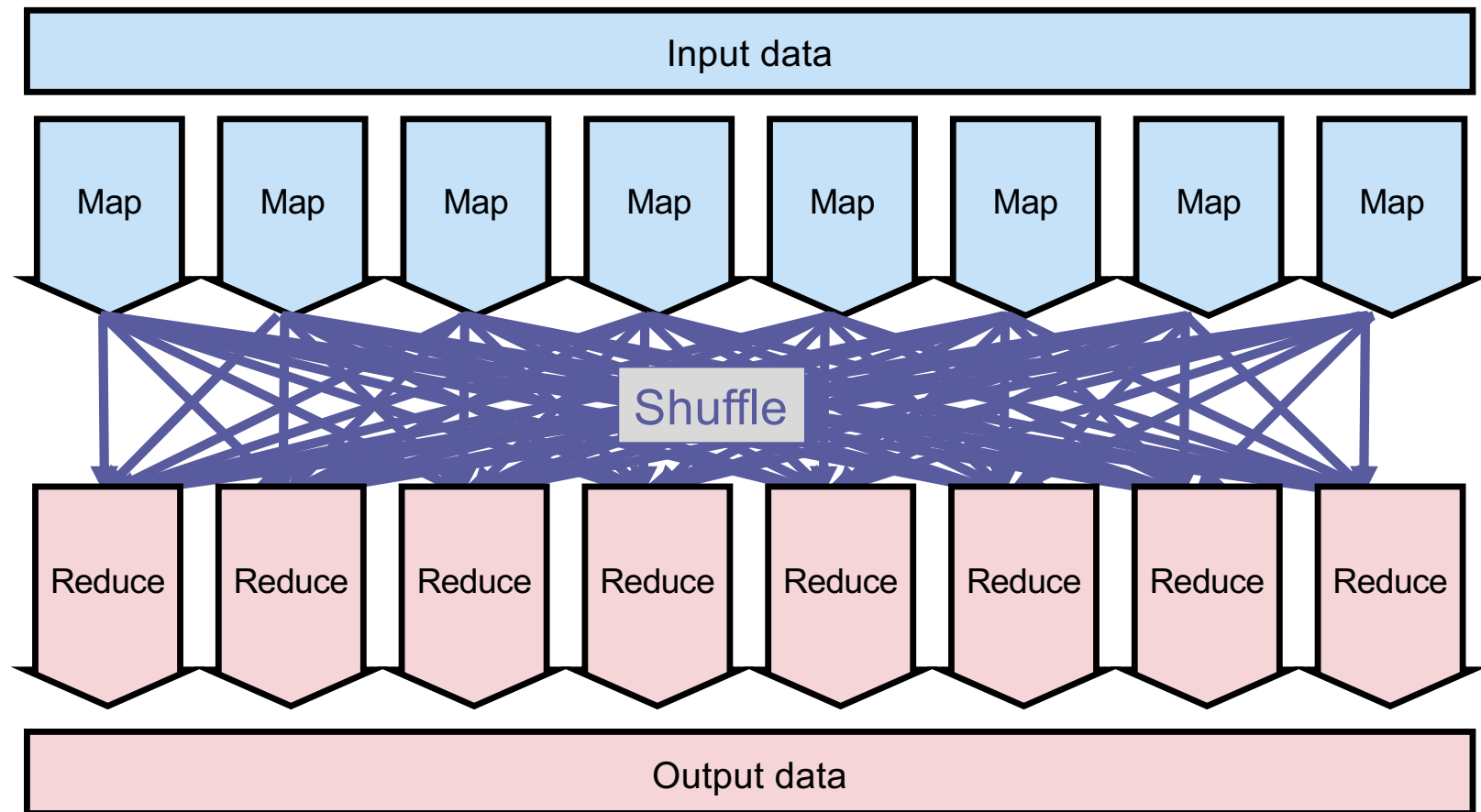


MapReduce

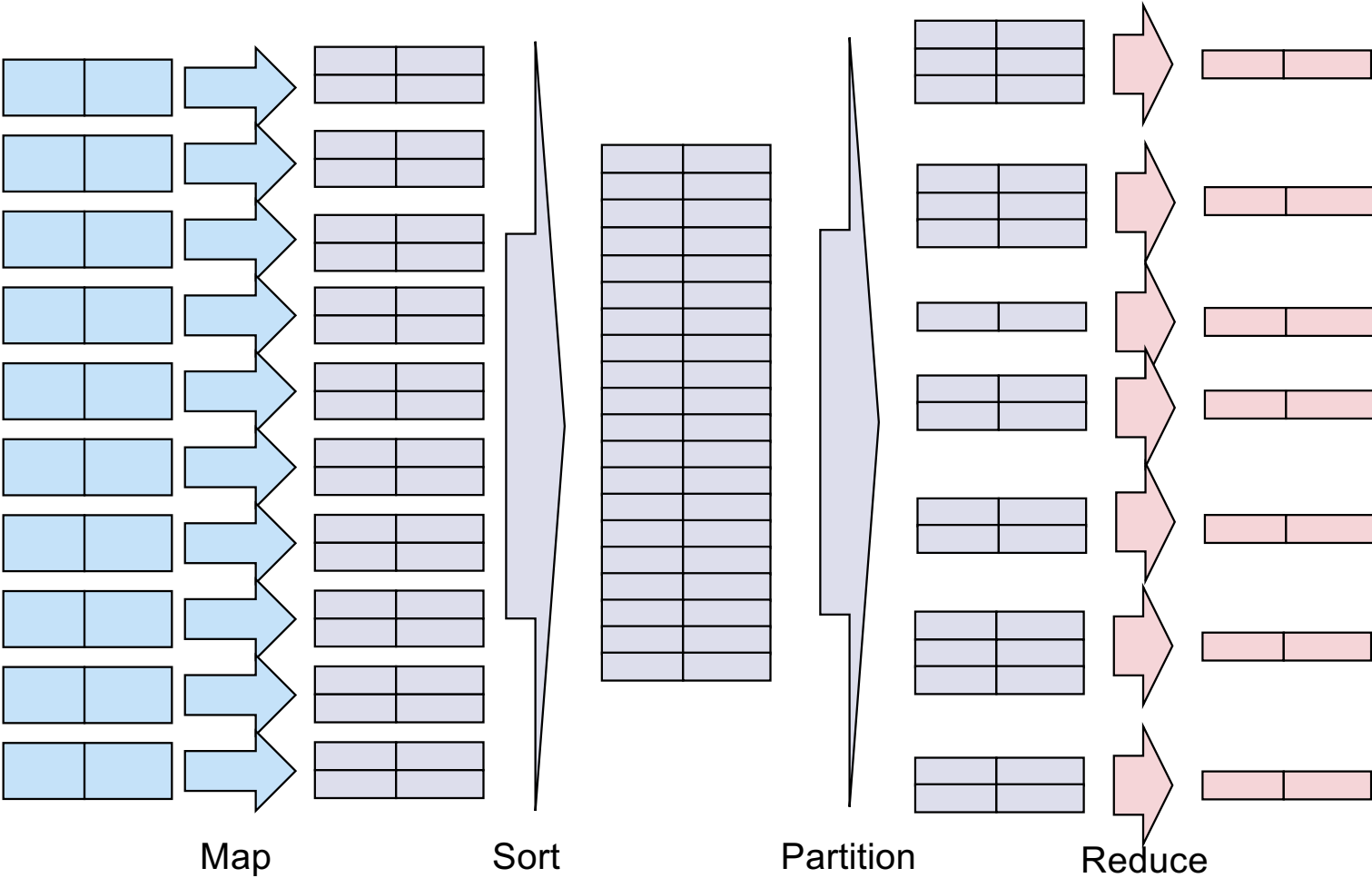
Data Processing: data comes in chunks



A common and useful sub-case: MapReduce



Overall



MapReduce 101



Counting Pokemons



Desired end result

	?		?
	?		?
	?		?
	?		?
	?		?

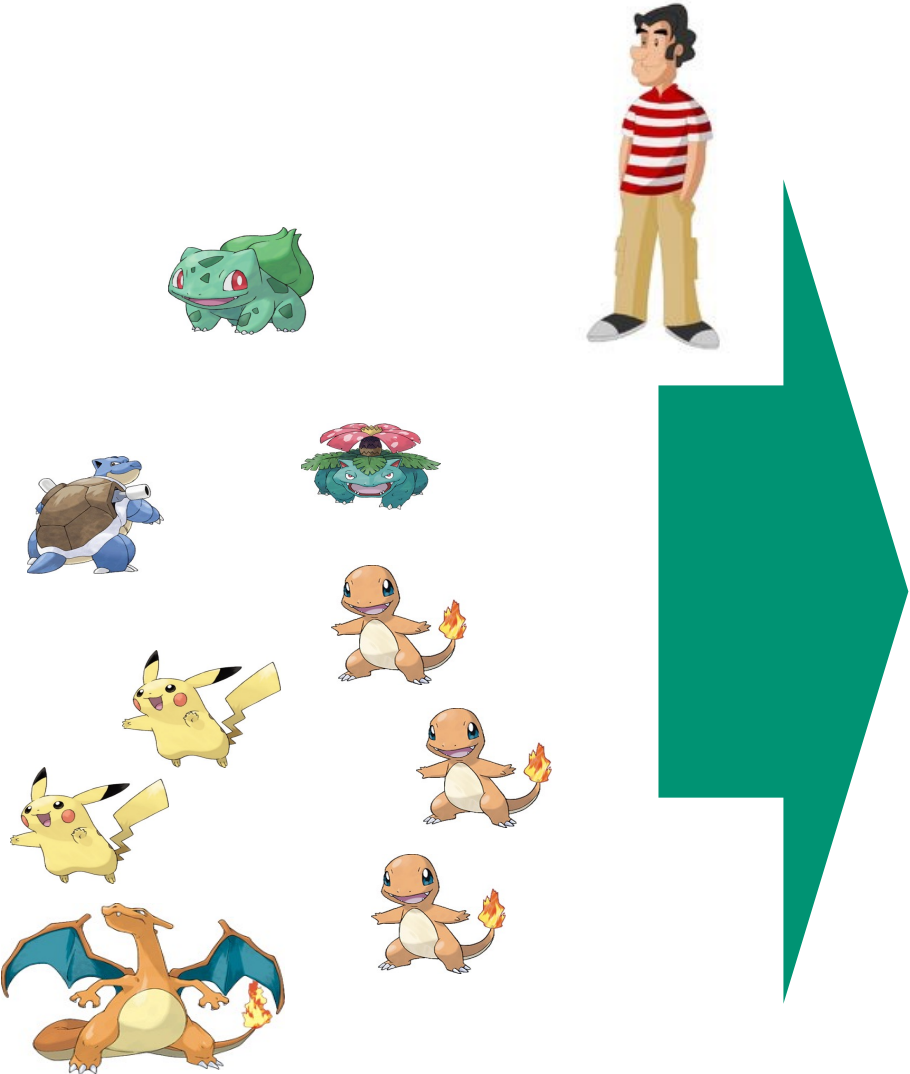
Mappers



Mappers



Mappers

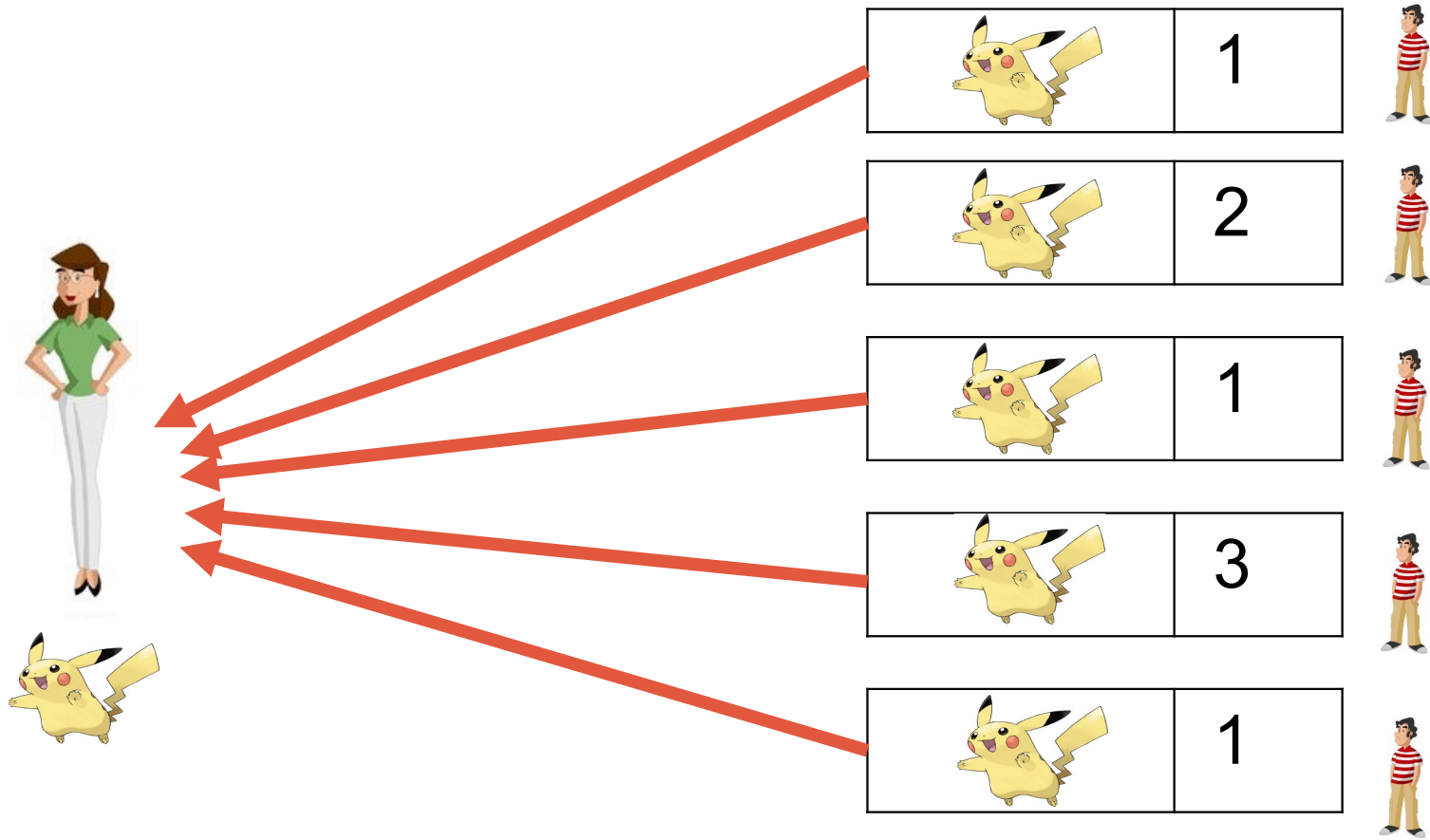


	1
	3
	1
	2
	1
	1

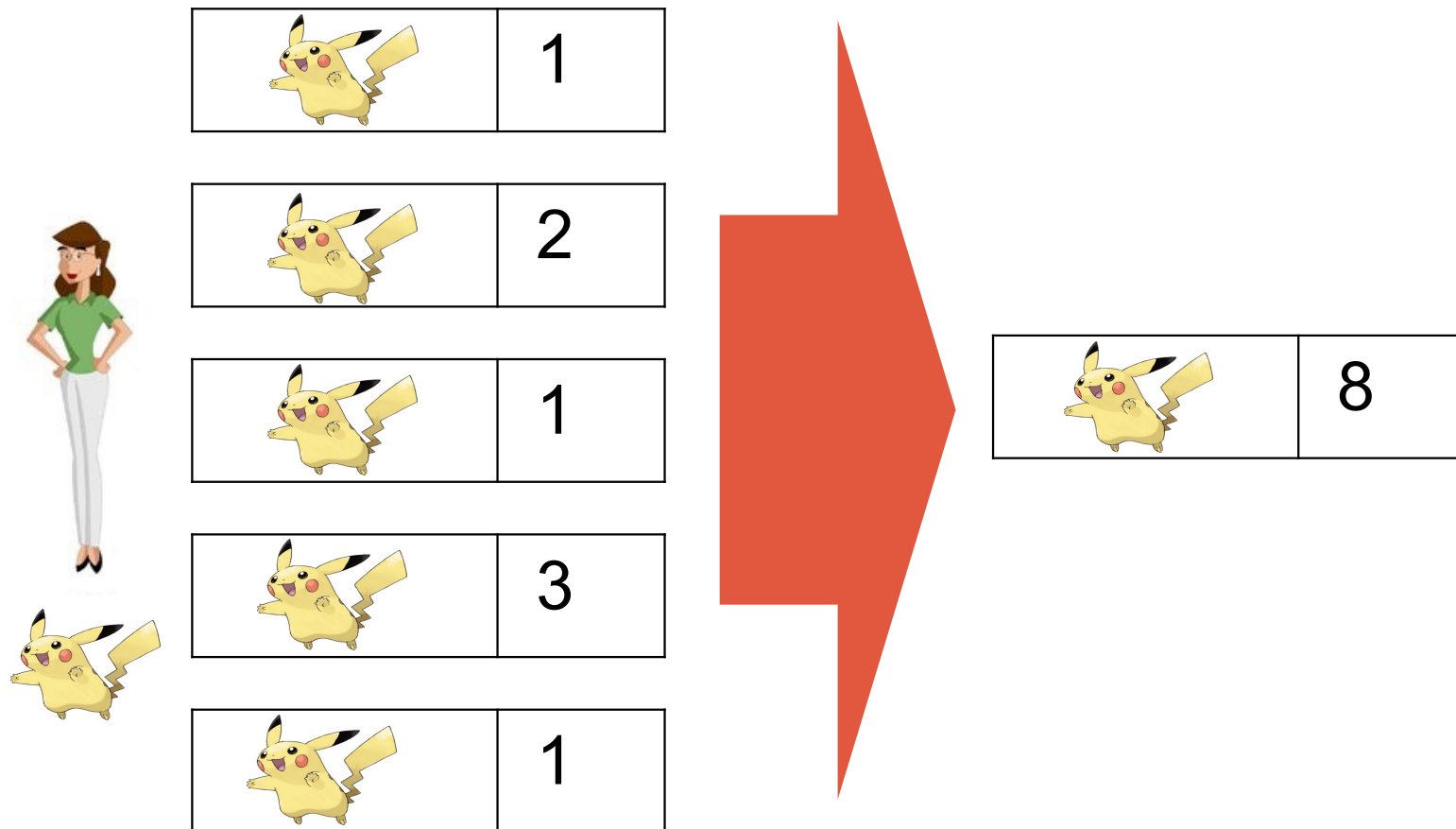
Reducers



Shuffling



Reducers



Final summary



	8
---	---



	6
---	---



	4
--	---



	2
---	---



	10
---	----



	5
---	---



	3
--	---

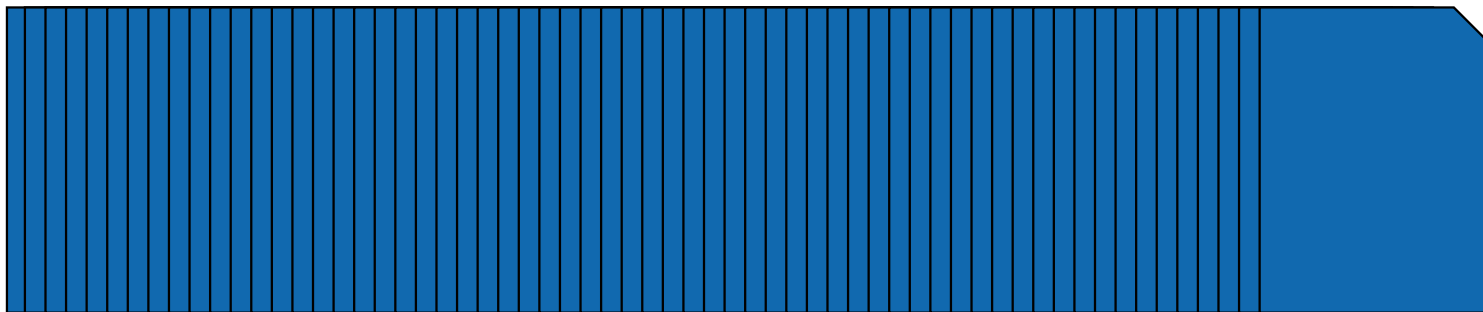


	7
---	---



Building an index with MapReduce

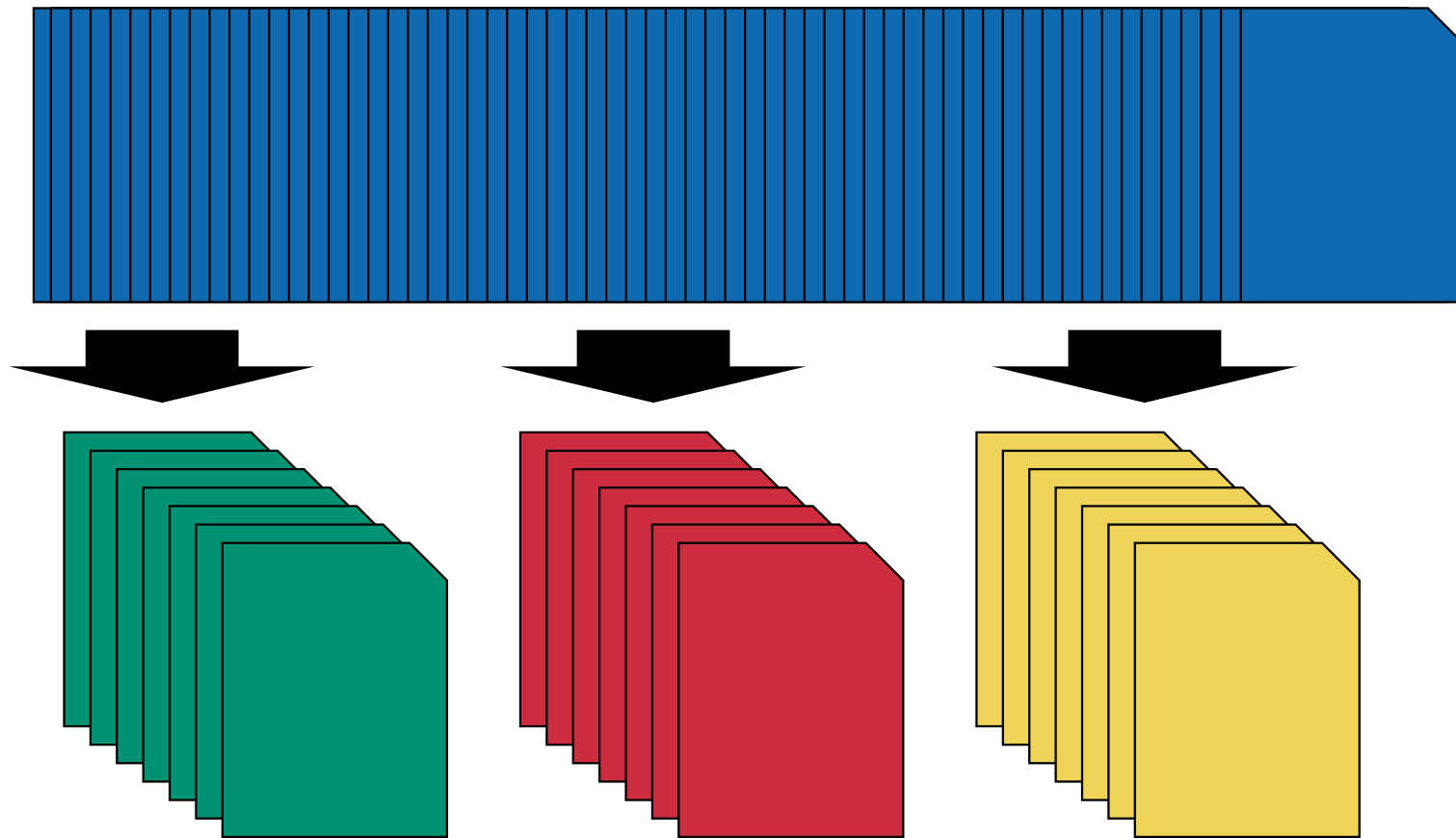
Indexing documents



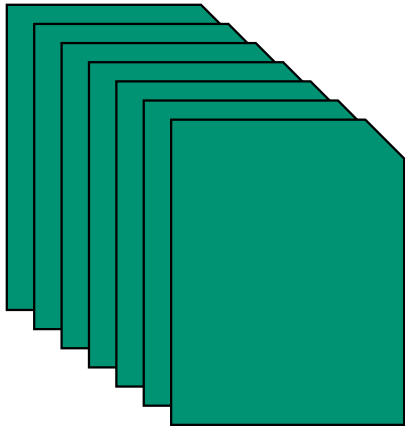
Desired end result

ETH	➡	6	1	2	3	5	6	8
Zürich	➡	5	3	4	7	8	9	
computer	➡	5	1	2	4	5	7	
data	➡	5	1	3	5	8	9	
CPU	➡	4	2	3	4	7		
information	➡	6	1	2	4	5	8	9
retrieval	➡	4	3	5	7	8		

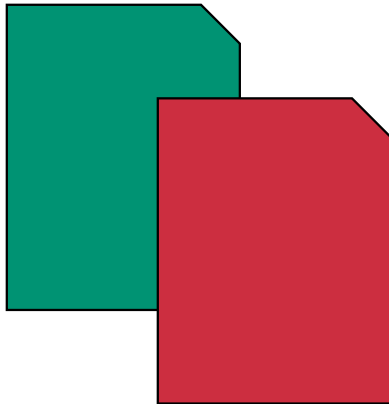
Shard the collection of documents



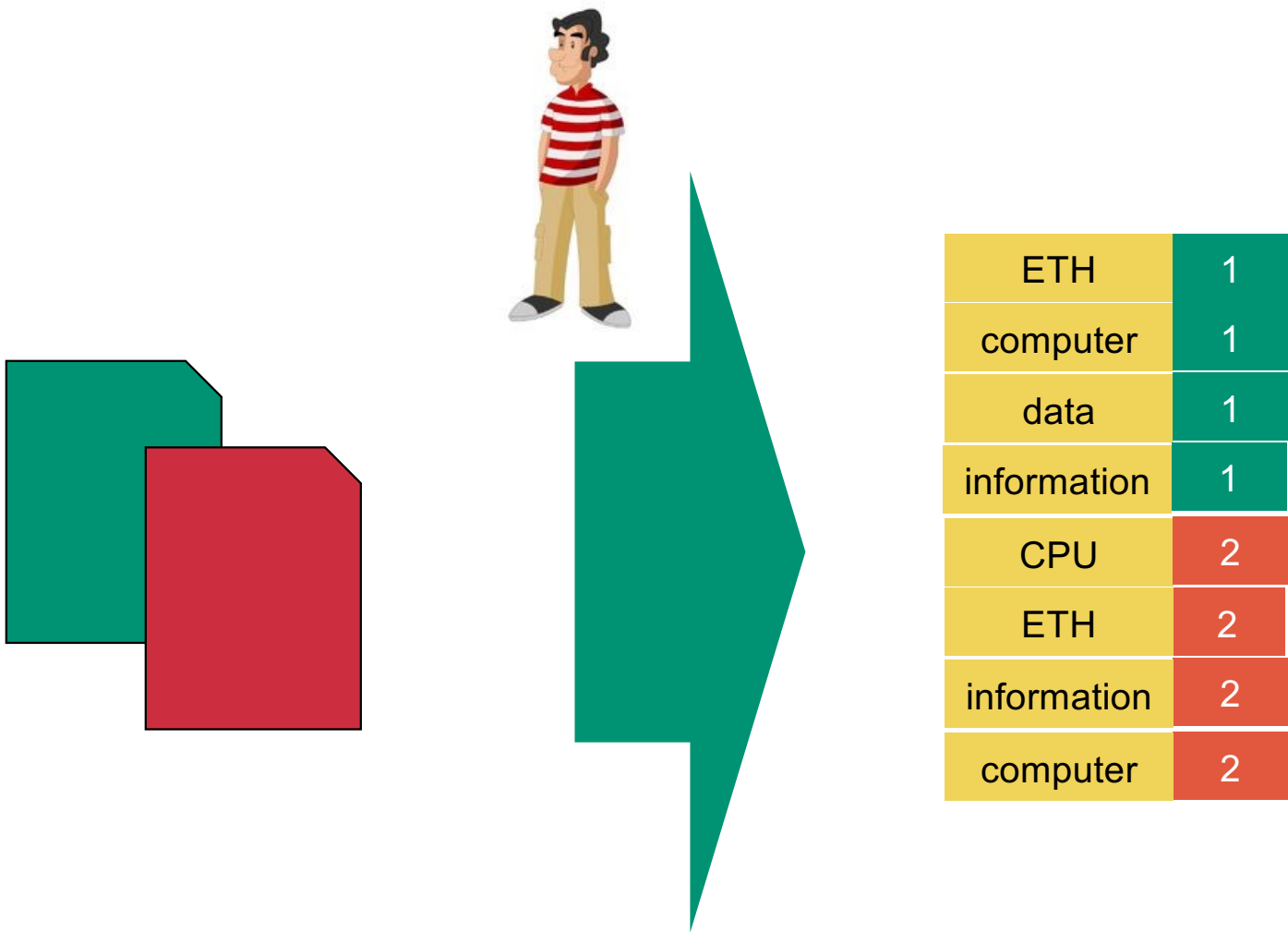
Mappers



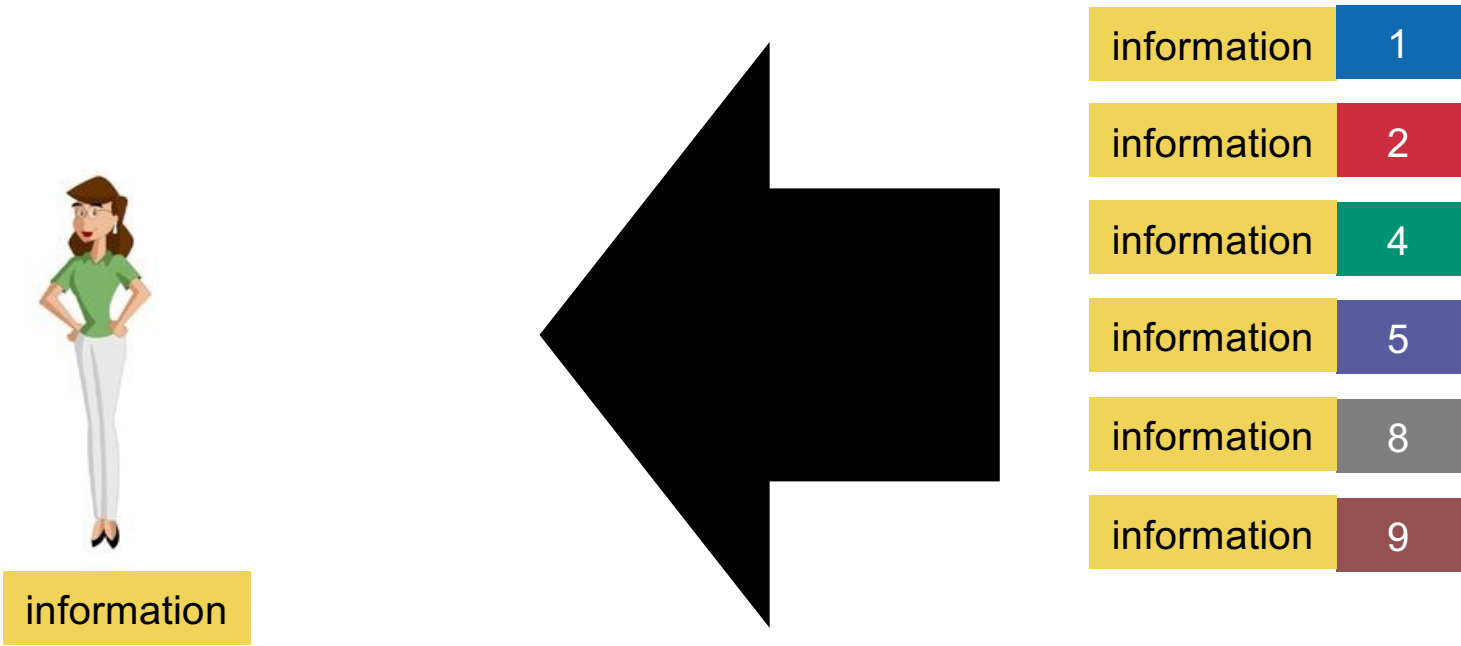
Mappers



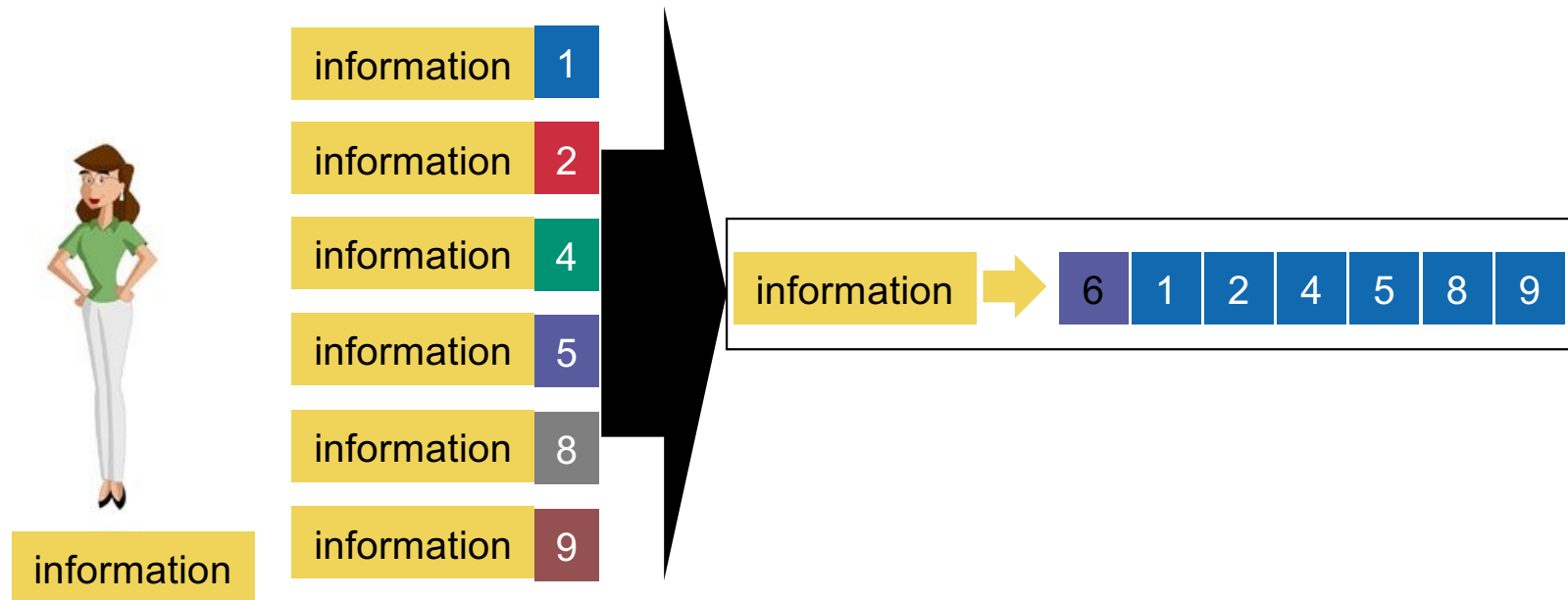
Mappers



Reducers



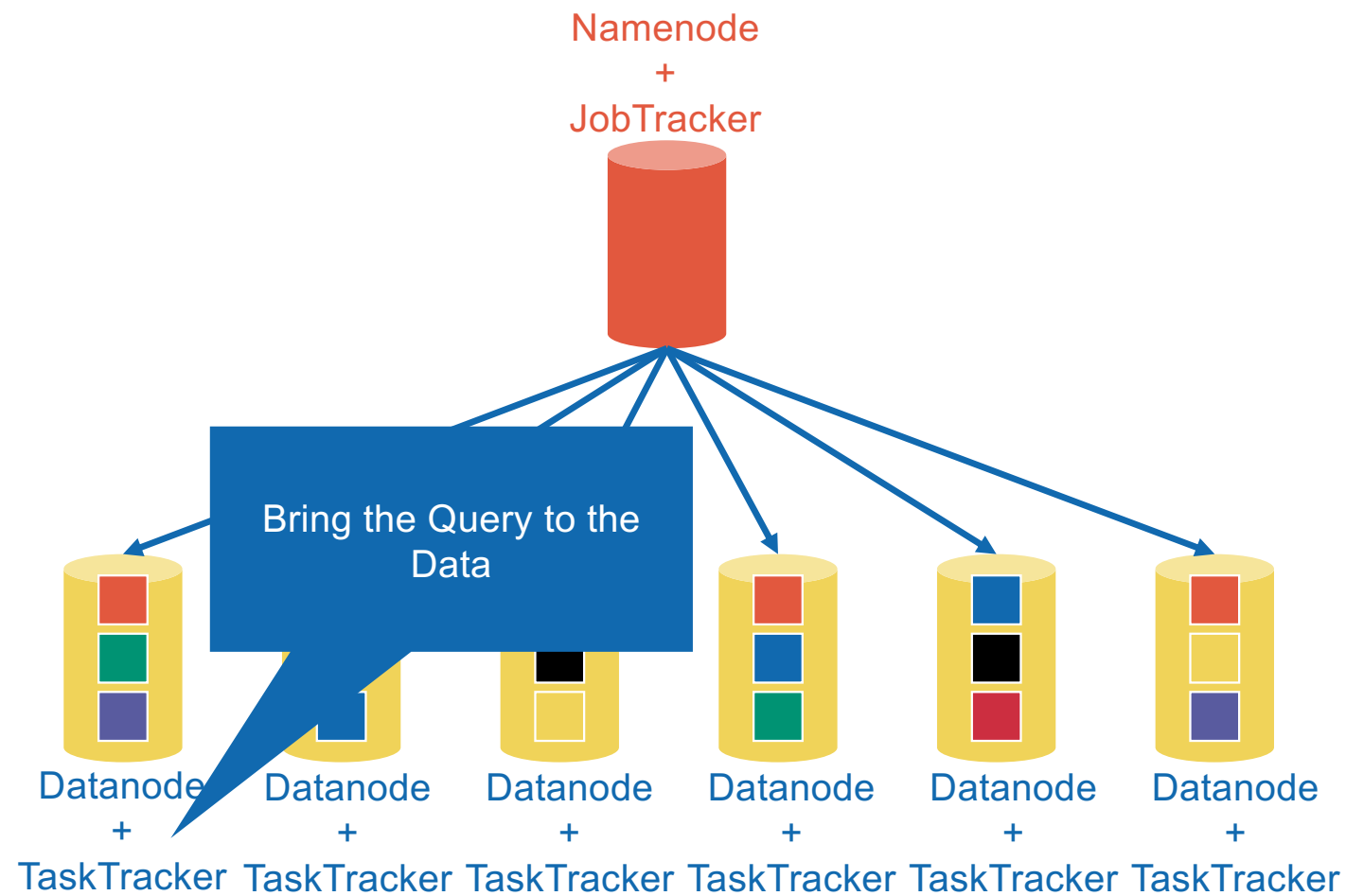
Reducers



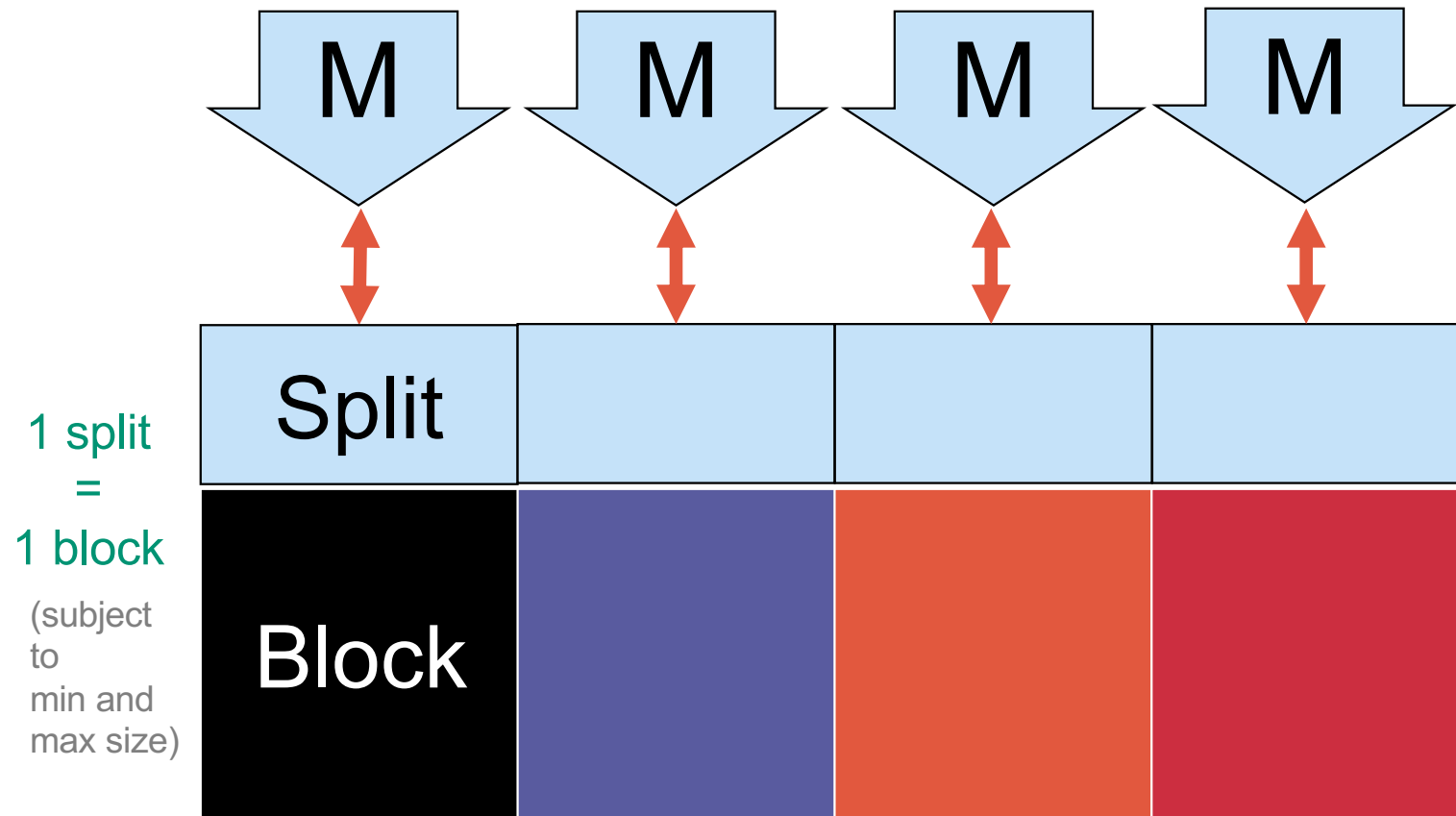
End result

ETH	➡	6	1	2	3	5	6	8
Zürich	➡	5	3	4	7	8	9	
computer	➡	5	1	2	4	5	7	
data	➡	5	1	3	5	8	9	
CPU	➡	4	2	3	4	7		
information	➡	6	1	2	4	5	8	9
retrieval	➡	4	3	5	7	8		

MapReduce architecture (version 1)



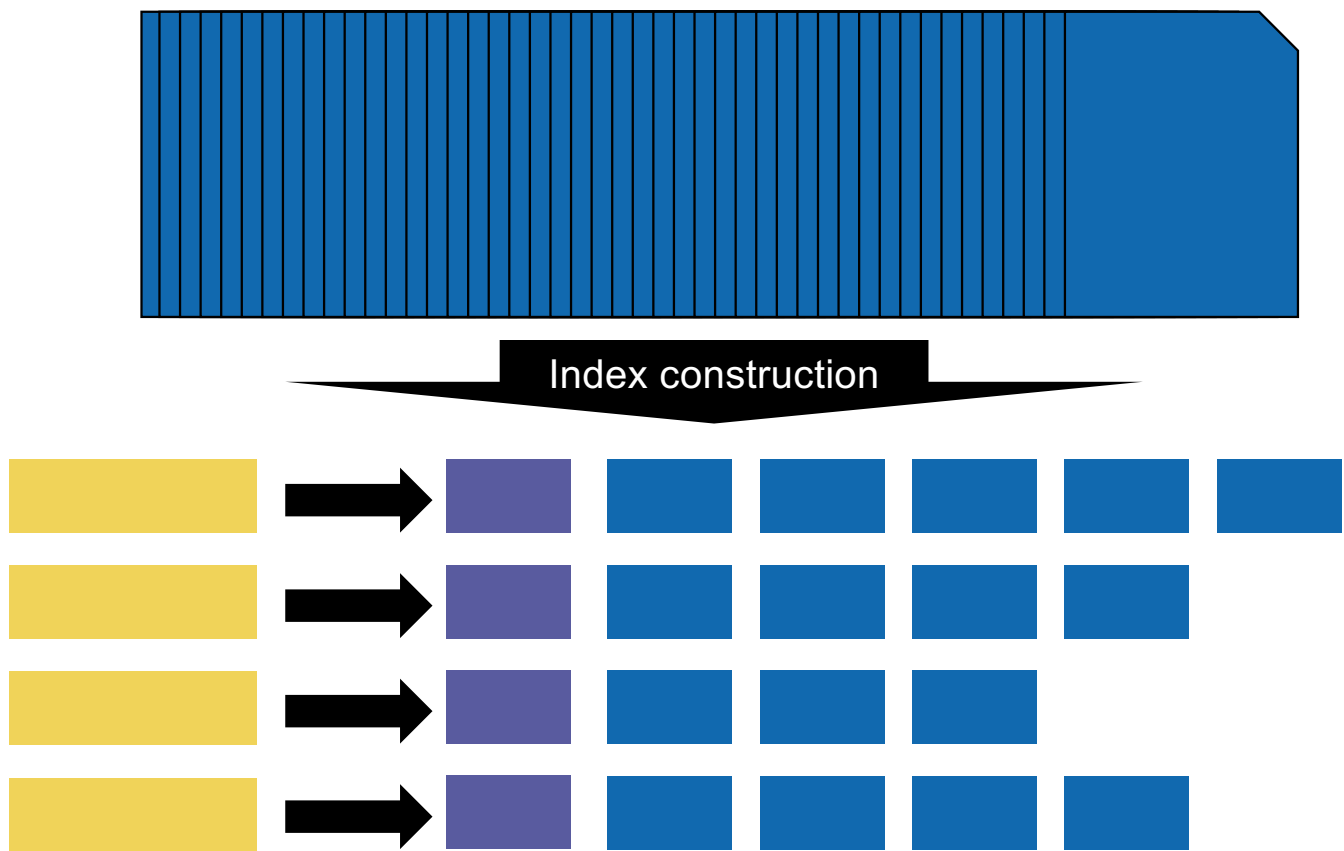
Distribution to nodes in the cluster



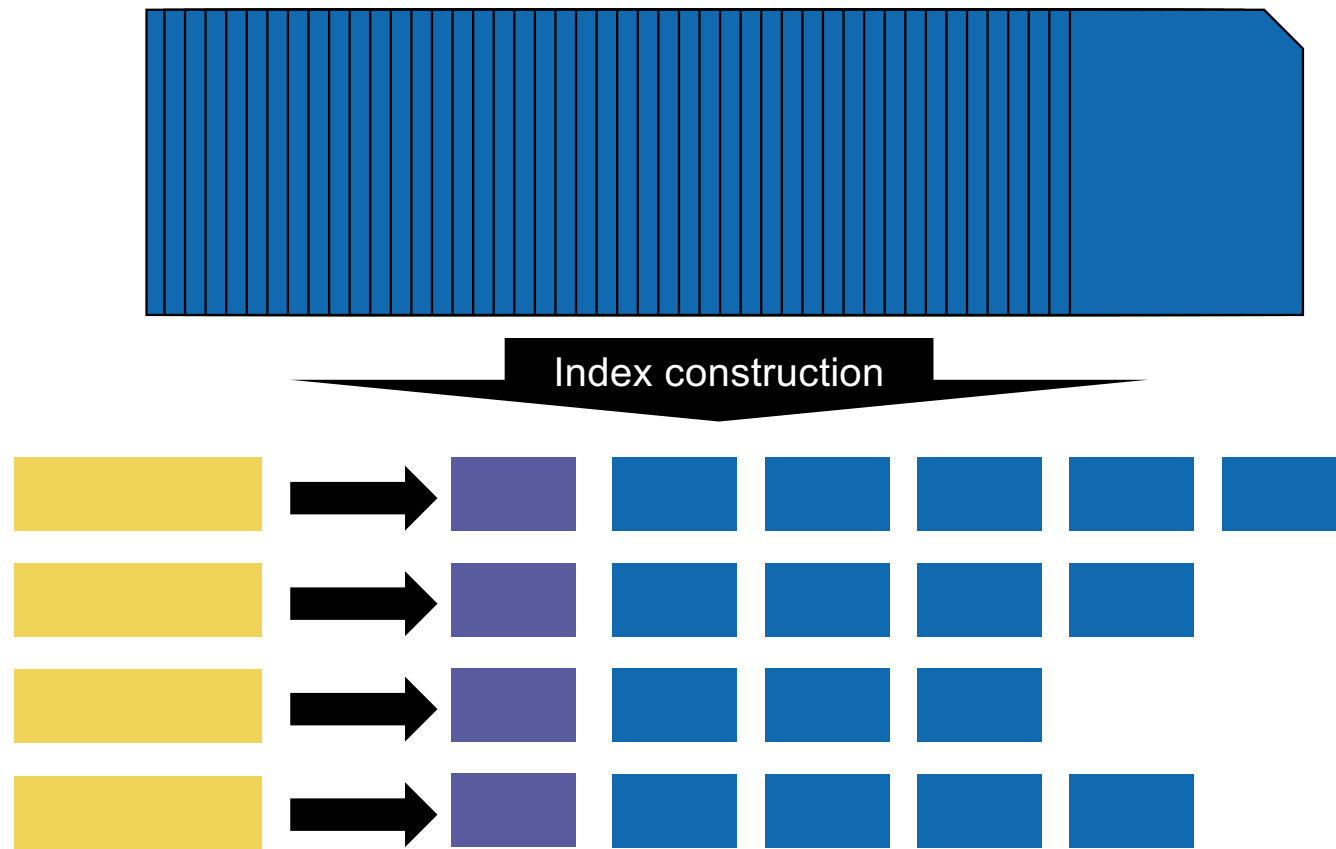


Online index construction

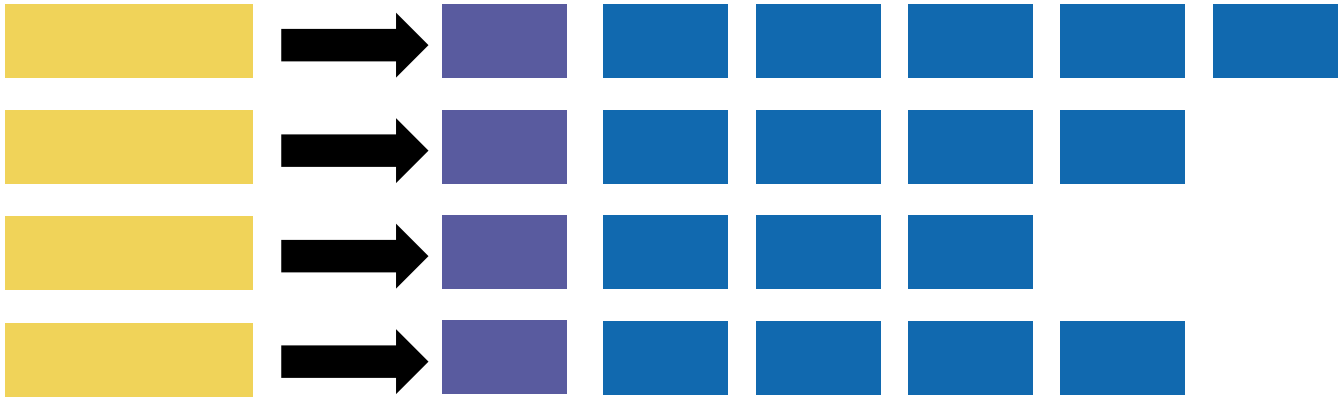
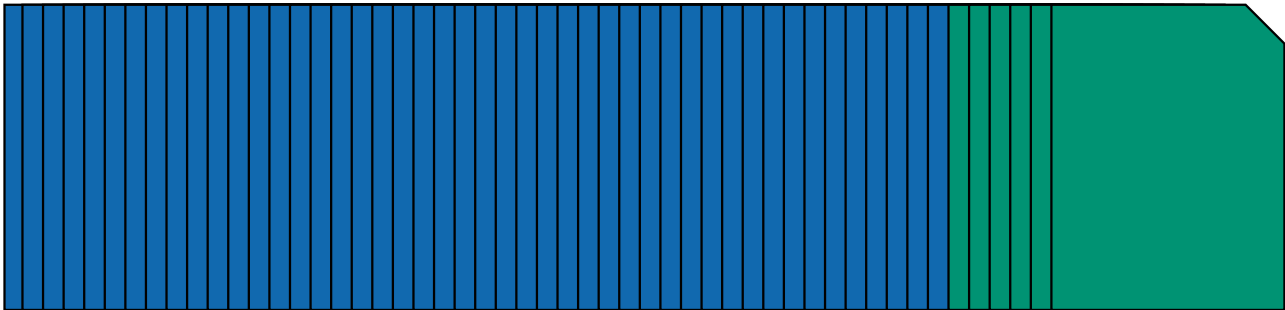
Distribution to nodes in the cluster



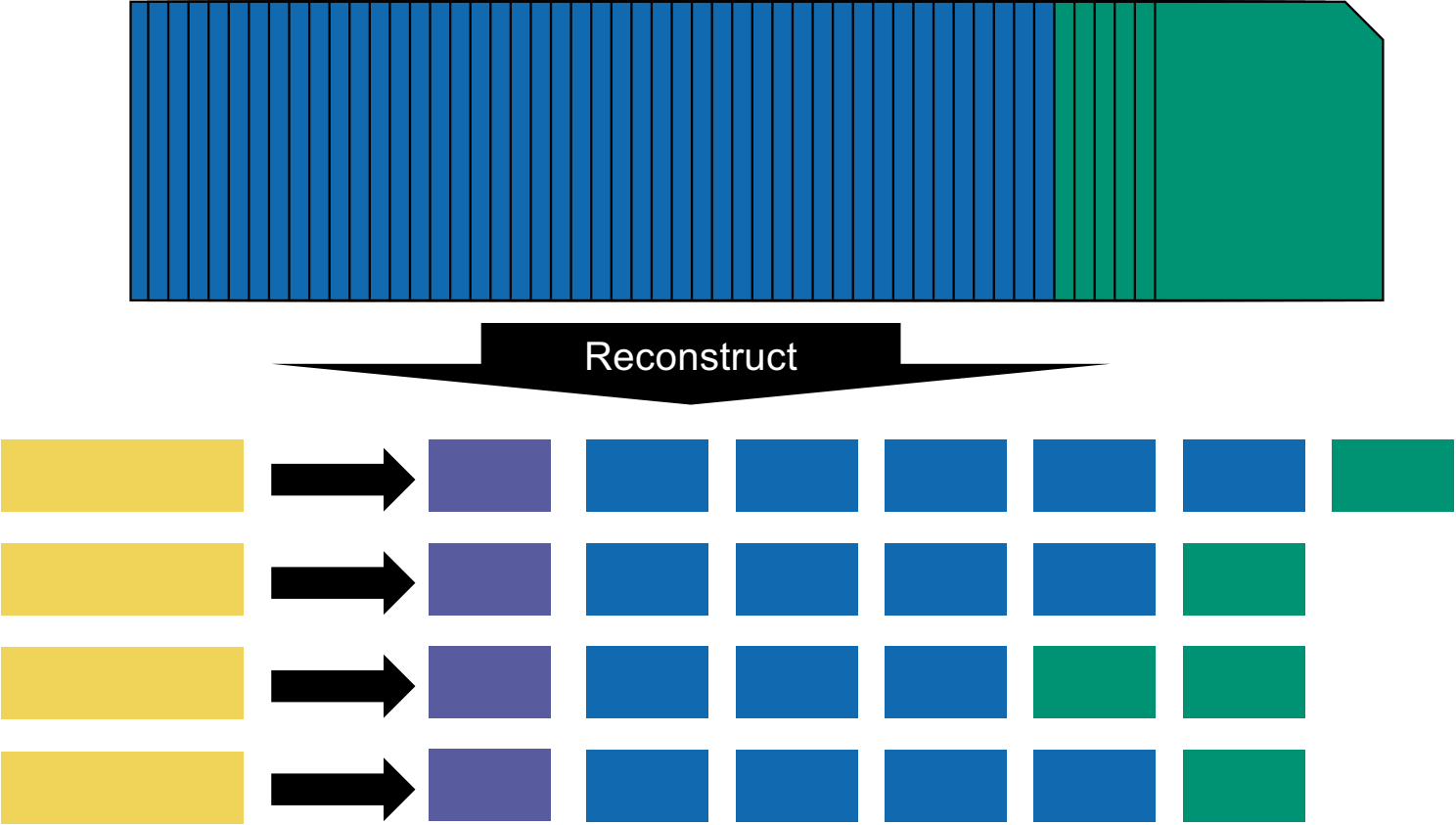
New documents are added...



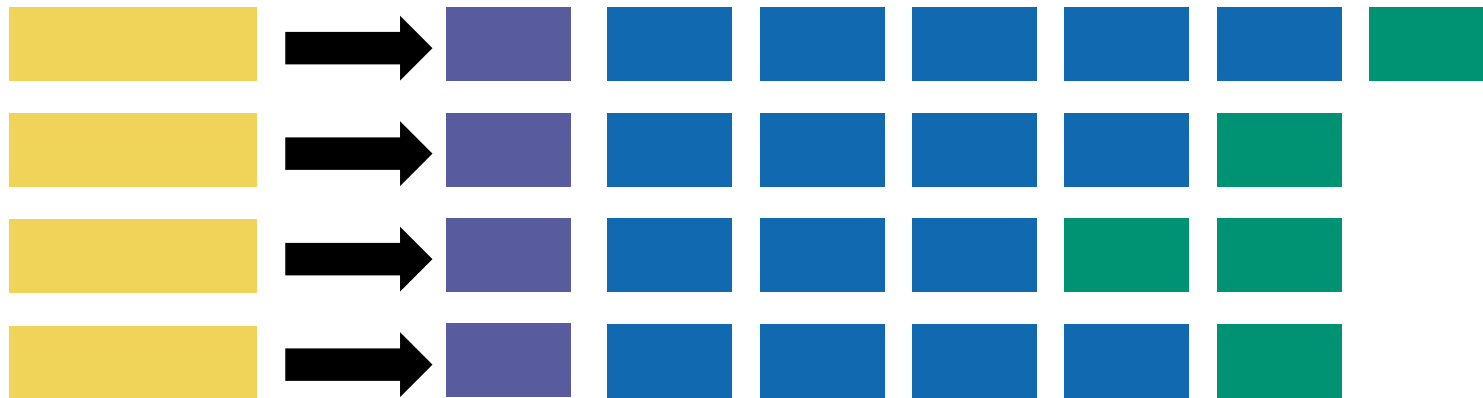
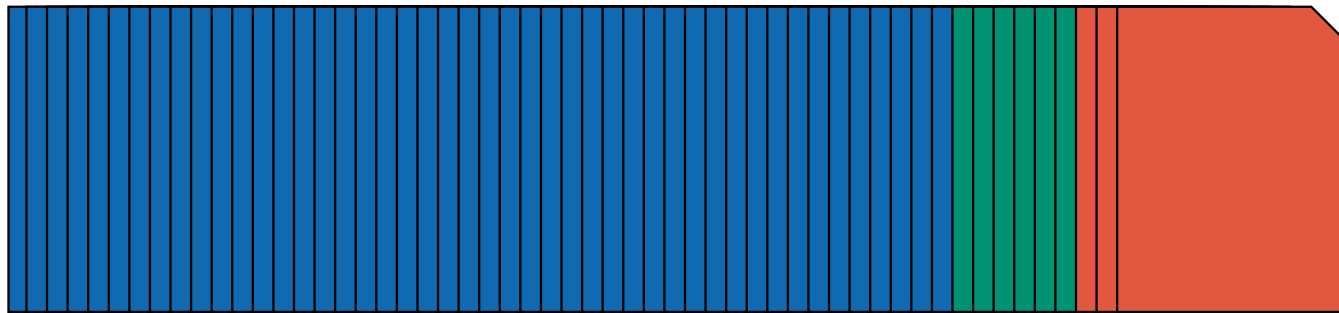
New documents are added...



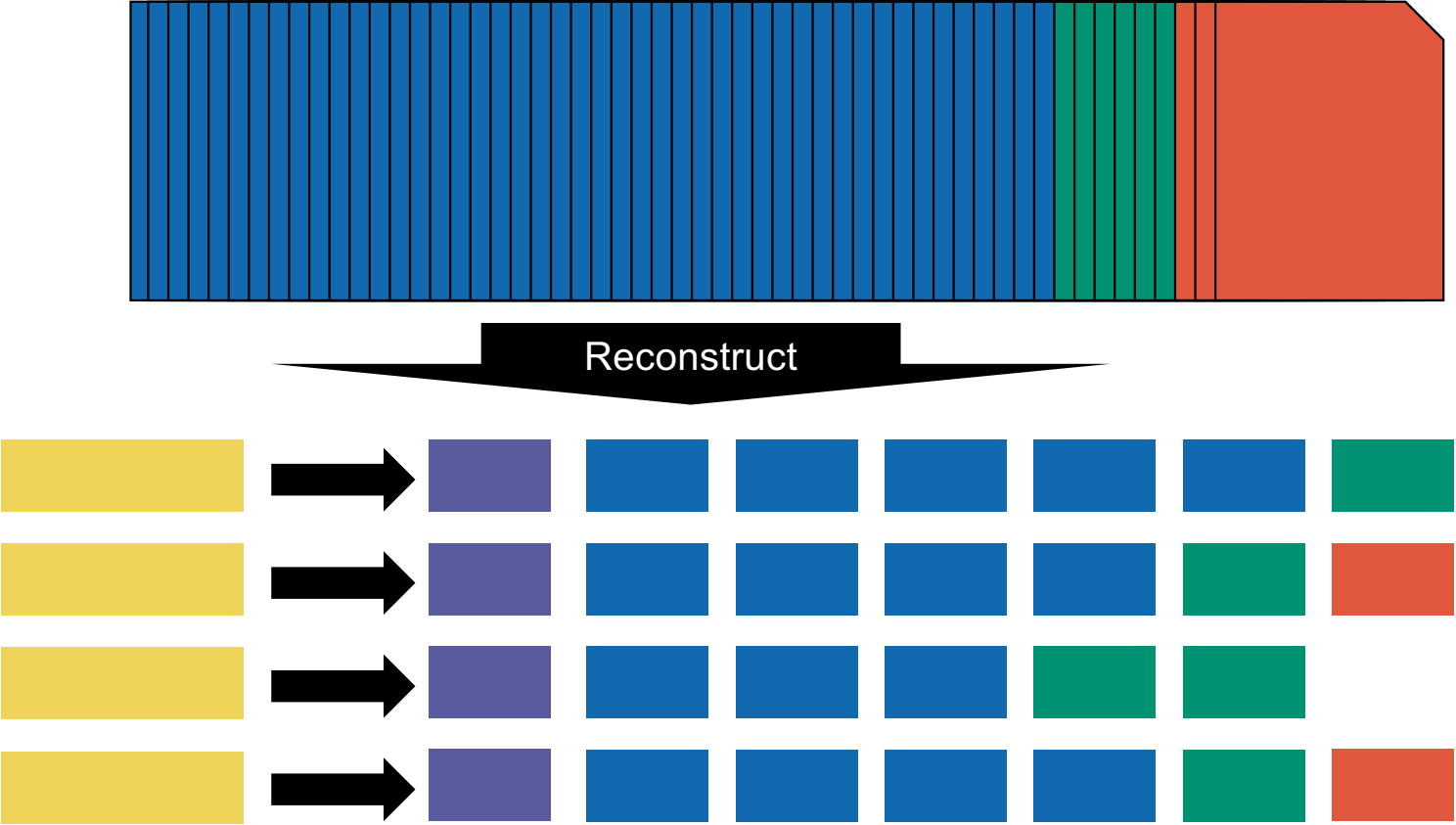
New documents are added...



New documents are added...



New documents are added...



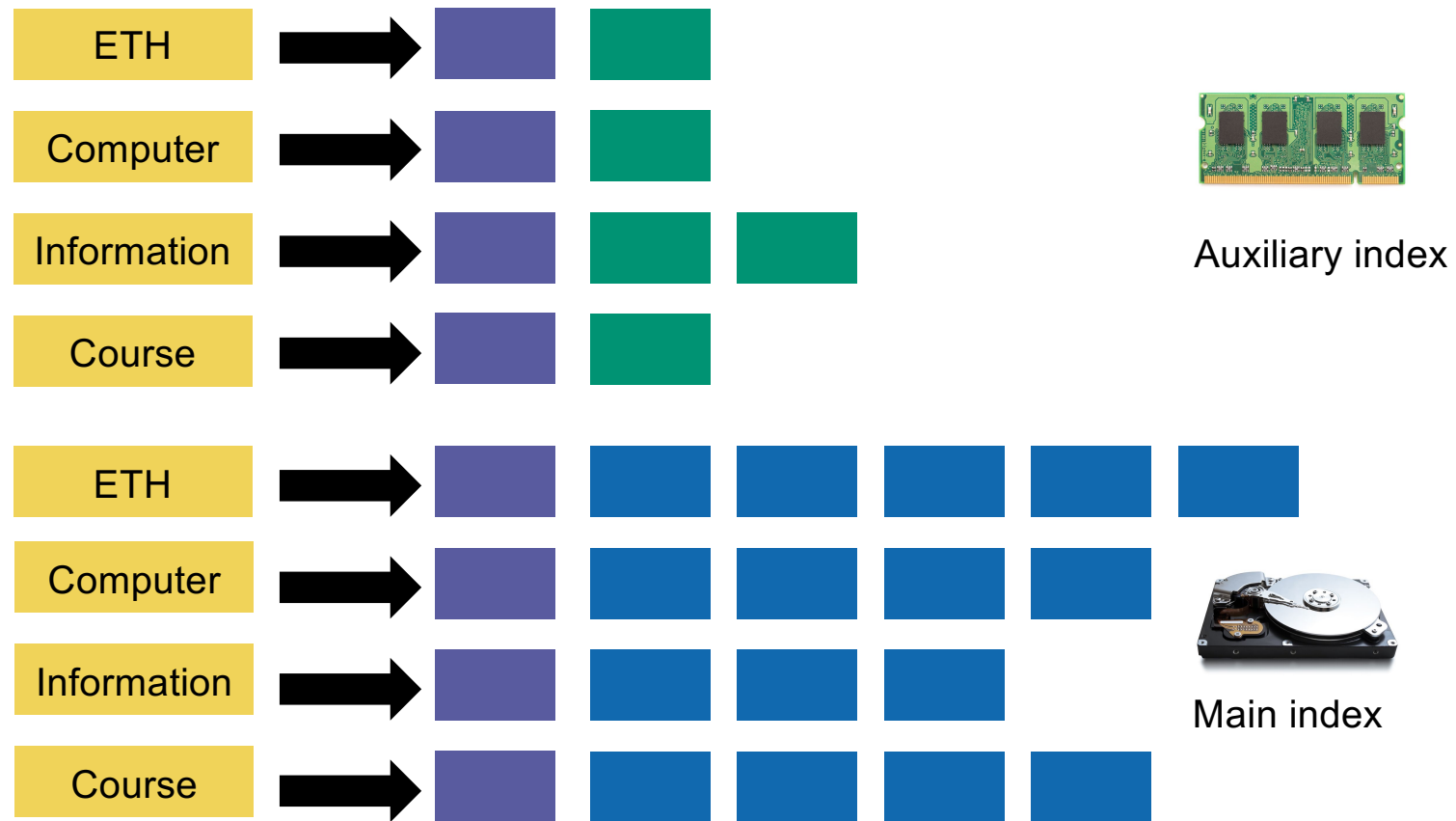
Problem with periodic reconstruction

Result staleness

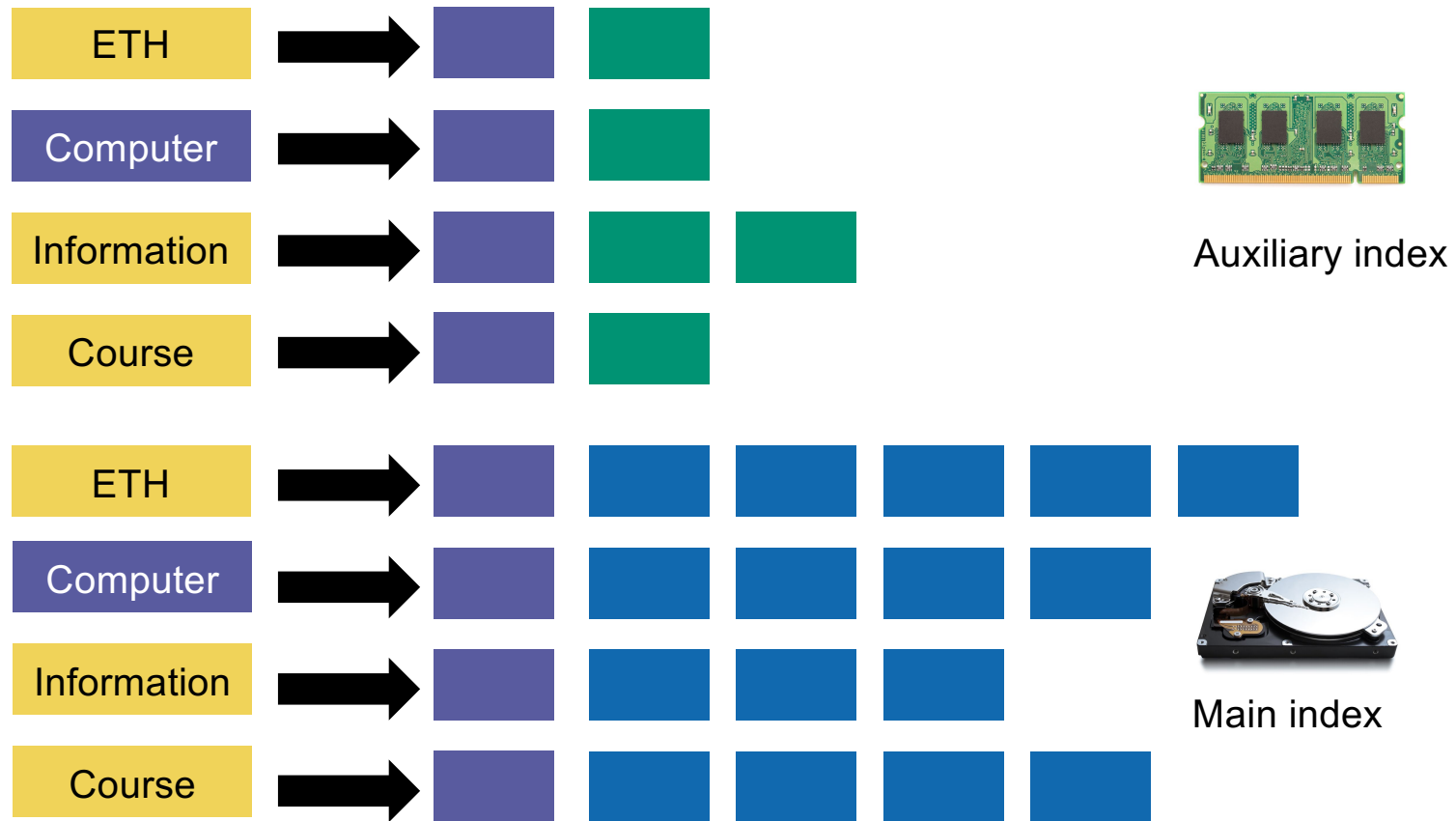
Potentially unavailable,
or slower, during index
construction



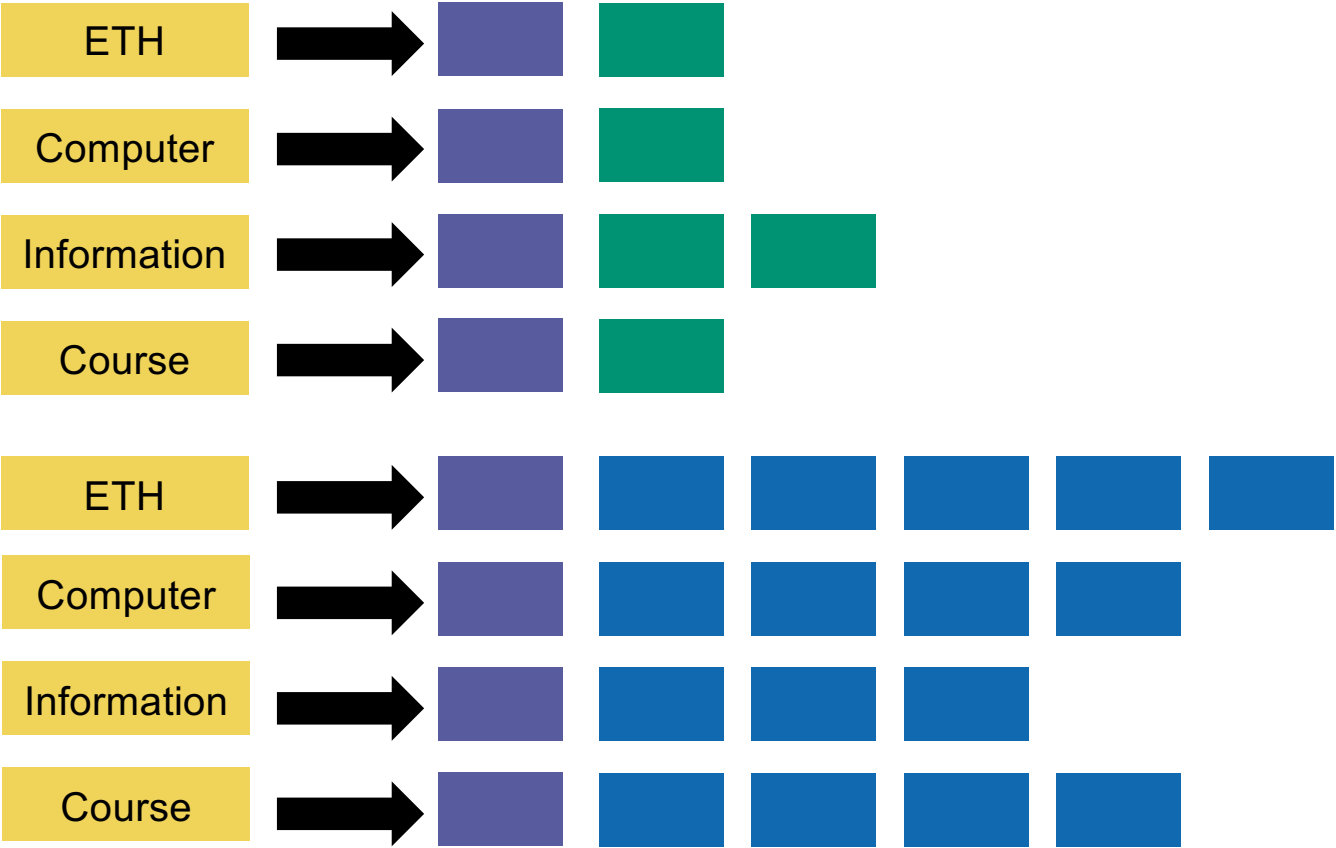
Solution: auxiliary index



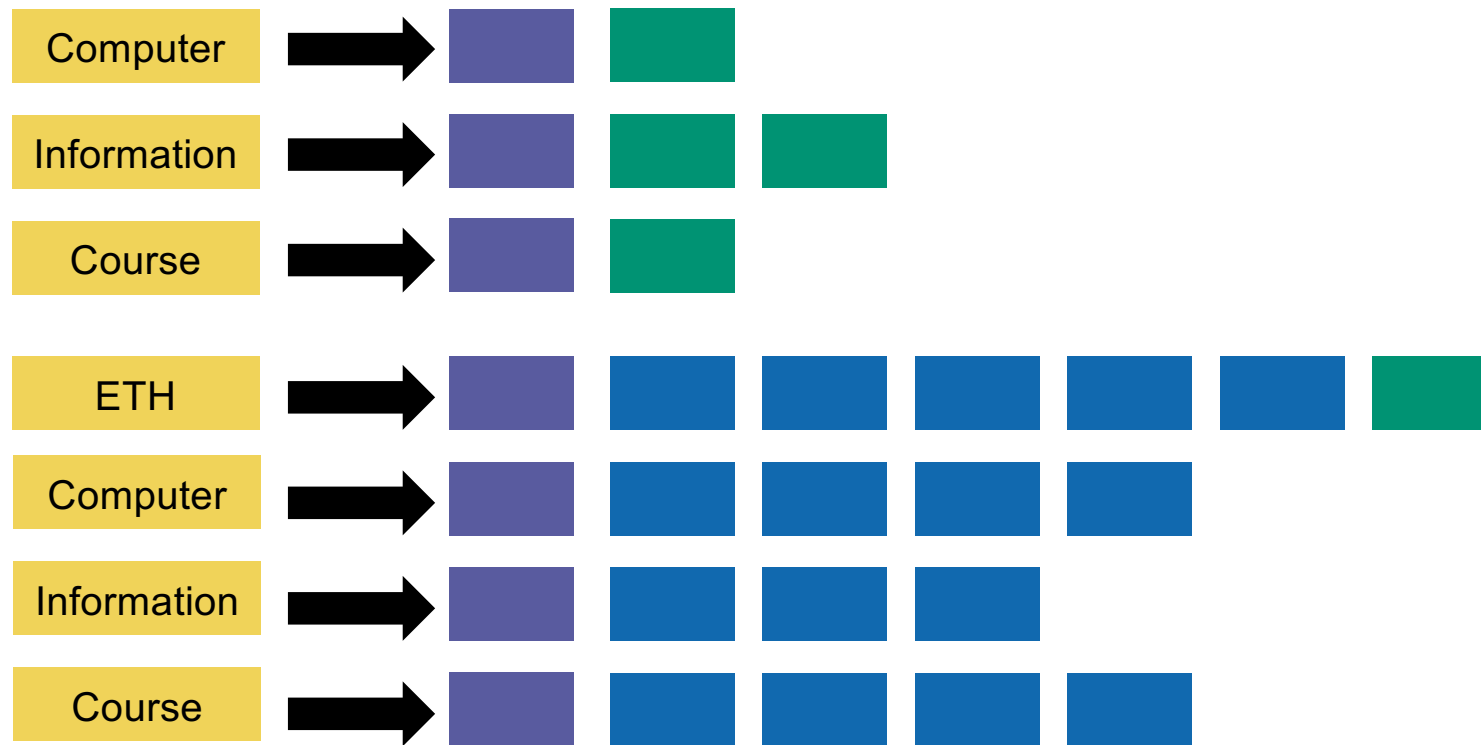
Always query both (union)



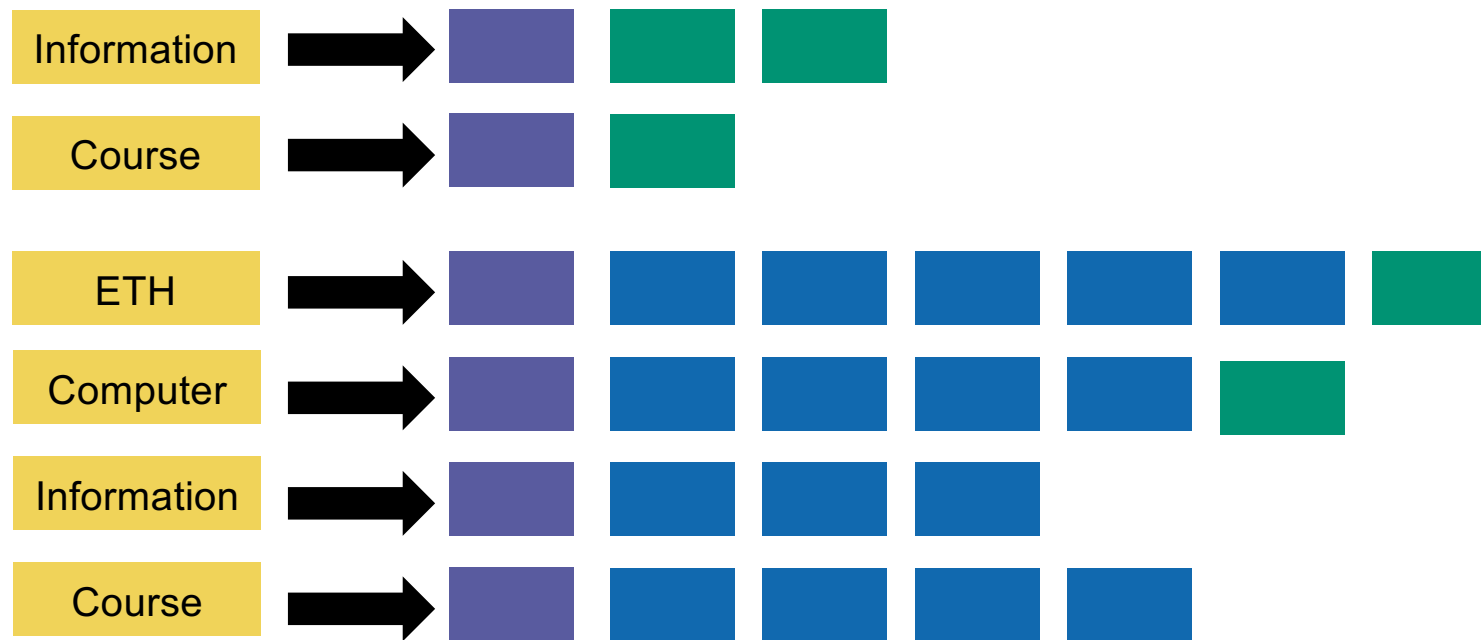
Merge when auxiliary index too large



Merge when auxiliary index too large



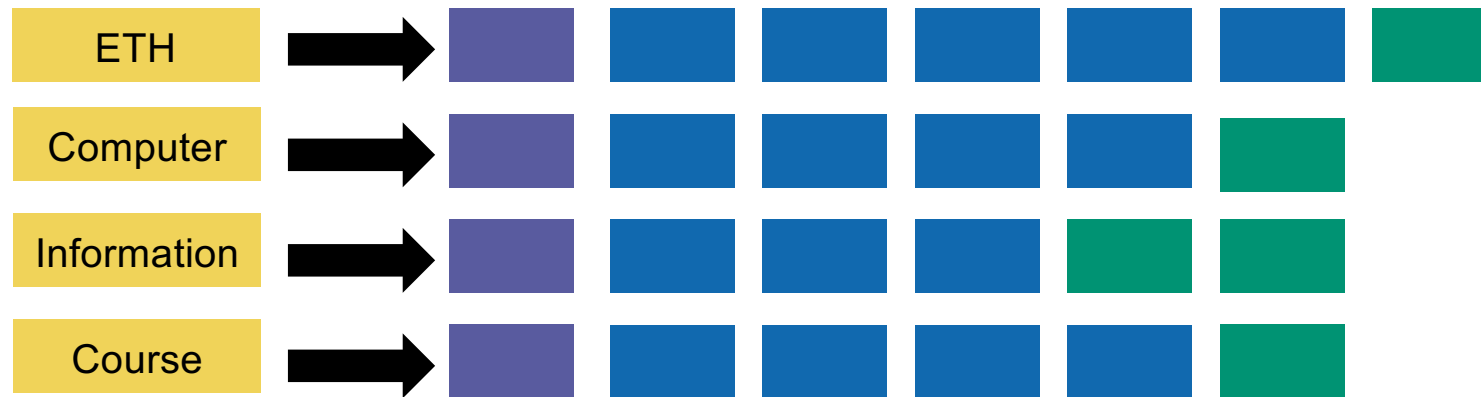
Merge when auxiliary index too large



Merge when auxiliary index too large

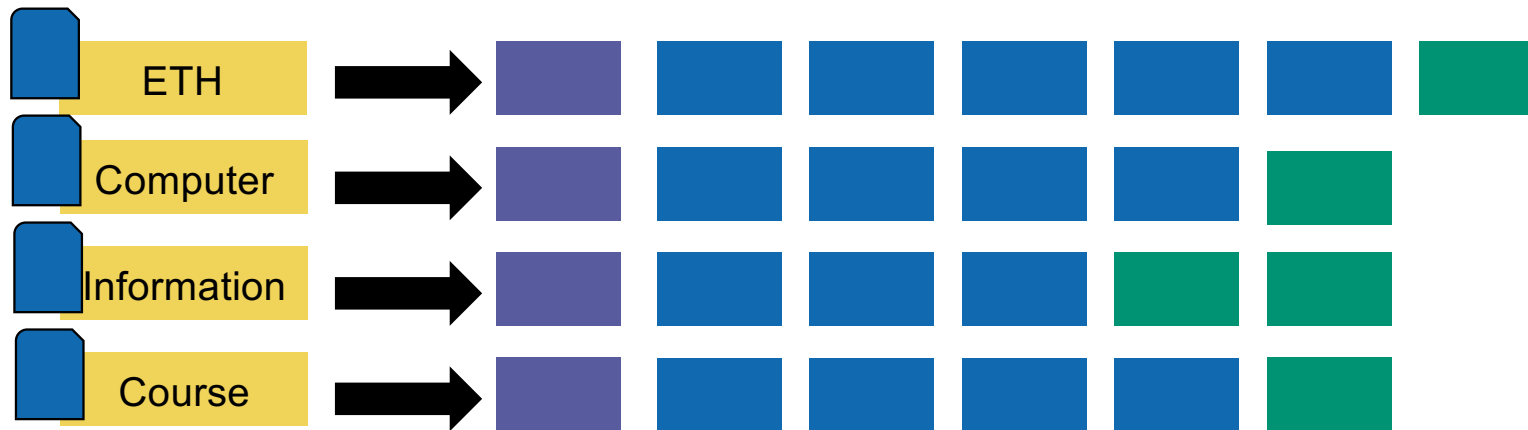


Merge when auxiliary index too large



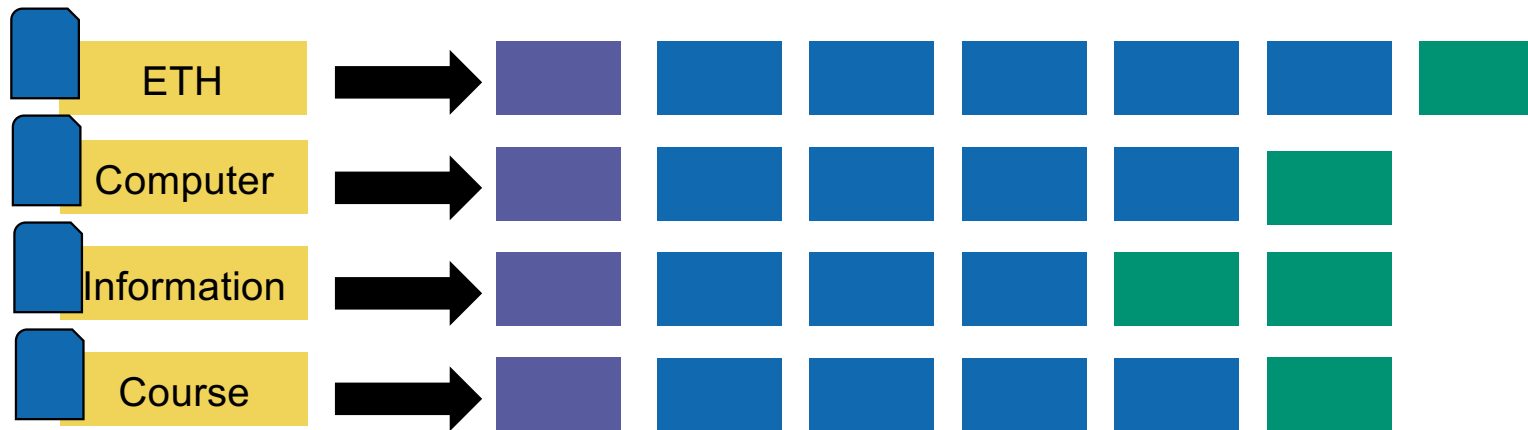
Merge when auxiliary index too large

Simple appends if each postings list is a single file.



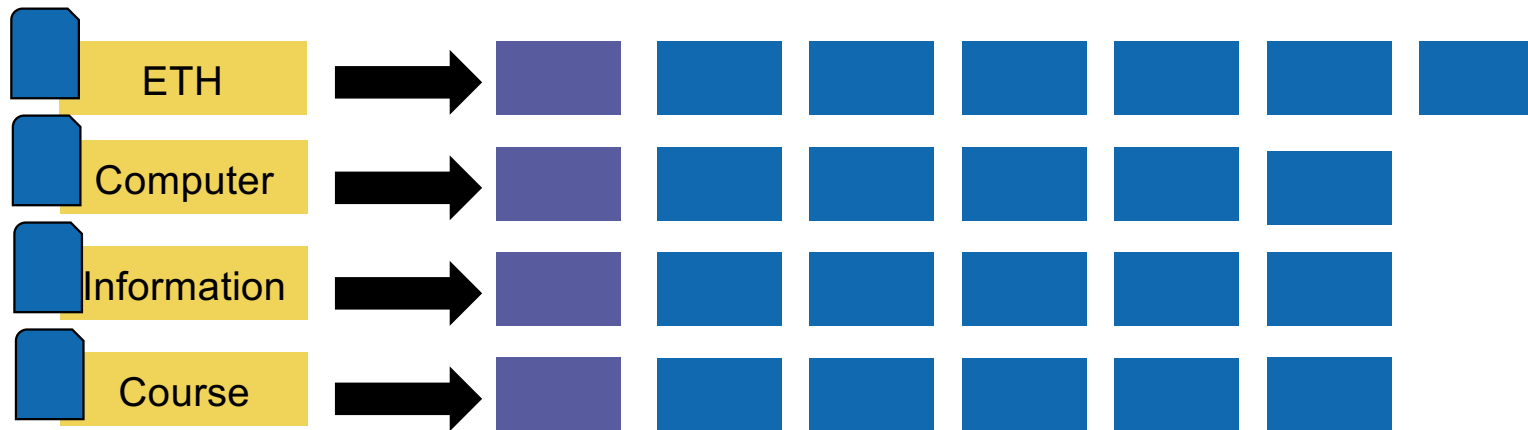
Merge when auxiliary index too large

Only one seek per file!



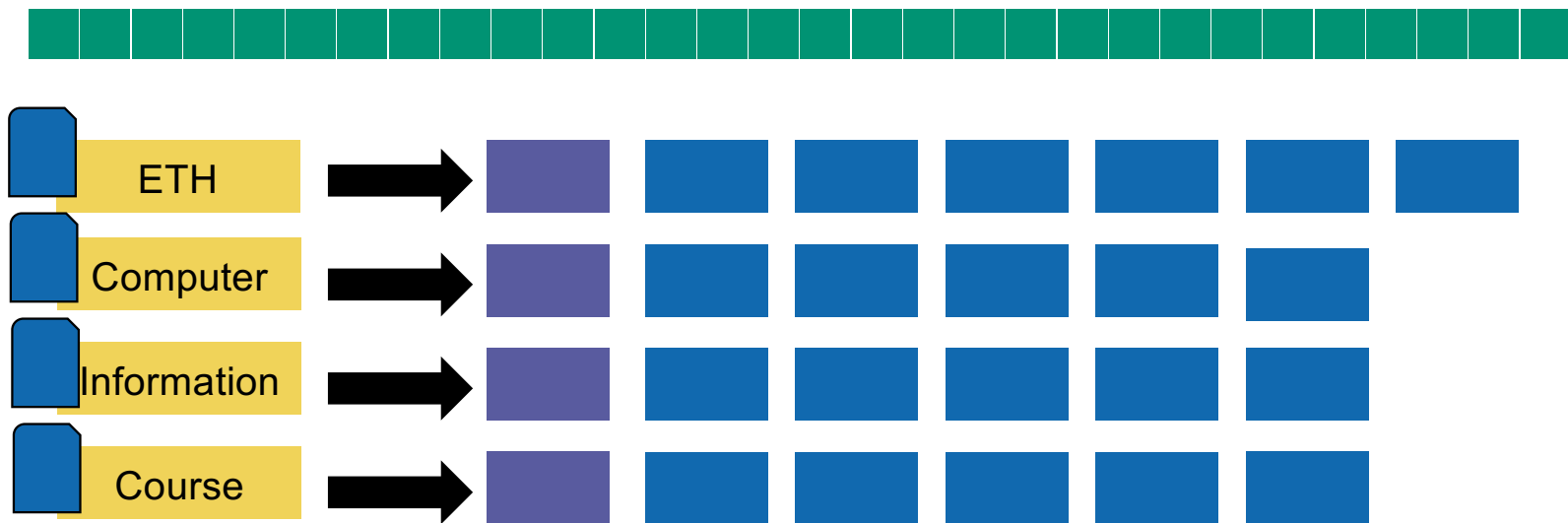
Deletion of a document

What if a document gets deleted? ❌



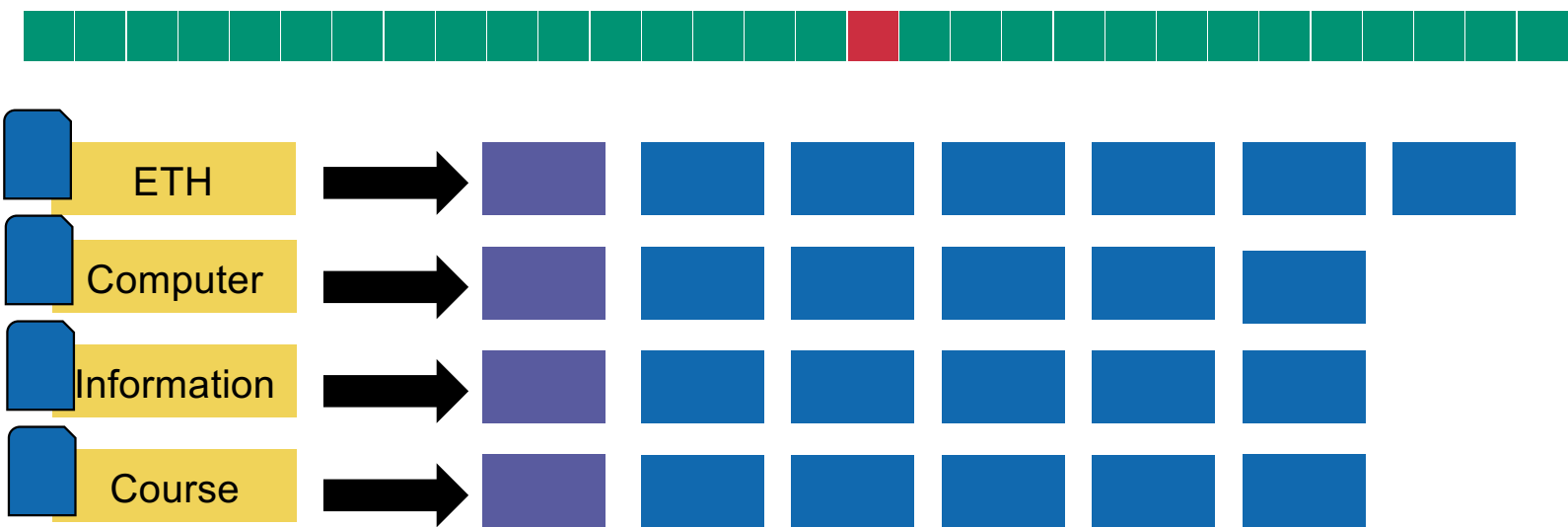
Deletion of a document

What if a document gets deleted? ❌

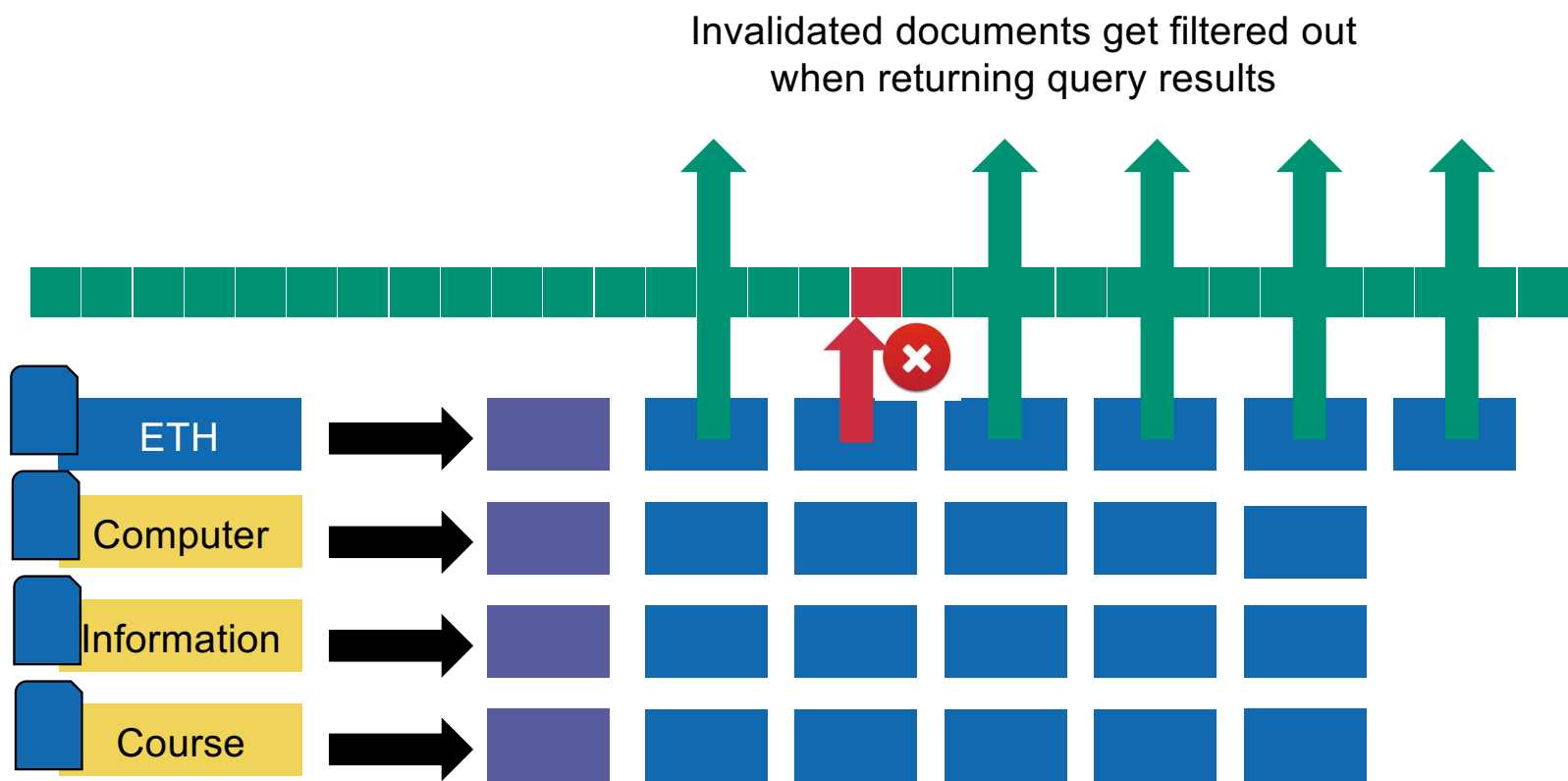


Deletion of a document

Invalidation bit vector

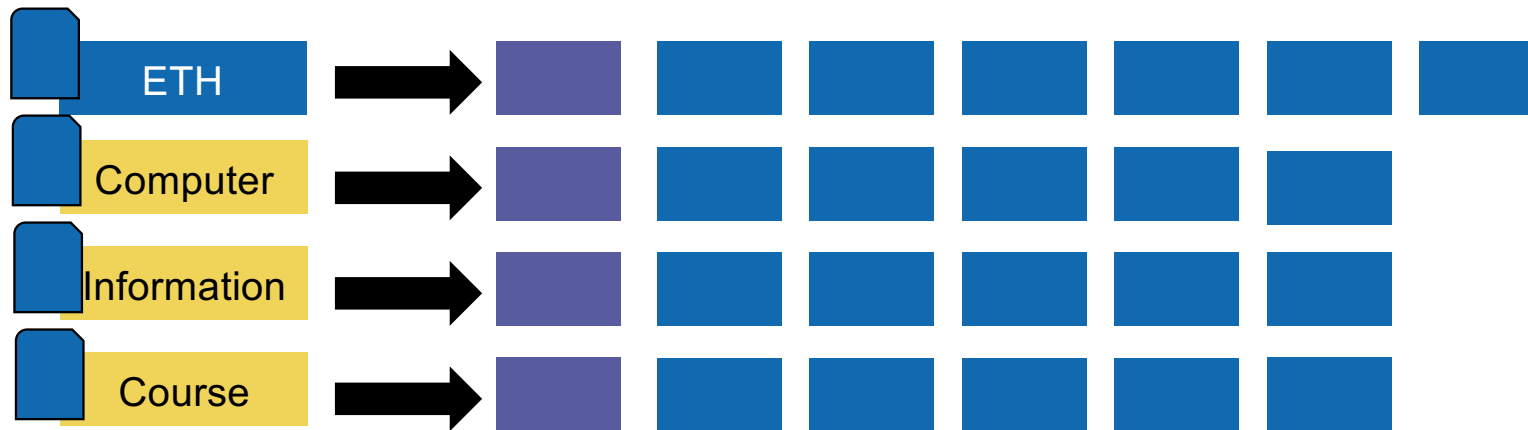


Deletion of a document



In practice

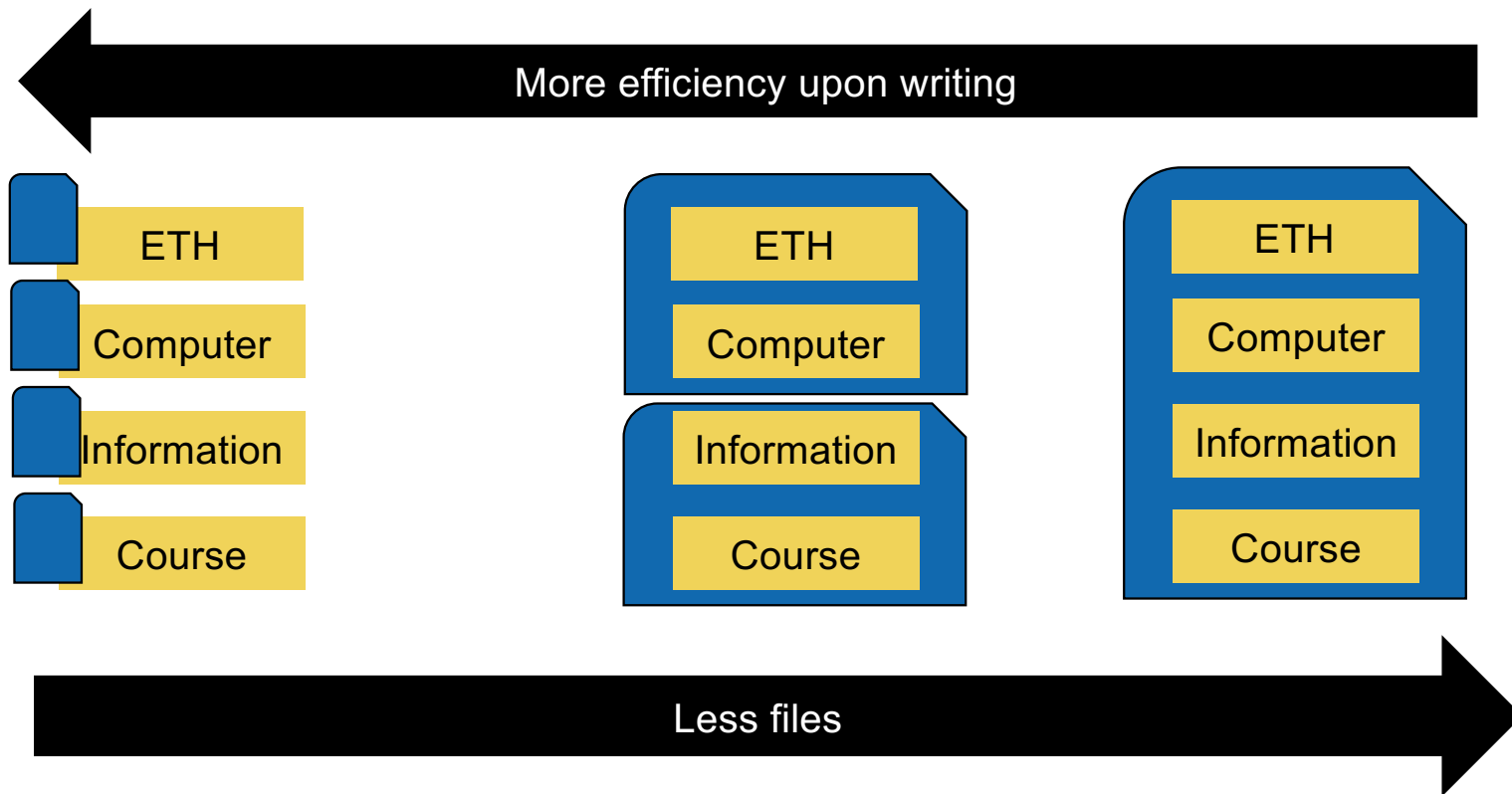
One file per posting list is **not practicable**.



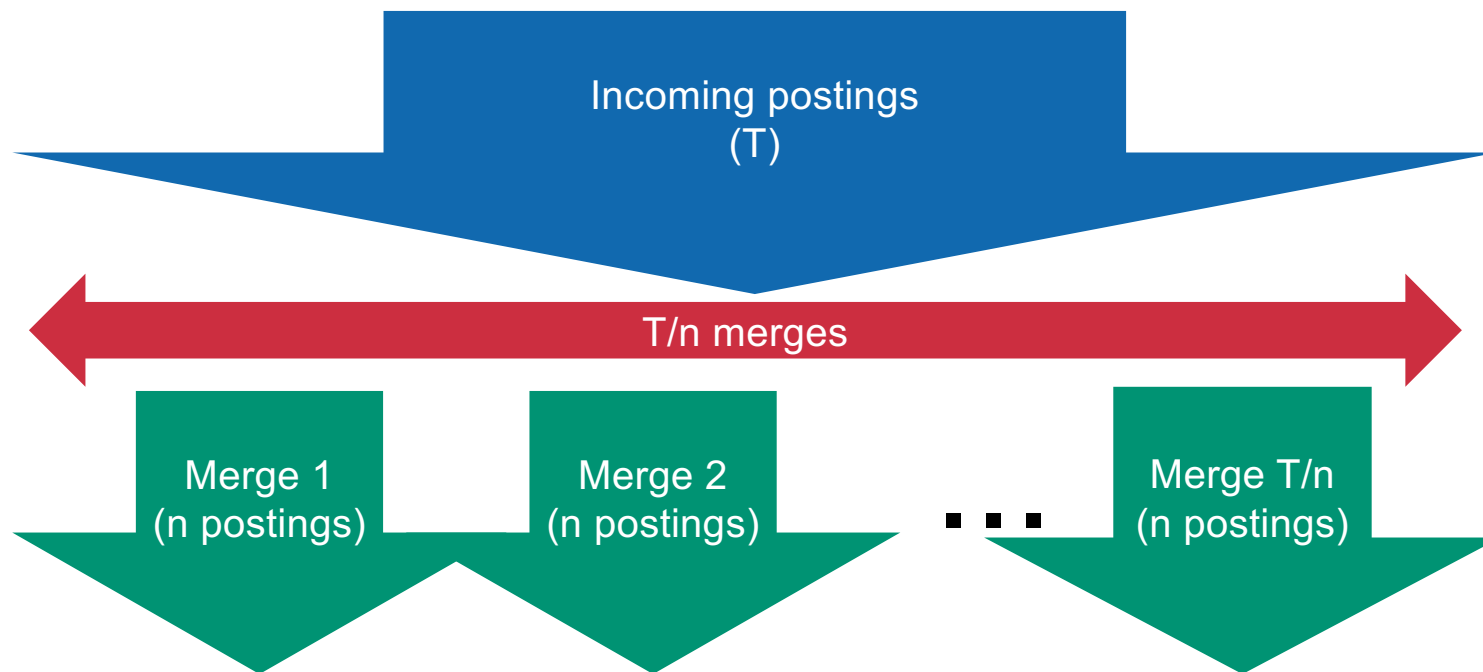
Scaling Issues



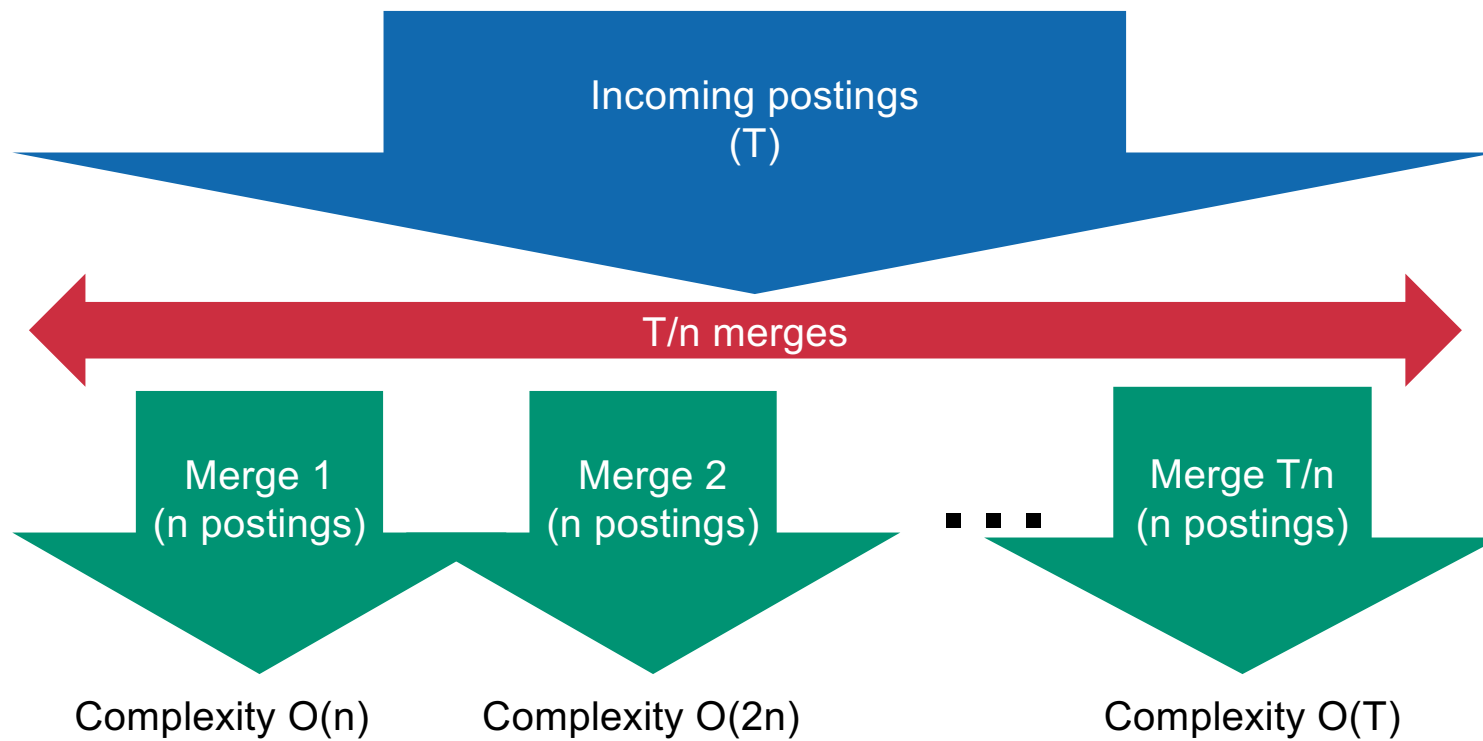
Decision spectrum



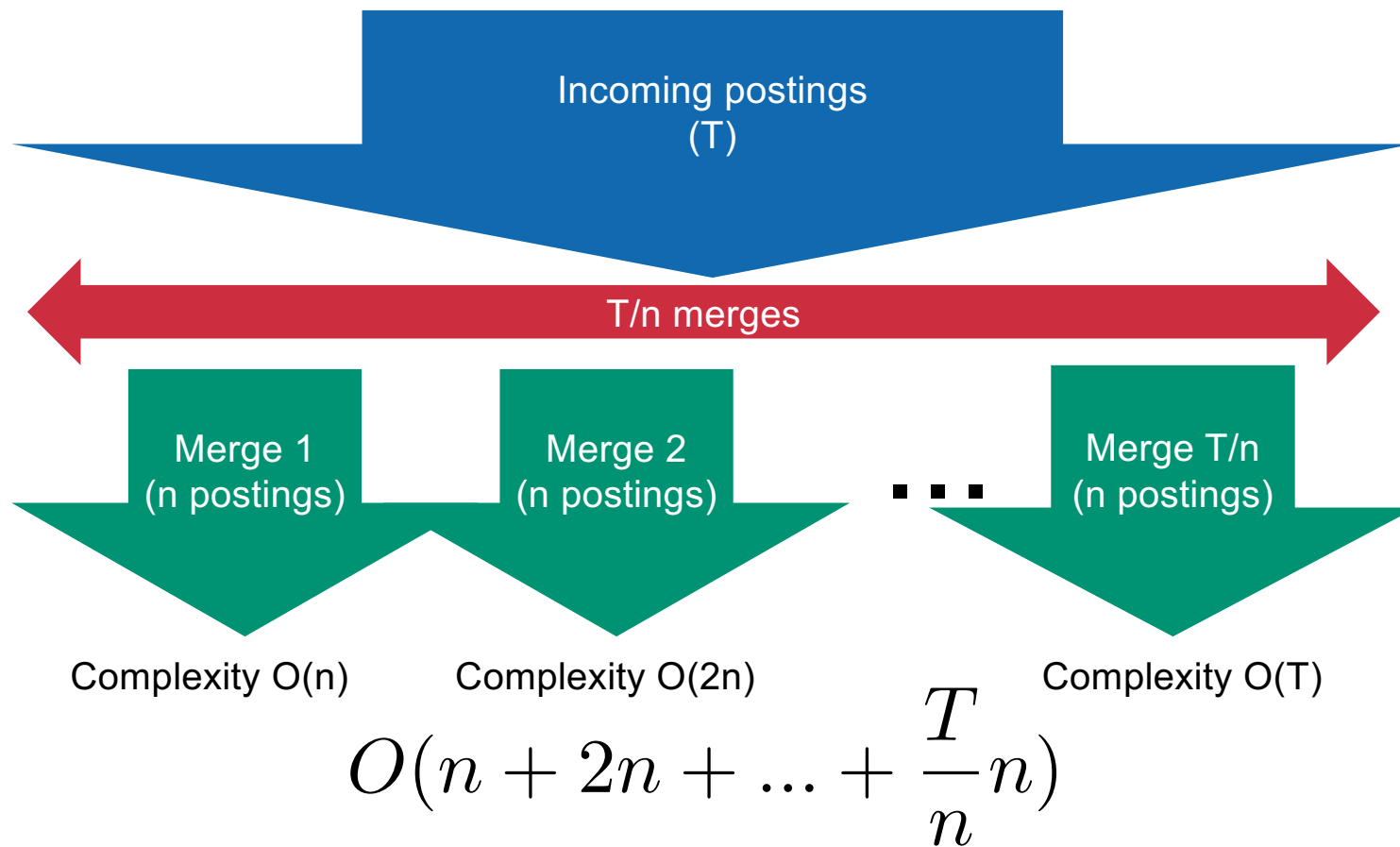
Analysis



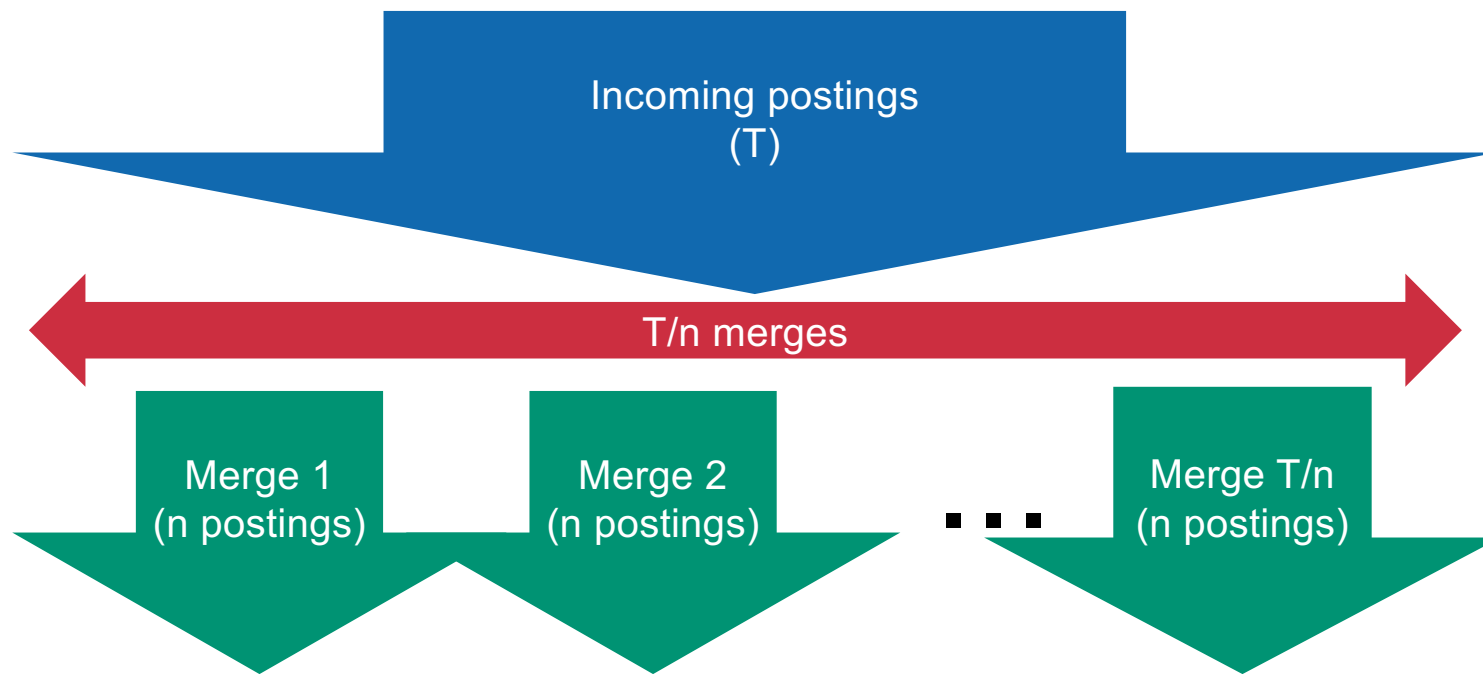
Analysis



Analysis



Analysis



$$O\left(\frac{T^2}{n}\right)$$

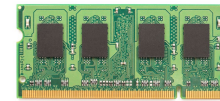
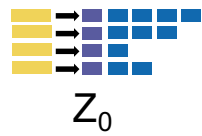
Analysis

$$O\left(\frac{T^2}{n}\right)$$

Can we do better?

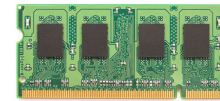
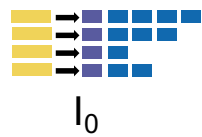
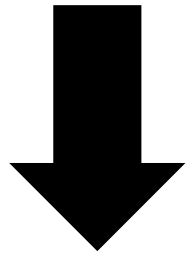
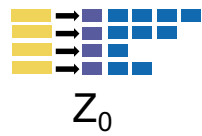
Logarithmic Merging

n postings



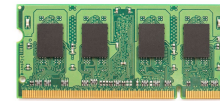
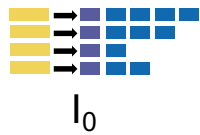
Logarithmic Merging

n postings



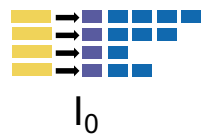
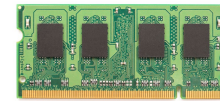
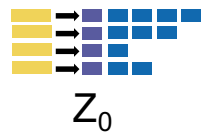
Logarithmic Merging

n postings

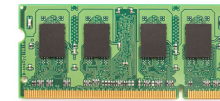
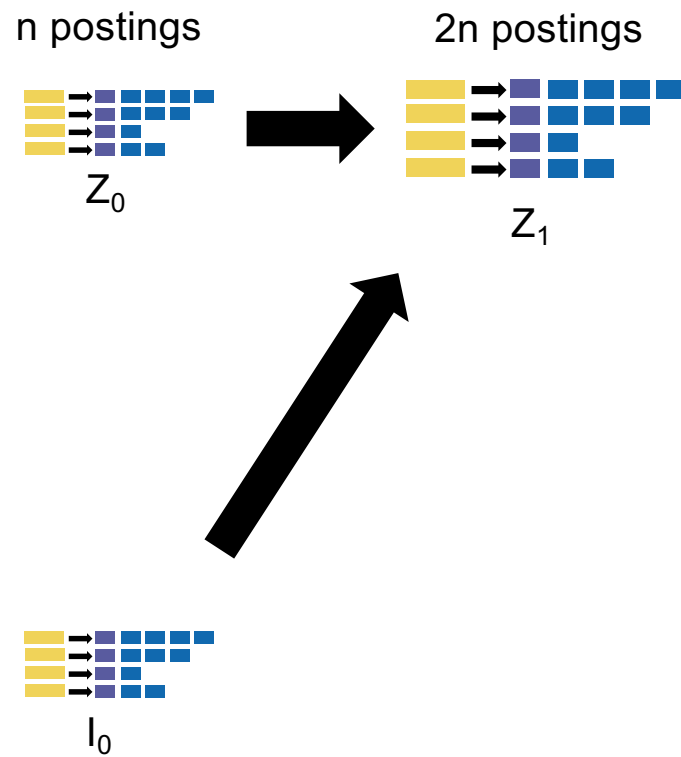


Logarithmic Merging

n postings



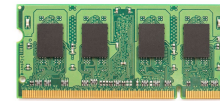
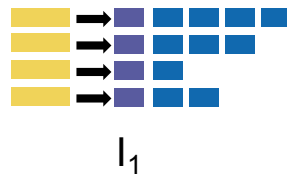
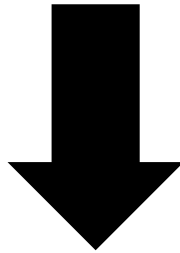
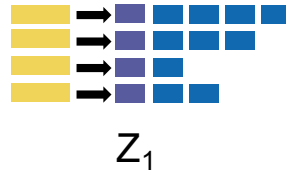
Logarithmic Merging



Logarithmic Merging

n postings

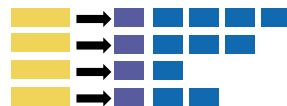
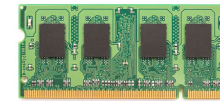
2n postings



Logarithmic Merging

n postings

2n postings

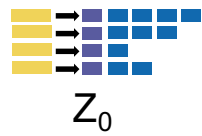


l_1

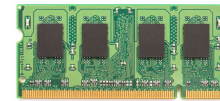
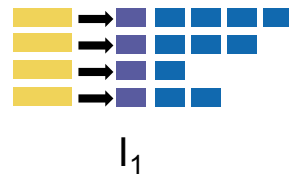


Logarithmic Merging

n postings

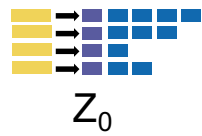


2n postings

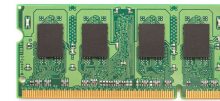
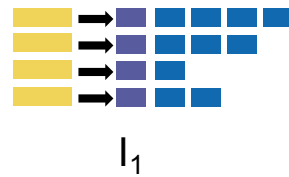
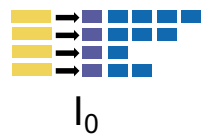
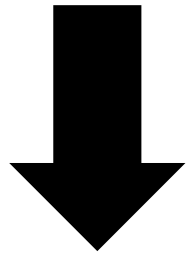


Logarithmic Merging

n postings



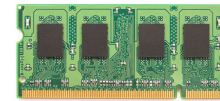
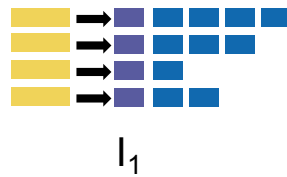
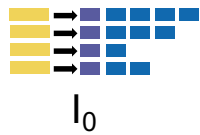
2n postings



Logarithmic Merging

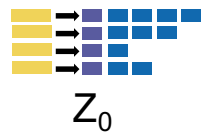
n postings

$2n$ postings

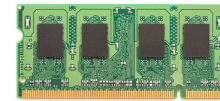
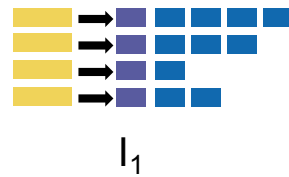
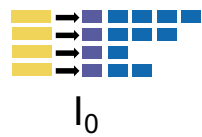


Logarithmic Merging

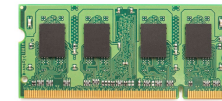
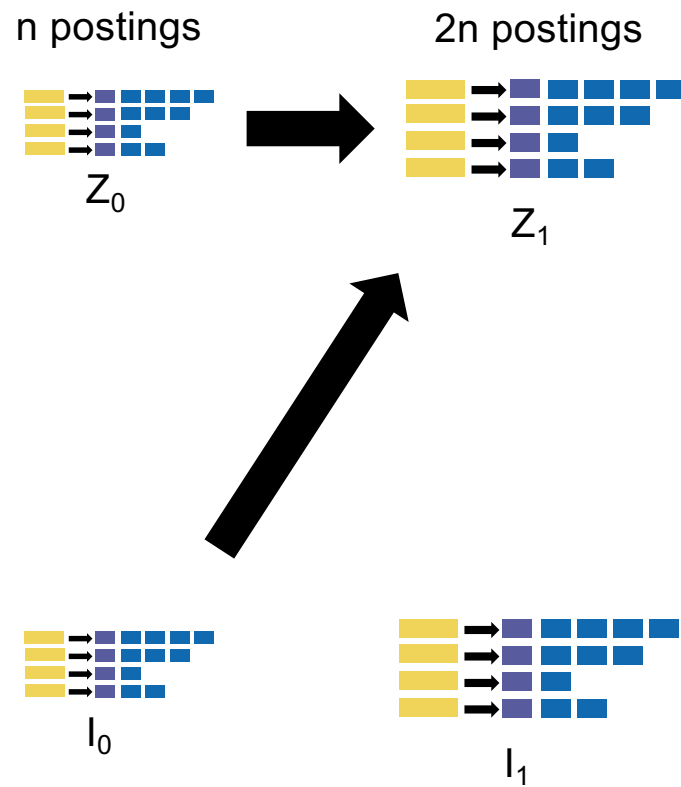
n postings



$2n$ postings



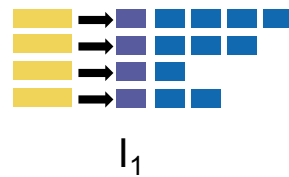
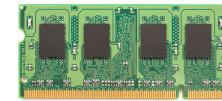
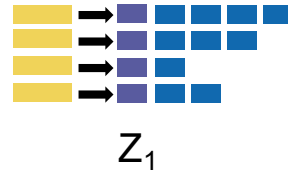
Logarithmic Merging



Logarithmic Merging

n postings

2n postings

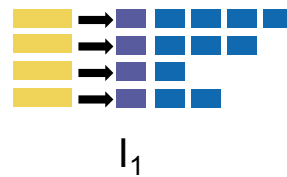
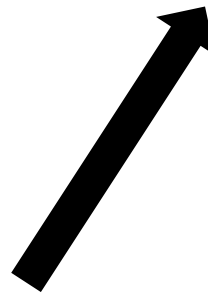
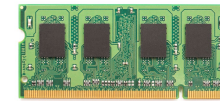
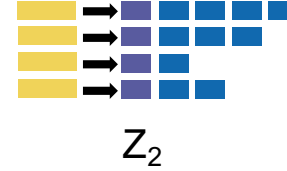
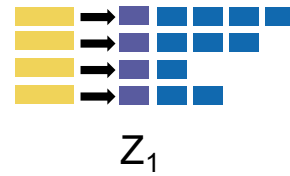


Logarithmic Merging

n postings

$2n$ postings

$4n$ postings

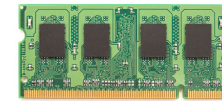
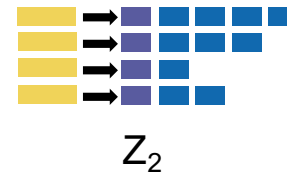


Logarithmic Merging

n postings

2n postings

4n postings

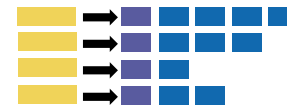


Logarithmic Merging

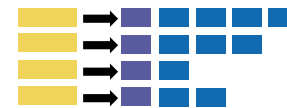
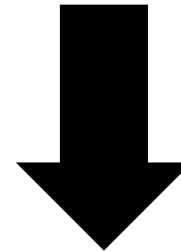
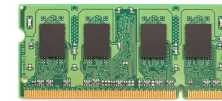
n postings

2n postings

4n postings



Z_2



I_2

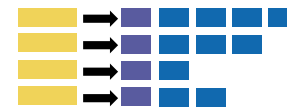
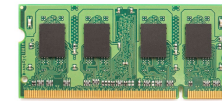


Logarithmic Merging

n postings

2n postings

4n postings

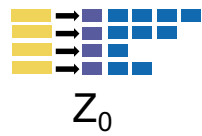


l_2



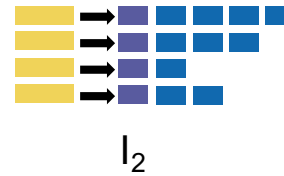
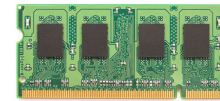
Logarithmic Merging

n postings



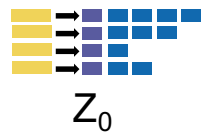
$2n$ postings

$4n$ postings



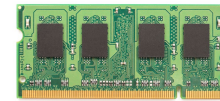
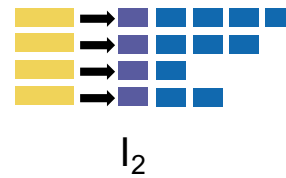
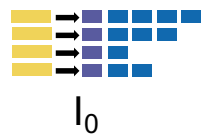
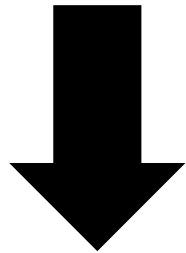
Logarithmic Merging

n postings



$2n$ postings

$4n$ postings

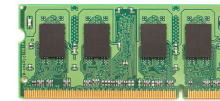
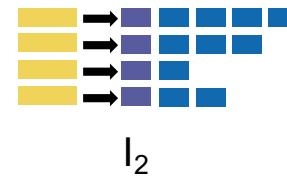
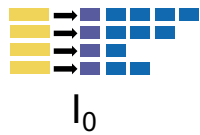


Logarithmic Merging

n postings

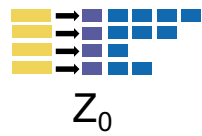
$2n$ postings

$4n$ postings



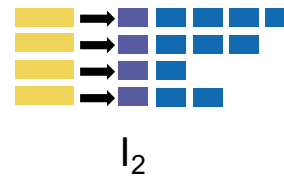
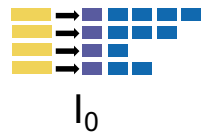
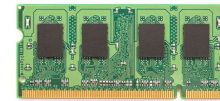
Logarithmic Merging

n postings



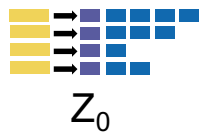
$2n$ postings

$4n$ postings



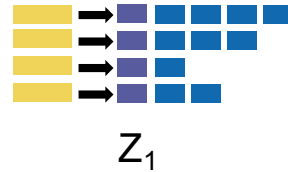
Logarithmic Merging

n postings



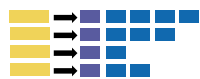
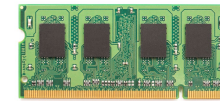
Z_0

$2n$ postings

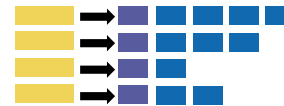


Z_1

$4n$ postings



I_0



I_2

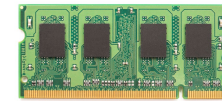
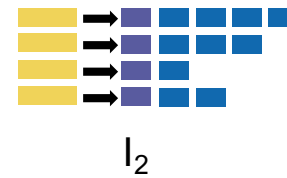
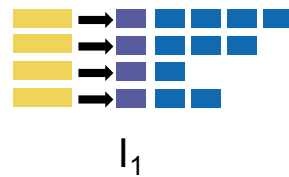
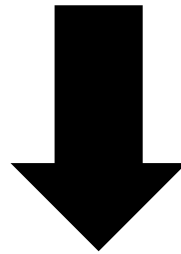
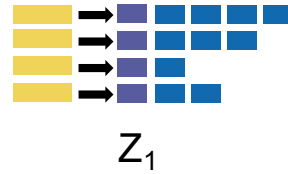


Logarithmic Merging

n postings

$2n$ postings

$4n$ postings

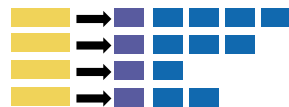


Logarithmic Merging

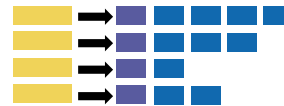
n postings

$2n$ postings

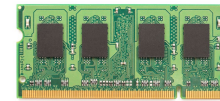
$4n$ postings



l_1

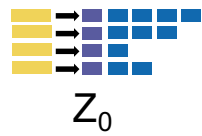


l_2

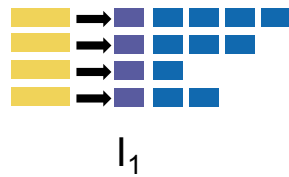


Logarithmic Merging

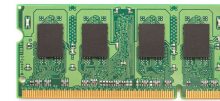
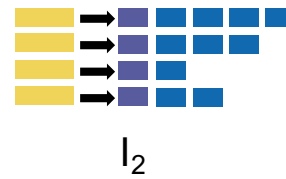
n postings



$2n$ postings

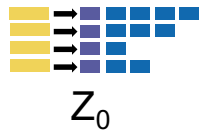


$4n$ postings

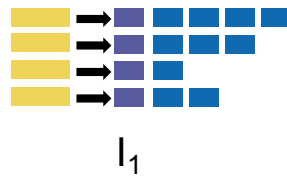


Logarithmic Merging

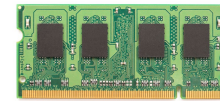
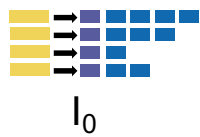
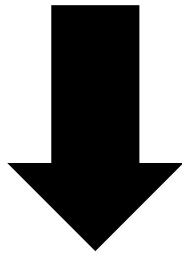
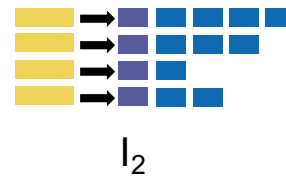
n postings



$2n$ postings



$4n$ postings

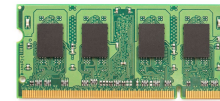
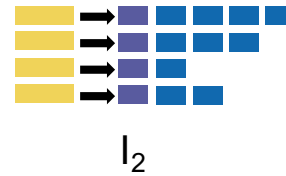
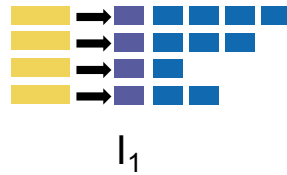
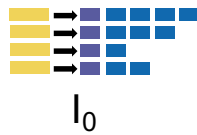


Logarithmic Merging

n postings

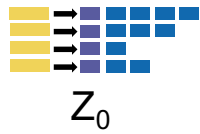
$2n$ postings

$4n$ postings

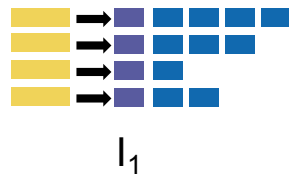


Logarithmic Merging

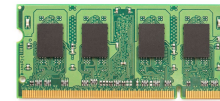
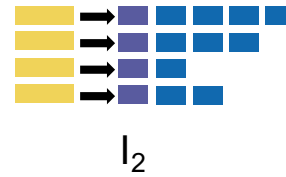
n postings



$2n$ postings

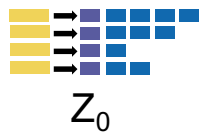


$4n$ postings

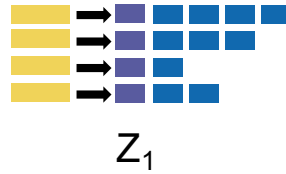


Logarithmic Merging

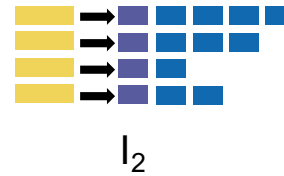
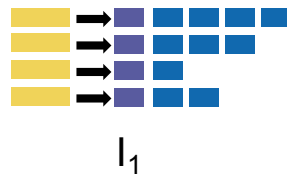
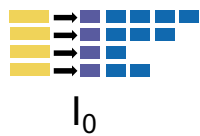
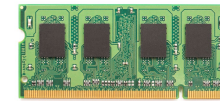
n postings



$2n$ postings



$4n$ postings

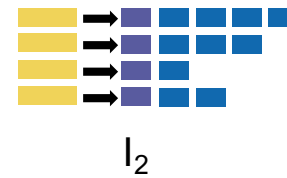
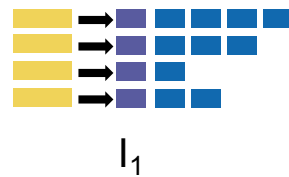
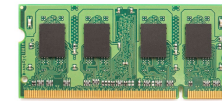
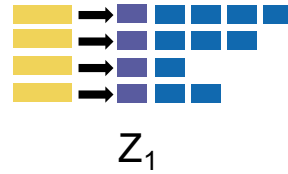


Logarithmic Merging

n postings

$2n$ postings

$4n$ postings

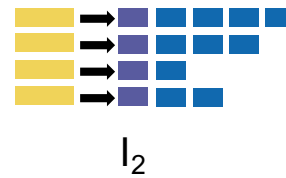
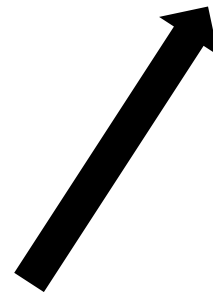
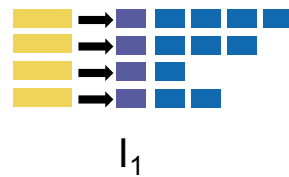
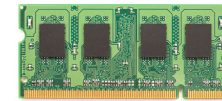
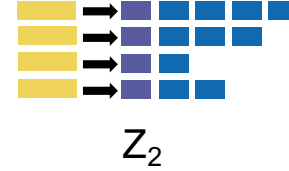
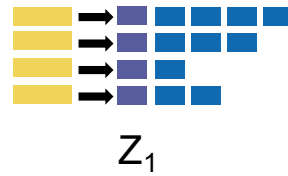


Logarithmic Merging

n postings

$2n$ postings

$4n$ postings

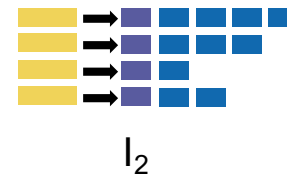
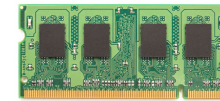
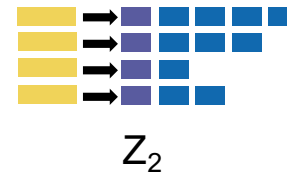


Logarithmic Merging

n postings

2n postings

4n postings

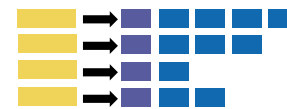


Logarithmic Merging

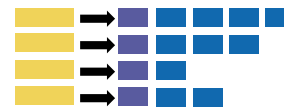
n postings

2n postings

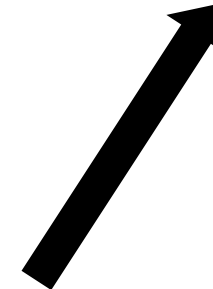
4n postings



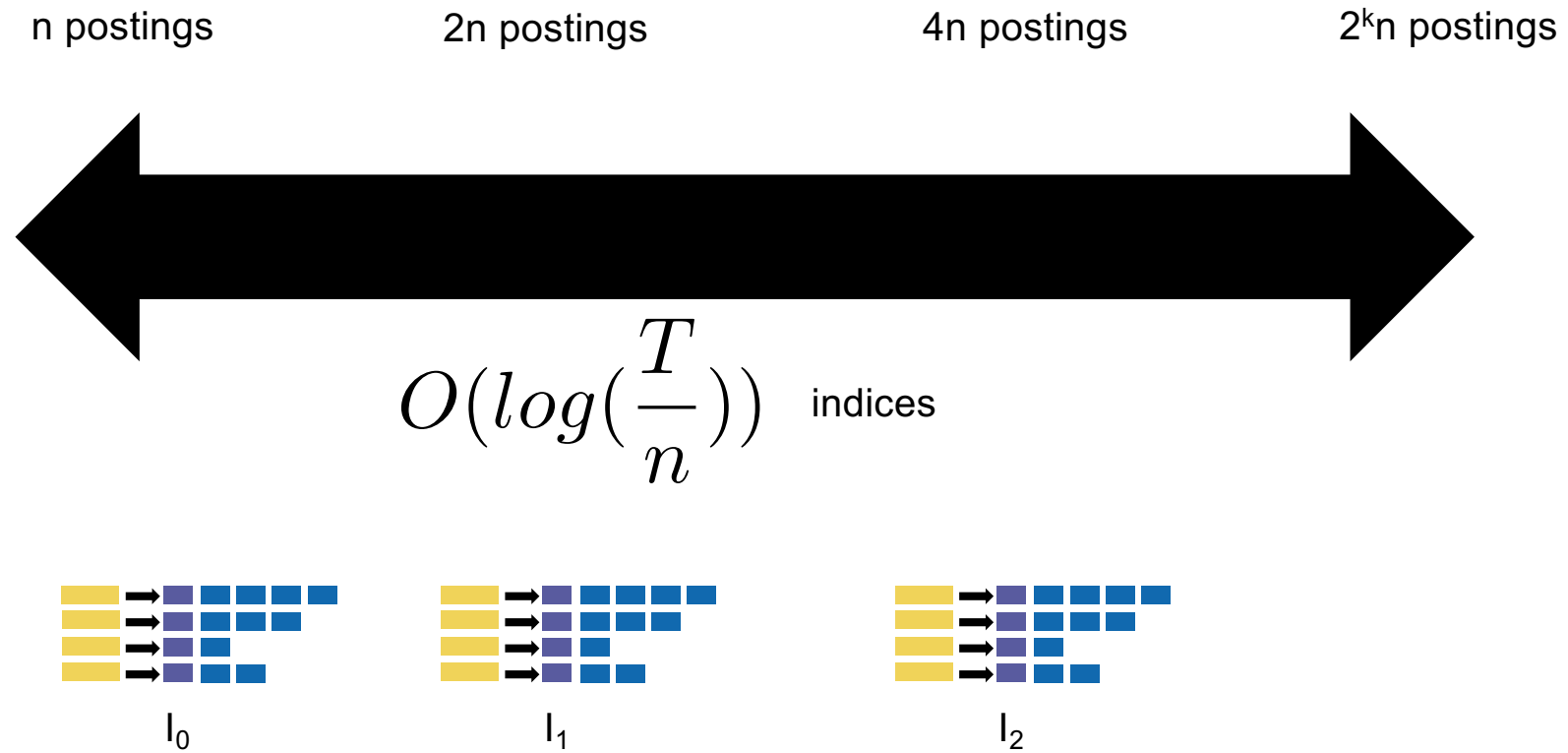
Z_2



I_2



Logarithmic Merging



Logarithmic Merging

Construction time

$$O(T \log(\frac{T}{n}))$$

Impact on querying

Query time

$$O(\log(\frac{T}{n}))$$

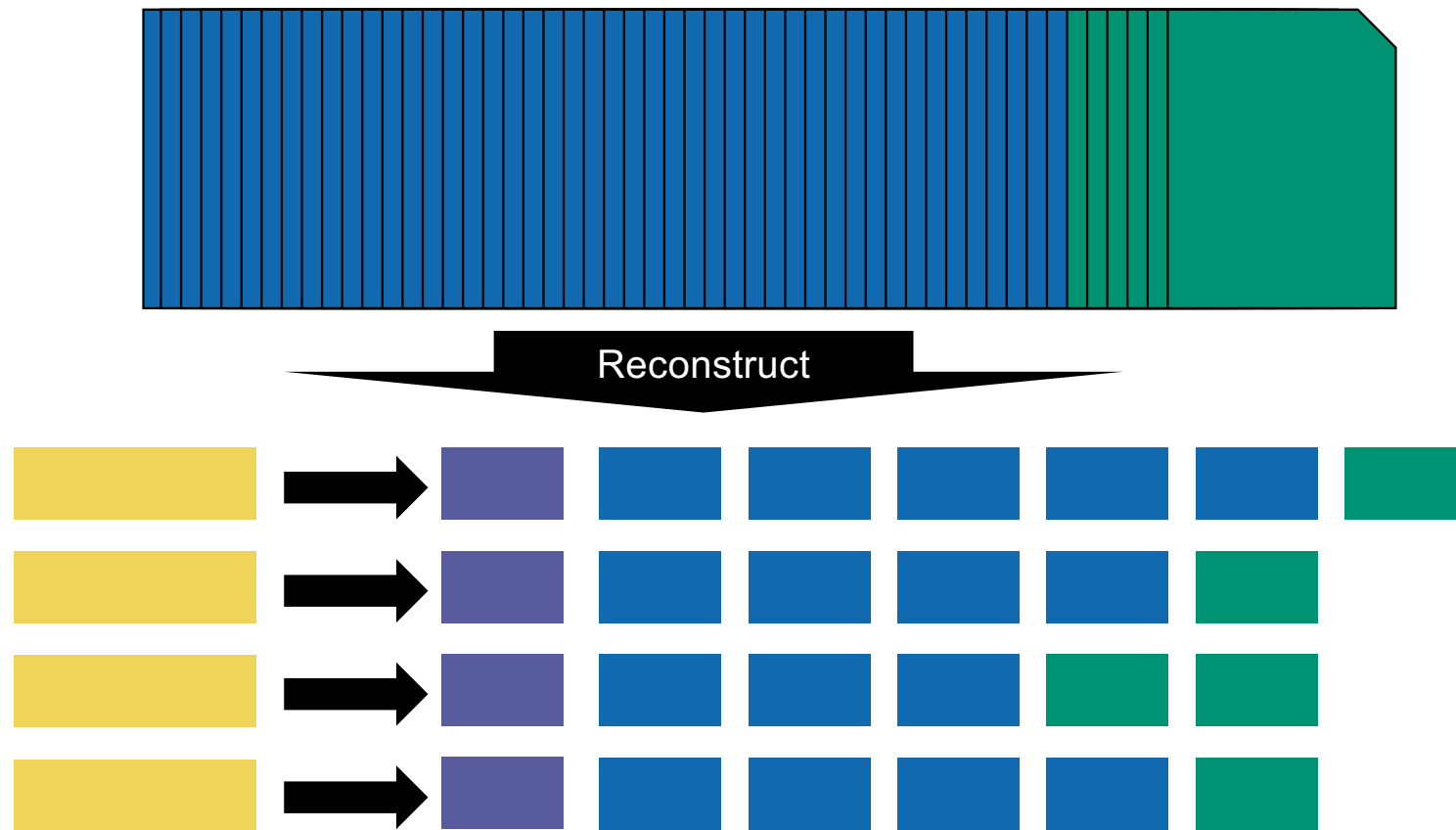
Compromise



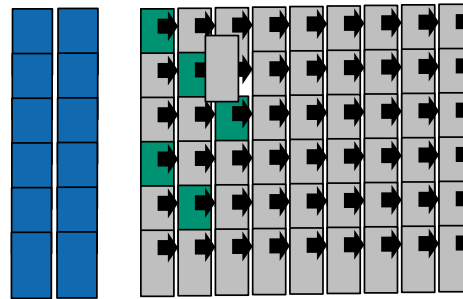
Index construction time

Querying time

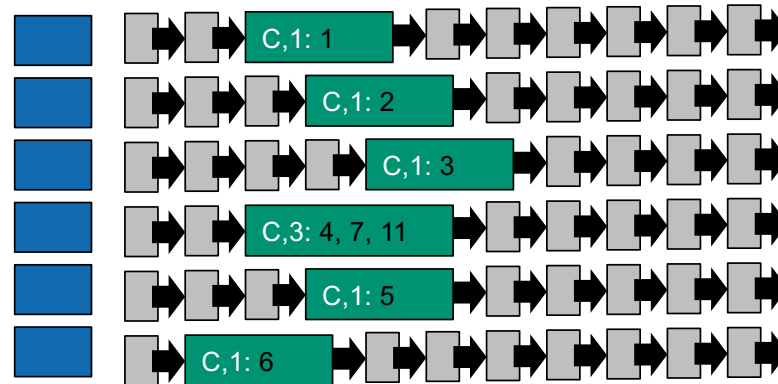
In practice, we use index reconstruction

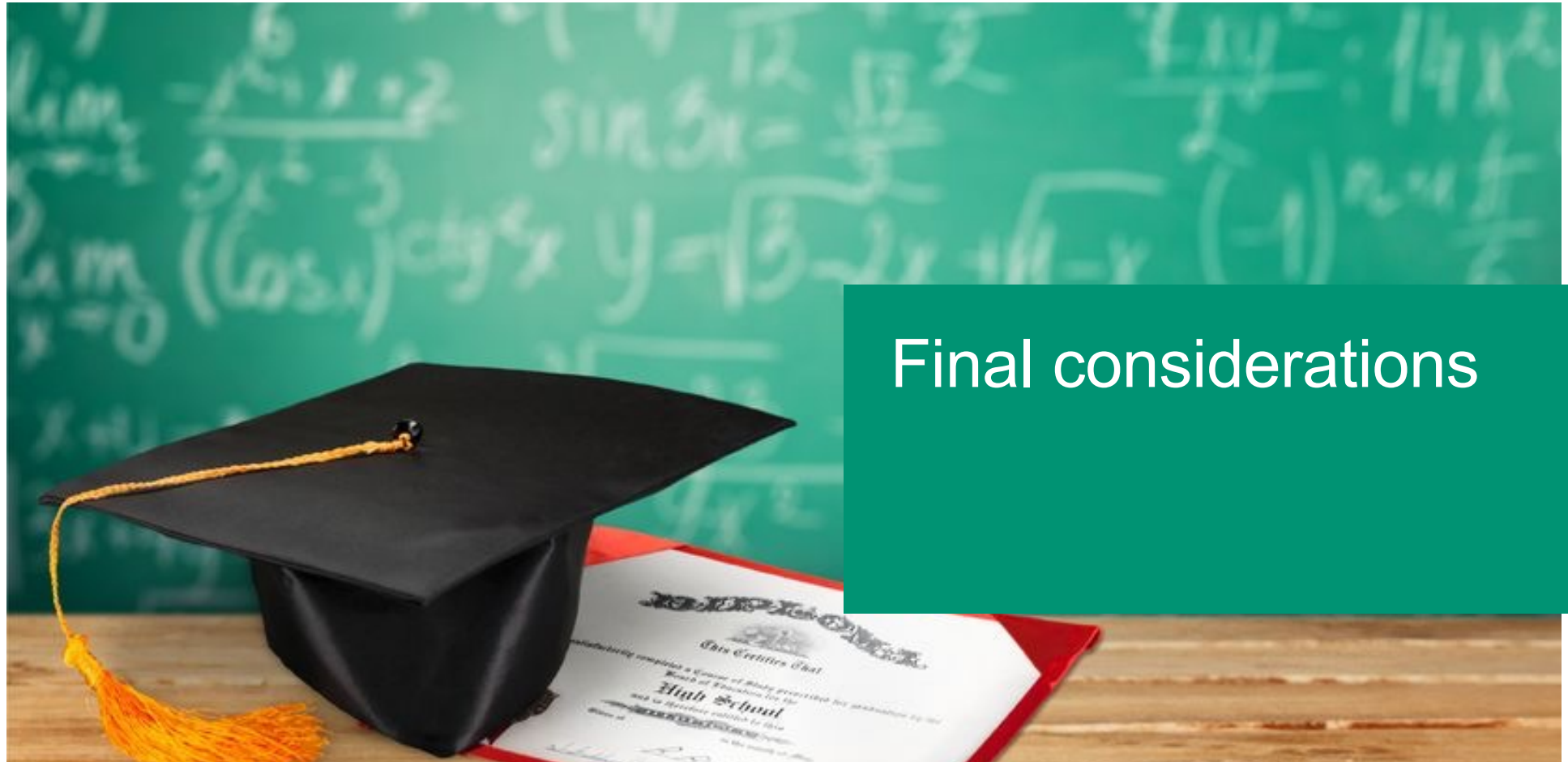


Other indexes



Other indices (positional, biword, etc)
can be easily adapted.



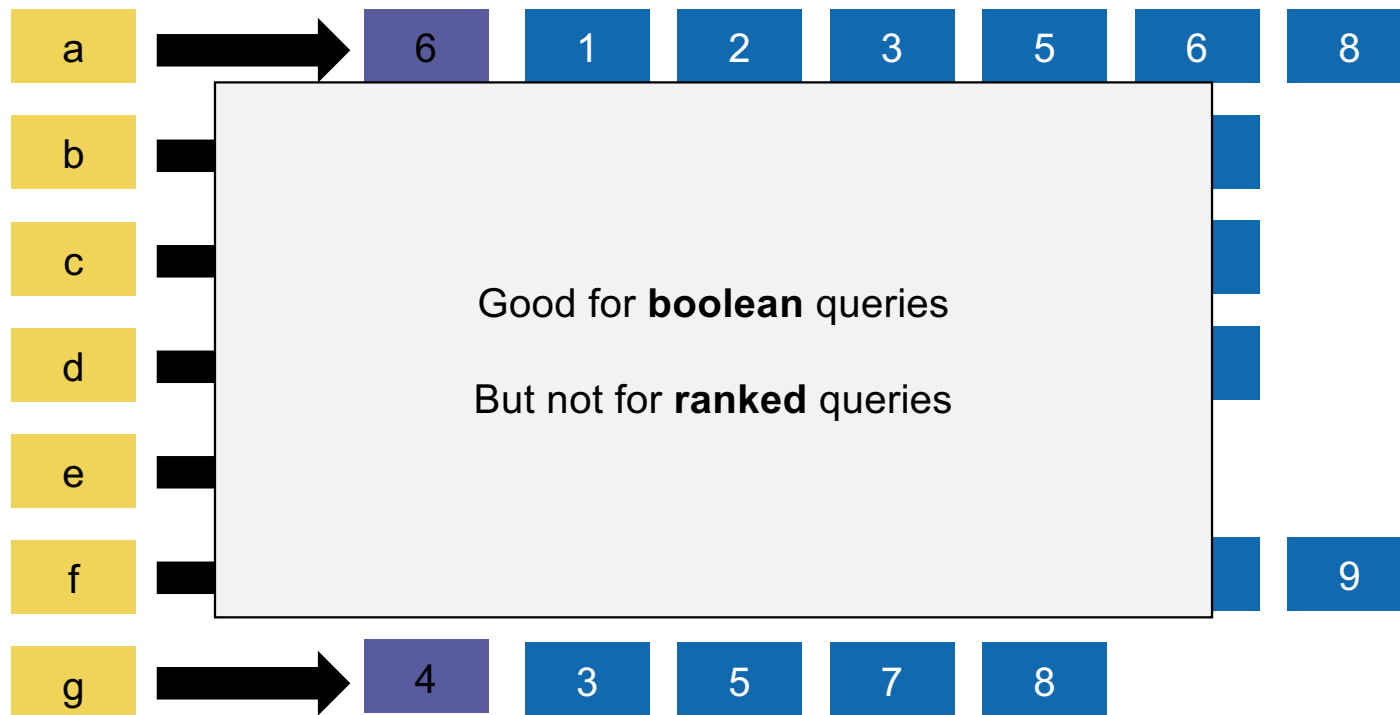


Final considerations

Sorted documents

a	→	6	1	2	3	5	6	8
b	→	5	3	4	7	8	9	
c	→	5	1	2	4	5	7	
d	→	5	1	3	5	8	9	
e	→	4	2	3	4	7		
f	→	6	1	2	4	5	8	9
g	→	4	3	5	7	8		

Sorted documents

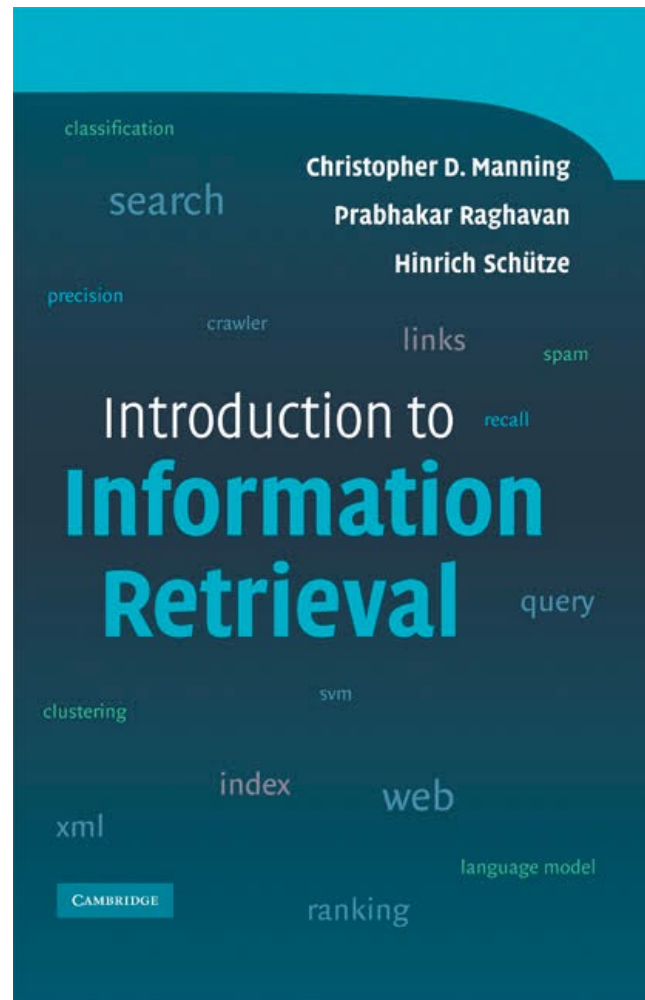


Security



Access-Control Lists

Credits



This week:

Chapter 4

Index Construction